

Natural Language Processing and Understanding

Exam-Ready Revision Notes

Approximately 100 pages of explanations, formulas, comparisons, traps, and answer frames

Every syllabus topic retained

109 exact topic IDs across 14 teaching weeks

Tier A = high-yield core concept; Tier B = supporting/application concept; Tier C = introduction, setup, or recap.

Build date: 8 August 2026

How to Use This Exam Edition

Read the definition first, then reproduce the core explanation without looking. For calculation or architecture questions, write the formula or flow before the prose. Use the red exam-trap line to protect marks, and finish each week with its answer frame. The exact topic badge is also the coverage key used by the audit report.

Coverage and priority. All 109 source lessons are present: Tier A 82, Tier B 22, Tier C 5. Condensation changes depth, not syllabus coverage. Definitions, mechanisms, applicable formulas, examples or contrasts, quick-revision facts, and pitfalls are selected from the edited course notes.

Mark-wise method. For 2 marks: definition + one distinction. For 5 marks: definition + mechanism + example + limitation. For 10 marks: add a diagram or formula, compare alternatives, state metrics, and justify the final decision.

Week Roadmap

Week	Focus	Coverage
Week 0	Course Introduction	2 topics
Week 1	NLP Foundations and Necessary Linguistics	6 topics
Week 2	Text Preprocessing and Cleaning	7 topics
Week 3	POS Tagging and Named Entity Recognition	11 topics
Week 4	Text Corpora and Corpus Linguistics	7 topics
Week 5	Traditional Word Representations	10 topics
Week 6	Modern Word Embeddings	7 topics
Week 7	Sequential Modeling and Recurrent Networks	7 topics
Week 8	The Transformer Revolution	6 topics
Week 9	Natural Language Understanding with BERT	4 topics
Week 10	Text Classification and Sentiment Analysis	7 topics
Week 11	Topic Modeling	8 topics
Week 12	Natural Language Generation with LLMs	14 topics
Week 13	Capstone: QuizGenius AI	13 topics

Contents

How to Use This Exam Edition	2
Week 0: Course Introduction	4
Week 1: NLP Foundations and Necessary Linguistics	7
Week 2: Text Preprocessing and Cleaning	13
Week 3: POS Tagging and Named Entity Recognition	19
Week 4: Text Corpora and Corpus Linguistics	26
Week 5: Traditional Word Representations	32
Week 6: Modern Word Embeddings	40
Week 7: Sequential Modeling and Recurrent Networks	47
Week 8: The Transformer Revolution	53
Week 9: Natural Language Understanding with BERT	59
Week 10: Text Classification and Sentiment Analysis	64

Week 11: Topic Modeling	71
Week 12: Natural Language Generation with LLMs	78
Week 13: Capstone: QuizGenius AI	89
Rapid Revision Master Sheet	98

Week 0: Course Introduction

Week roadmap: Natural Language Processing and Understanding: Course Overview → Practitioner-Led NLP Education: What to Expect.

Coverage: 2 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W00-T01 Natural Language Processing and Understanding: Course Overview Tier C

Definition and why it matters: Machines that read, interpret, and generate human language are no longer research curiosities - they are production infrastructure. Search engines rank billions of documents, translation systems bridge languages in milliseconds, keyboards predict the next word, and large language models draft code, summarise reports, and power conversational agents

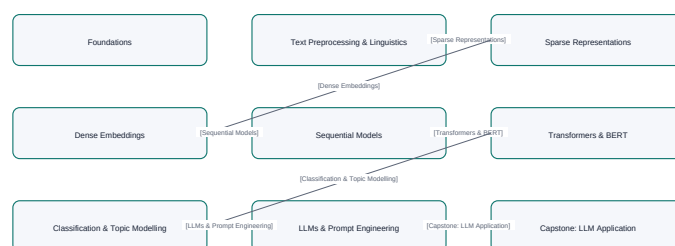
Core explanation: Natural Language Processing (NLP) sits at the centre of this shift. It is the discipline that turns unstructured text into actionable signals and fluent output. For postgraduate ML practitioners, NLP literacy is among the highest-leverage skills in modern AI engineering

Must remember:

- NLP is the interface layer of modern AI - search, translation, sentiment, LLMs
- Course arc: foundations → sparse/dense representations → RNNs/LSTMs → transformers/BERT → applications → LLM capstone
- Sparse: one-hot, BoW, TF-IDF; Dense: Word2Vec, GloVe; Contextual: BERT, LLMs
- Sequential models (RNN, LSTM, GRU) precede transformers; vanishing gradients motivate gated architectures
- Hands-on: semantic search, sentiment classifier, LLM quiz application
- Goal: understand both the why behind language AI and the how of implementation

Exam trap: Treating NLP as only "ChatGPT usage" - classical preprocessing and representations remain foundational and examinable Skipping early modules to jump to transformers - attention mechanisms assume understanding of sequences, embeddings, and task framing

Natural Language Processing and Understanding: Course Overview



Vector reconstruction of the source Mernail visual

Concept map: Natural Language Processing and Understanding: Course Overview

NLP-W00-T02 Practitioner-Led NLP Education: What to Expect Tier A

Definition and why it matters: Academic NLP curricula often emphasise theory and benchmark datasets. Industry NLP work demands something additional: shipping pipelines that handle messy production text, integrating pretrained models with business APIs, and iterating under latency and cost constraints. A course led by an experienced AI engineer bridges that gap - grounding concepts in patterns seen across real product deployments This note frames what practitioner-led instruction typically emphasises and how students should engage to maximise transfer to professional work

Core explanation: Experienced NLP practitioners typically work across multiple domains: Concept flow: NLP Engineering - Product / SaaS - Finance - Healthcare - E-commerce - Education Tech Each domain imposes different constraints: Finance - compliance, audit trails, low tolerance for hallucination Healthcare - specialised vocabulary, HIPAA-style privacy, high precision for NER Product/SaaS - scale, multilingual support, A/B

testing of model versions Practice: Implement, don't only read; Rationale: NLP intuition comes from seeing tokenisation edge cases and embedding failures firsthand E-commerce - real-time sentiment, search relevance, personalisation Practice: Compare approaches; Rationale: Week 10's NLTK vs BERT vs Flair vs spaCy comparison mirrors real tool-selection decisions Exposure to varied industries teaches transferable patterns (preprocessing pipelines, evaluation discipline) while highlighting where domain adaptation is non-negotiable Practice: Build the capstone end-to-end; Rationale: QuizGenius exercises prompt design, API integration, and architecture - skills directly transferable to LLM product work Effective practitioner instruction often includes: Practice: Document trade-offs; Rationale: For each technique, note accuracy, speed, cost, and interpretability - the dimensions practitioners optimise Worked examples from production-adjacent scenarios, not only textbook datasets Tool literacy - NLTK, spaCy, Flair, Hugging Face, LLM APIs - with guidance on when each fits Concept flow: Understand language fundamentals - Build preprocessing pipelines - Train/evaluate classical and neural models - Leverage pretrained transformers - Deploy LLM-powered applications Debugging mindset - why a sentiment model fails on sarcasm, why LDA topics are incoherent, why prompts drift Career-relevant framing - how NLP skills compose with MLOps, data engineering, and full-stack delivery Students benefit by treating labs as portfolio artefacts, not checkbox exercises The course trajectory aligns with a common professional path: Practitioner-led delivery accelerates the final steps - students see how pretrained models and APIs compress time-to-value while still requiring solid fundamentals for debugging and customisation

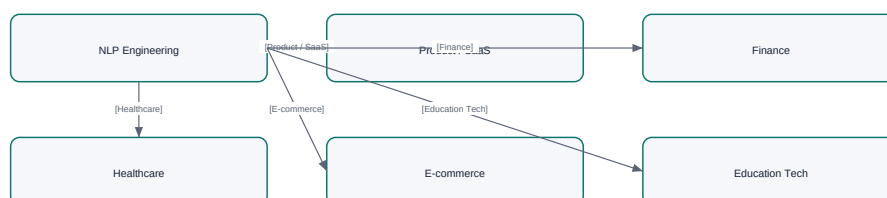
Must remember:

- Practitioner-led NLP education emphasises production trade-offs alongside theory
- Industry work spans domains with different compliance, vocabulary, and latency needs
- Mentorship adds worked examples, tool selection guidance, and debugging mindset
- Engage by implementing, comparing approaches, and building the capstone as a portfolio piece
- Course path mirrors professional growth: fundamentals → pipelines → models → transformers → LLM deployment
- Balance practitioner insights with exam-focused conceptual mastery

Example or contrast: Dimension: Data; Academic Focus: Clean benchmark corpora (IMDB, CoNLL); Practitioner Focus: Noisy user-generated text, domain jargon, PII Dimension: Success metric; Academic Focus: Accuracy, F1, BLEU on test sets; Practitioner Focus: Latency, cost per request, maintainability, user satisfaction Dimension: Model choice; Academic Focus: State-of-the-art novelty; Practitioner Focus: Fit-for-purpose: VADER vs BERT vs LLM by constraint

Exam trap: Assuming industry experience means skipping theory - exams test conceptual understanding (LSTM gates, attention, LDA assumptions), not just API usage Believing one tool wins everywhere - practitioners constantly trade off VADER speed vs BERT accuracy vs LLM flexibility

Practitioner-Led NLP Education: What to Expect



Vector reconstruction of the source Mermaid visual

Concept map: Practitioner-Led NLP Education: What to Expect

Week 0 Exam Lens

Likely questions

- Define and explain Practitioner-Led NLP Education: What to Expect; include the mechanism and one limitation.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Mistakes to avoid: Treating NLP as only "ChatGPT usage" - classical preprocessing and representations remain foundational and examinable Assuming industry experience means skipping theory - exams test conceptual understanding (LSTM gates, attention, LDA assumptions), not just API usage

Week 1: NLP Foundations and Necessary Linguistics

Week roadmap: Introduction to Natural Language Processing → What Is Natural Language Processing? → NLP, NLU, and NLG: Understanding the Relationship → Applications of Natural Language Processing by Domain → Morphology of Text: Stems, Roots, and Affixes → Ambiguity in Language: Polysemy and Synonymy.

Coverage: 6 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W01-T01 Introduction to Natural Language Processing

Tier B

Definition and why it matters: How do machines work with human language? Text and speech are everywhere - search queries, support tickets, clinical notes, social posts - yet computers natively operate on numbers, not words. Natural Language Processing (NLP) is the discipline that bridges that gap: it gives machines the ability to read, interpret, and respond in human language

Core explanation: This opening module establishes the vocabulary, scope, and linguistic foundations needed for every technique that follows - from tokenisation and tagging to transformers and large language models
 Pillar: Core NLP concepts; Focus: What NLP is, what problems it solves, and where it appears in production systems
 Pillar: NLP, NLU, and NLG; Focus: How understanding and generation relate under the NLP umbrella
 Pillar: Linguistic foundations; Focus: Morphology, ambiguity, polysemy, and synonymy - why language is hard for machines
 Concept flow: What is NLP? - NLP / NLU / NLG - Applications by Domain - Morphology - Ambiguity & Word Relations - Week 2+: Preprocessing & Models
 The module is organised around three pillars:
 Unlike structured database fields, natural language is: Unstructured - no fixed schema; meaning emerges from word order, punctuation, and context
 Ambiguous - the same surface form can carry multiple interpretations
 Context-dependent - word meaning shifts with domain, culture, and surrounding text
 NLP systems must therefore handle noise, variation, and implicit meaning - not just pattern-match strings
 Define NLP and its high-level goals (read, extract, generate)
 Distinguish NLU (understanding) from NLG (generation)
 Survey real-world applications across search, healthcare, finance, and education
 Introduce morphology - how words are built from roots and affixes
 Examine ambiguity, polysemy, and synonymy - core reasons embedding-based methods eventually replace naive dictionary lookups

Must remember:

- NLP enables machines to understand, process, and interact using human language
- Language is unstructured, ambiguous, and context-dependent - unlike typical tabular data
- This module covers NLP scope, NLU/NLG relationship, applications, morphology, and ambiguity
- Morphology studies word structure; it motivates preprocessing and vocabulary management
- Polysemy and synonymy explain why context-aware representations (embeddings) are essential
- Foundation concepts here underpin every later week: preprocessing, tagging, embeddings, transformers, and LLMs

Exam trap: Treating NLP as only "chatbots" - it spans search ranking, fraud monitoring, clinical summarisation, and more
 Assuming NLP = NLG - NLU (classification, NER, sentiment) is equally central and often the upstream step

NLP-W01-T02 What Is Natural Language Processing?

Tier B

Definition and why it matters: Every day, billions of people type queries, send messages, and speak to devices. Computers, however, only manipulate bits and floating-point numbers. Natural Language Processing (NLP) is the field of artificial intelligence devoted to closing that representational gap - enabling machines to understand, process, and interact using human language as humans do
 NLP sits at the intersection of computer science, artificial intelligence, and linguistics. Its goal is not merely to store text, but to extract meaning and act on it

Core explanation: Human language carries properties that make naive string processing insufficient: Property: Unstructured; Meaning: No rigid schema; free-form sentences; Machine Impact: Requires parsing, segmentation, and feature extraction
 Property: Ambiguous; Meaning: Multiple valid interpretations per utterance;

Machine Impact: Context and world knowledge needed to disambiguate Property: Context-dependent; Meaning: Meaning shifts with speaker, domain, prior sentences; Machine Impact: Models must track discourse and situational cues Concept flow: Human Language - Unstructured - Ambiguous - Context-dependent - Machine Challenge - NLP Pipeline Required These properties explain why NLP pipelines are multi-stage (tokenise → tag → embed → classify/generate) rather than single-regex solutions At the highest level, NLP systems pursue three capabilities: Read and interpret - assign structure and meaning to raw text Extract information - pull entities, intents, sentiments, and facts into machine-usable form Generate responses - produce human-readable output (summaries, answers, translations) These map to downstream tasks: search ranking, spam detection, dialogue, summarisation, and machine translation

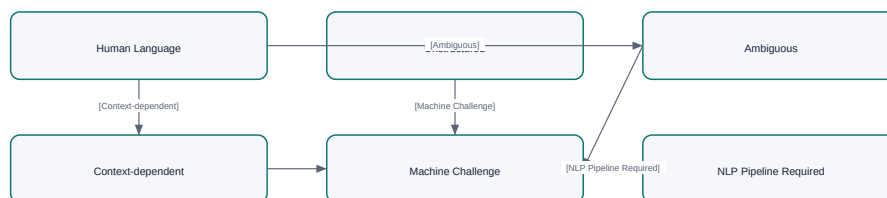
Must remember:

- NLP = AI field enabling machines to understand, process, and generate human language
- It spans computer science, AI, and linguistics
- Language is unstructured, ambiguous, and context-dependent - the root difficulty
- High-level goals: read/interpret, extract information, generate responses
- Real examples: search engines, voice assistants, spam filters, autocomplete, LLM chat apps
- NLP bridges human communication and machine computation

Example or contrast: Domain: Search; Example System: Google, Bing; NLP Task: Query understanding, document ranking Domain: Voice; Example System: Siri, Alexa; NLP Task: Speech recognition + language understanding Domain: Email; Example System: Gmail spam filter; NLP Task: Text classification (spam vs not spam)

Exam trap: Defining NLP as only "text generation" - understanding and extraction are equally core Ignoring ambiguity when designing systems - rule-only approaches fail on real corpora

What Is Natural Language Processing?



Vector reconstruction of the source Mermaid visual

Concept map: What Is Natural Language Processing?

NLP-W01-T03 NLP, NLU, and NLG: Understanding the Relationship

Tier A

Definition and why it matters: Not every language task asks the same question. Sometimes the machine must figure out what a human meant; other times it must say something useful back. These two directions - comprehension and production - are the twin engines of practical NLP systems Natural Language Processing (NLP) is the umbrella. Under it sit Natural Language Understanding (NLU) and Natural Language Generation (NLG). Most production applications use both, often in sequence

Core explanation: Concept flow: Natural Language Processing - Natural Language Understanding - Natural Language Generation - What does the text mean? - How should the machine express information? Term: NLP; Core Question: How do we process human language computationally?; Direction: Umbrella - includes both understanding and generation Term: NLU; Core Question: What does the input text mean?; Direction: Input - structured meaning Term: NLG; Core Question: How should the machine express information?; Direction: Structured data - human-readable text NLU focuses on comprehending text: extracting intent, entities, sentiment, and other semantic signals Primary concern: What does the text mean? Sentiment analysis - classify opinion polarity (positive, negative, neutral) Named Entity Recognition (NER) - locate and type entities (person, organisation, date) Text classification - assign documents or utterances to predefined categories Intent detection - map user utterances to actions in dialogue systems NLU transforms unstructured language into

machine-actionable representations - labels, entity spans, embedding vectors, or structured slots

Concept flow: User - NLU - Backend - NLG - date NLG focuses on producing human-like language from data or internal representations

Stage: Input; Component: NLU; Role: Parse intent and slots from user text

Primary concern: How can a machine communicate information clearly in human language?

Stage: Reasoning; Component: Backend / KB; Role: Execute business logic, retrieve data

Text summarisation - condense long documents into concise summaries

Stage: Output; Component: NLG; Role: Render results as natural language

Chatbots and question answering - formulate natural responses to user queries

Report generation - turn analytics or database records into narrative prose

Machine translation - render content in another human language

NLG transforms structured or semi-structured input into fluent, readable output

Real-world systems rarely stop at understanding or generation alone

Examples: customer-support bots (classify ticket → draft reply), search assistants (understand query → summarise results), clinical tools (extract entities from notes → generate discharge summary)

Must remember:

- NLP is the umbrella; NLU and NLG are its two major branches
- NLU asks what does the text mean? - sentiment, NER, classification, intent
- NLG asks how should the machine express information? - summarisation, QA, reports, translation
- NLU = understanding; NLG = generation
- Production pipelines typically chain NLU → backend logic → NLG
- Conceptual split remains valuable even when one model handles multiple tasks

Example or contrast: Aspect: Input; NLU: Human language (text/speech); NLG: Structured data, prompts, or internal state

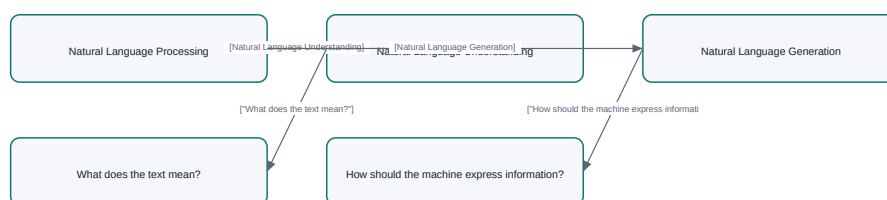
Aspect: Output; NLU: Labels, entities, embeddings, parsed meaning; NLG: Sentences, paragraphs, dialogue turns

Aspect: Evaluation; NLU: Accuracy, F1, entity-level metrics; NLG: Fluency, coherence, factual correctness

Exam trap: Claiming NLU and NLG are disjoint - most applications combine both (understand → act → respond)

Equating NLG only with chatbots - summarisation, translation, and report generation are equally NLG

NLP, NLU, and NLG: Understanding the Relationship



Vector reconstruction of the source Mermaid visual

Concept map: NLP, NLU, and NLG: Understanding the Relationship

NLP-W01-T04

Applications of Natural Language Processing by Domain

Tier B

Definition and why it matters: NLP is not a single product - it is a toolkit applied wherever unstructured text must be understood or produced at scale. The same underlying techniques (classification, entity extraction, summarisation, retrieval) reappear across industries; only the data, labels, and compliance constraints change

Concept flow: NLP Applications

Core explanation: Search engines must interpret vague, misspelled, or conversational queries and return relevant documents from billions of pages

Application: Query understanding; What It Does: Parse intent, expand synonyms, correct spelling; NLP Techniques: Tokenisation, embeddings, intent classification

Application: Document ranking; What It Does: Order results by relevance to query; NLP Techniques: TF-IDF, neural rankers, transformer encoders

Application: Chatbots; What It Does: Answer FAQs, route complex issues; NLP Techniques: Intent detection (NLU) + response generation (NLG)

Application: Ticket classification; What It Does: Auto-label urgency, product area, sentiment; NLP Techniques: Text classification, NER for product mentions

Application: Sentiment analysis; What It Does: Gauge public opinion toward brands,

events, policies; NLP Techniques: Polarity classification, aspect-based sentiment Application: Trend detection; What It Does: Surface emerging topics and hashtags; NLP Techniques: Topic modelling, clustering, time-series aggregation Example: A user searches "cheapest GPU for LLM inference" - query understanding maps to product-search intent; ranking uses textual similarity and click signals Support organisations process high volumes of repetitive, semi-structured text (tickets, chats, emails) Application: Clinical note analysis; What It Does: Extract diagnoses, medications, procedures; NLP Techniques: NER, relation extraction, medical ontologies Application: Medical record summarisation; What It Does: Condense lengthy charts for handoffs; NLP Techniques: Abstractive/extractive summarisation Example: A SaaS platform classifies incoming tickets as billing, bug, or feature request before human agents see them - reducing mean time to resolution Social platforms generate continuous streams of short, informal, multimodal text Application: News analysis; What It Does: Extract market-moving events from headlines; NLP Techniques: NER, event extraction, sentiment Application: Risk and compliance monitoring; What It Does: Detect policy violations in communications; NLP Techniques: Classification, anomaly detection on text Example: A brand monitors Twitter/X during a product launch - sentiment dashboards flag negative spikes tied to a specific feature complaint

Must remember:

- NLP applications span search, support, social media, healthcare, finance, and education
- Search: query understanding + document ranking
- Customer support: chatbots + ticket classification
- Social media: sentiment analysis + trend detection
- Healthcare: clinical note analysis + record summarisation
- Finance: news analysis + risk/compliance monitoring
- Education: automated grading + language learning tools
- Most deployments follow preprocess → NLU → business logic → optional NLG

Exam trap: Listing applications without linking to underlying NLP tasks - exam questions often ask which technique (NER vs classification vs summarisation) fits a scenario Assuming one model generalises across domains - medical NER requires domain-specific corpora and ontologies

NLP-W01-T05

Morphology of Text: Stems, Roots, and Affixes

Tier B

Definition and why it matters: Consider the words running, runner, runs, and ran. Humans instantly recognise these as variants of the same core idea - run. Morphology is the linguistic study of how words are built from smaller meaningful parts. For NLP systems, this matters because treating every surface form as a unique token explodes vocabulary size and prevents generalisation Morphology directly motivates text preprocessing choices: stemming, lemmatisation, and subword tokenisation all exist to exploit word-internal structure

Core explanation: Unit: Root; Definition: Core lexical meaning; often not a standalone word in English; Example: struct in structure, construct, destruct Unit: Stem; Definition: Reduced form produced by chopping affixes (algorithmic, not always linguistic); Example: comput from computing, computed, computer Unit: Affix; Definition: Prefix or suffix attached to a root/stem; Example: un- (prefix), -ing (suffix) in unhappiness Unit: Lemma; Definition: Dictionary base form; Example: run for running, ran, runs Concept flow: Prefix: un- - Root: happy - Suffix: -ness - Lemma: happy - Stem (algo Operation: Stemming; Approach: Heuristic chop (Porter, Snowball); Trade-off: Fast; may produce non-words (studies - studi) Operation: Lemmatisation; Approach: Dictionary + POS lookup; Trade-off: Slower; linguistically correct (studies - study) Without morphological awareness, a corpus containing run, runs, running, and ran allocates four separate dimensions in a bag-of-words vector - sparse and redundant Concept flow: Morphological Structure - Stemming / Lemmatisation - Subword Tokenisation - Smaller Vocabulary - OOV Robustness - Better Generalisation Models that recognise shared roots generalise better to unseen inflections. A spam filter trained on "click" should ideally recognise "clicking" and "clicked"

Must remember:

- Morphology studies word internal structure and word-formation relations
- Roots carry core meaning; affixes modify it; lemmas are dictionary base forms

- Inflection adjusts grammar; derivation often changes word class
- Morphology motivates vocabulary reduction and better generalisation in NLP
- Stemming is fast/heuristic; lemmatisation is slower/linguistically grounded
- Subword tokenisation in modern models handles morphology via learned splits

Example or contrast: Process: Inflection; Changes: Grammatical features (tense, number, case); Example: walk - walked, walks; Same part of speech?: Yes Process: Derivation; Changes: Creates new word, often new POS; Example: happy - happiness (adj - noun); Same part of speech?: Often no NLP pipelines must decide whether to collapse inflected forms (usually yes for IR/classification) while being cautious with derivations (compute vs computer are related but not identical)

Exam trap: Confusing stem and lemma - stems are algorithmic truncations; lemmas are canonical dictionary forms Assuming stemming always improves performance - over-stemming can merge unrelated words (university / universe with naive Porter)

NLP-W01-T06

Ambiguity in Language: Polysemy and Synonymy

Tier B

Definition and why it matters: Humans resolve ambiguous language effortlessly using context, tone, and world knowledge. Machines start with only character sequences. Ambiguity - multiple valid interpretations for the same surface form - is therefore one of the central challenges in NLP Two especially important phenomena are polysemy (one word, many related meanings) and synonymy (many words, similar meanings). Together they explain why dictionary lookups fail and why context-aware embeddings became the dominant representation strategy

Core explanation: Ambiguity occurs when a word, phrase, or sentence admits more than one valid interpretation Humans disambiguate using: Surrounding words and sentences Level: Word; Example: bank (financial vs river); Challenge: Lexical disambiguation Domain and situational context Level: Phrase; Example: "telescope man" (man with telescope vs man visible through telescope); Challenge: Attachment ambiguity Pragmatic and cultural knowledge Level: Sentence; Example: "I saw her duck"; Challenge: Structural/syntactic ambiguity Machines lack this unless explicitly modelled - making ambiguity a first-class engineering problem Level: Discourse; Example: Pronoun "it" referring to multiple antecedents; Challenge: Coreference resolution Word: bank; Sense 1: Financial institution; Sense 2: River edge Word: paper; Sense 1: Writing material; Sense 2: Academic article Ambiguity is not limited to single words: Word: run; Sense 1: Jog physically; Sense 2: Manage (run a team); Sense 3: Execute (run a program) Rule-based systems that map each token to a single dictionary sense struggle at every level Polysemy is when a single word has multiple related meanings - senses share an etymological or conceptual connection Word A: big; Word B: large; Relation: Near-synonyms Word A: buy; Word B: purchase; Relation: Near-synonyms Word A: happy; Word B: joyful; Relation: Near-synonyms Polysemy is extremely common in natural language. The correct sense is almost always determined by context, not the word alone

Must remember:

- Ambiguity = multiple valid interpretations; humans use context, machines must learn it
- Ambiguity exists at word, phrase, sentence, and discourse levels
- Polysemy: one word, multiple related meanings (bank, run, paper) - context selects sense
- Synonymy: different words, similar meaning (big/large, buy/purchase) - causes sparsity and matching issues
- Dictionary lookups alone are insufficient for both phenomena
- Embedding-based and contextual representations address polysemy and synonymy via distributional context
- Rule-based systems fail on implicit context, domain senses, and lexical variation

Example or contrast: Interpretation: Verb reading; Meaning: I saw her bend down quickly (duck as verb) Interpretation: Noun reading; Meaning: I saw her pet bird (duck as animal) Concept flow: I saw her duck - Verb parse: duck = crouch - Noun parse: duck = animal - Context disambiguates

Exam trap: Treating polysemy and homonymy as identical - homonyms (bank river vs bank money) are historically unrelated; polysemous senses are related Assuming synonyms are perfectly interchangeable in all contexts - big vs large have different collocations (big mistake vs ?large mistake)

Week 1 Exam Lens

Likely questions

- Define and explain NLP, NLU, and NLG: Understanding the Relationship; include the mechanism and one limitation.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Mistakes to avoid: Treating NLP as only "chatbots" - it spans search ranking, fraud monitoring, clinical summarisation, and more Defining NLP as only "text generation" - understanding and extraction are equally core Claiming NLU and NLG are disjoint - most applications combine both (understand → act → respond)

Week 2: Text Preprocessing and Cleaning

Week roadmap: Text Preprocessing and Cleaning: Foundations for NLP → Tokenisation: Splitting Text into Processable Units → Noise Removal with Regular Expressions → Stopword Removal: Filtering Function Words → Stemming: Rule-Based Morphological Reduction → Lemmatisation: Dictionary-Based Morphological Normalisation → 1 more topics.

Coverage: 7 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W02-T01

Text Preprocessing and Cleaning: Foundations for NLP

Tier A

Definition and why it matters: Human language is messy: inconsistent spelling, informal grammar, HTML markup in web scrapes, emojis in social posts, and domain-specific jargon. Machine learning models and NLP algorithms operate on structured numerical representations - they cannot directly consume unstructured prose

Core explanation: Text preprocessing transforms raw text into a form algorithms can process reliably. Text cleaning removes non-linguistic noise; text preprocessing applies linguistic normalisation. The distinction matters because cleaning without normalisation leaves surface variation intact, while normalisation without cleaning feeds garbage tokens into downstream steps

Aspect: Goal; Text Cleaning: Remove unwanted elements; Text Preprocessing: Normalise linguistic form

Aspect: Examples; Text Cleaning: HTML tags, emojis, extra whitespace; Text Preprocessing: Tokenisation, stemming, lemmatisation

Aspect: Analogy; Text Cleaning: Washing vegetables; Text Preprocessing: Chopping and seasoning for a recipe

Aspect: When skipped; Text Cleaning: Structured logs with clean fields; Text Preprocessing: Already tokenised subword inputs

Concept flow: Raw Text - Text Cleaning - Text Preprocessing - NLP / ML Model - Noise Removal - Whitespace Normalisation - Tokenisation - Stopword Handling - Stemming / Lemmatisation

Almost every NLP system - from classical TF-IDF classifiers to modern LLM pipelines - depends on some preprocessing stage, even when it is hidden inside a tokenizer

API Customer support chatbots must strip HTML from ticket bodies before intent classification

Search engines normalise query terms so "running," "runs," and "ran" match the same documents

LLM fine-tuning pipelines still apply cleaning when ingesting PDFs, web pages, or OCR output - garbage tokens inflate vocabulary and degrade convergence

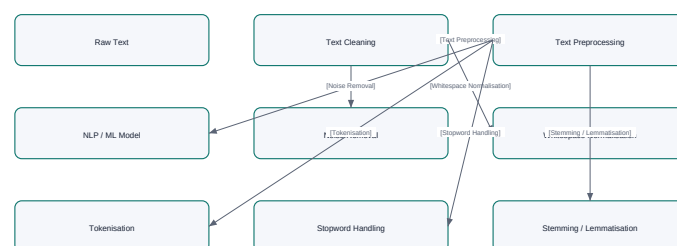
This module covers both cleaning and preprocessing as a unified foundation: Tokenisation - breaking text into units Noise removal - regex-based cleaning Stopword removal - filtering high-frequency function words Stemming and lemmatisation - morphological normalisation Pipeline construction - ordering steps for reproducible workflows

Must remember:

- Raw human text is unstructured; NLP requires machine-readable form first
- Cleaning removes noise (HTML, symbols, whitespace); preprocessing normalises linguistics
- Both steps underpin classical and modern NLP pipelines
- Task-aware design is essential - no universal cleaning rule
- Module path: tokenisation → noise removal → stopwords → stemming/lemmatisation → pipeline

Exam trap: Treating cleaning and preprocessing as identical - exams may ask which step removes HTML vs which reduces "running" to "run" Assuming one universal pipeline fits all tasks - sentiment analysis keeps negation words; NER needs minimal preprocessing

Text Preprocessing and Cleaning: Foundations for NLP



Vector reconstruction of the source Mermaid visual

Concept map: Text Preprocessing and Cleaning: Foundations for NLP

NLP-W02-T02 Tokenisation: Splitting Text into Processable Units

Tier A

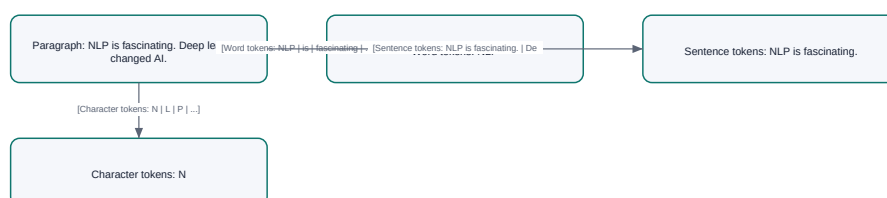
Definition and why it matters: Tokenisation is the process of breaking raw text into smaller units called tokens. Tokens can represent words, sentences, subwords, or individual characters. Every NLP pipeline - from simple word counting to transformer models - begins here

Core explanation: If tokenisation is wrong, every downstream step inherits the error: wrong POS tags, missed entities, skewed TF-IDF weights
 Level: Word; Unit: Space/punctuation-delimited tokens; Typical Use: Bag-of-words, classical NLP
 Level: Sentence; Unit: Clause/period boundaries; Typical Use: Summarisation, document segmentation
 Level: Subword; Unit: BPE/WordPiece fragments; Typical Use: BERT, GPT tokenizers
 Level: Character; Unit: Individual letters; Typical Use: Spell-checking, very low-resource languages
 Concept flow: Paragraph: NLP is fascinating. Deep learning changed AI. - Word tokens: NLP - Sentence tokens: NLP is fascinating. - Character tokens: N
 Natural language resists naive splitting: Punctuation ambiguity - "U.S.A." vs "USA"; decimals vs sentence endings Hyphenation - "state-of-the-art" as one token or four Numbers and symbols - "USD 1.2B", version strings, URLs Emojis and Unicode - multi-codepoint sequences Language and domain - Chinese has no spaces; biomedical text has chemical formulae There is no single correct tokenisation for all applications. The choice depends on language, domain, and downstream task
 NLTK provides two primary functions after downloading the punkt resource: Word tokenisation - `wordtokenize(text)` splits on word boundaries while preserving punctuation as separate tokens Sentence tokenisation - `senttokenize(text)` splits on sentence boundaries using a pre-trained Punkt model
 Production search - query tokenisation must match index tokenisation exactly Multilingual SaaS - different Punkt models or spaCy pipelines per language
 Transformer APIs - subword tokenisation (BPE) differs from NLTK word splits; comparing counts across tokenisers is invalid
 Code pattern: `import nltk; nltk.download('punkt'); from nltk.tokenize import wordtokenize, senttokenize; text = "Natural language processing is fascinating."; wordtokenize(text); ['Natural', 'language', 'processing', 'is', 'fascinating', '.']; senttokenize("First sentence. Second sentence."); ['First sentence.', 'Second sentence.']`

Must remember:

- Tokenisation = breaking text into tokens (words, sentences, subwords, or characters)
- First step in virtually every NLP pipeline; errors propagate downstream
- Complexity arises from punctuation, hyphens, numbers, emojis, and language-specific rules
- NLTK: `wordtokenize()` and `senttokenize()` after downloading punkt
- No universal tokenisation strategy - match granularity to task and model

Exam trap: Assuming split on whitespace is sufficient - fails on "don't", "U.K.", URLs Using word tokenisation for BERT - transformer models require their own subword tokenisers

Tokenisation: Splitting Text into Processable Units

Vector reconstruction of the source Mermaid visual

Concept map: Tokenisation: Splitting Text into Processable Units

NLP-W02-T03 Noise Removal with Regular Expressions

Tier B

Definition and why it matters: In NLP, noise refers to text elements that do not contribute semantic meaning but inflate vocabulary and distort statistics. Common sources: HTML/XML tags (, , etc.)

Core explanation: Regular expressions (regex) provide a flexible, pattern-based way to find and replace text at scale. Key properties: Language-independent - operates on character patterns, not grammar
 Concept flow:

Raw HTML Text - Regex Patterns - Strip HTML Tags - Keep Alphabetic + Spaces - Normalise Whitespace - Clean Text Noise Type: HTML tags; Example Input: This is important; Target Output: This is important Noise Type: Special chars; Example Input: @NLP!!! is awesome; Target Output: NLP is awesome Noise Type: Irregular spacing; Example Input: This sentence has gaps; Target Output: This sentence has gaps Industry standard - used in production ETL and NLP pipelines Composable - multiple patterns chained in sequence Pattern: `r' '` - matches anything between (non-greedy) Element: Numbers; Remove for topic modelling?: Often yes; Keep for financial NLP?: Must keep Element: Emojis; Remove for topic modelling?: Usually yes; Keep for financial NLP?: Keep for social media sentiment Rather than listing every unwanted symbol, invert the match: Element: Punctuation; Remove for topic modelling?: Often yes; Keep for financial NLP?: Keep for QA and sentiment ("?" matters) The `^` inside `[]` negates the character class - replace everything except listed characters Important: Replace with a single space `' '`, not empty string `''`, or words merge together There is no universal cleaning rule. Always align with the downstream task: Code pattern: `import re; re.sub(r' ', '', htmltext)` Code pattern: `re.sub(r'[^a-zA-Z\s]', '', text)` keep letters and spaces only Code pattern: `re.sub(r'\s+', ' ', text).strip()` collapse multiple spaces to one

Must remember:

- Noise = non-meaningful elements that inflate vocabulary and hurt models
- Regex enables scalable, language-independent pattern-based cleaning
- HTML: `r' '`; alphabetic-only: `r'[^a-zA-Z\s]'`; whitespace: `r'\s+' → single space`
- Regex is case-sensitive unless flagged otherwise
- Cleaning decisions must match the downstream NLP task - no one-size-fits-all rule

Example or contrast: By default, `re.sub('important', 'Important', text)` only matches lowercase. Use `re.IGNORECASE` flag or explicit character classes when case-insensitive matching is needed

Exam trap: Replacing `\s+` with empty string instead of single space - destroys word boundaries Forgetting regex is case-sensitive by default

NLP-W02-T04 Stopword Removal: Filtering Function Words

Tier B

Definition and why it matters: Stopwords are high-frequency words that carry little standalone semantic meaning - articles, prepositions, conjunctions, and common auxiliaries. Examples: the, is, of, and, a

Core explanation: They dominate word frequency distributions, making models overweight function words relative to content-bearing terms like policy, economy, or diagnosis Benefit: Reduced vocabulary size; Explanation: Fewer dimensions in bag-of-words / TF-IDF Benefit: Improved efficiency; Explanation: Faster training and smaller feature matrices Benefit: Better signal; Explanation: Models focus on content words that distinguish documents Task: Sentiment analysis; Why stopwords matter: not, no, never flip polarity - "not good" != "good" Task: Question answering; Why stopwords matter: Function words define query structure Task: Language modelling; Why stopwords matter: Next-word prediction needs full grammar Task: Dependency parsing; Why stopwords matter: Syntactic structure depends on determiners and auxiliaries Example: In topic modelling, the and is do not reveal themes; health, budget, and regulation do Stopword removal is applied after tokenisation: tokenise first, then filter tokens against a stopwords list NLTK ships stopwords lists for multiple languages. The English list is generic - customise for domain-specific needs Stopwords carry grammatical and semantic weight in several tasks: Example: "This movie is not good" - removing not inverts meaning entirely Domain-specific high-frequency words may add no value: Product reviews - product, item appear too often News corpora - said, reported are uninformative Practise: add domain terms to the stopwords set, or remove words like not from the default list for sentiment tasks Code pattern: `import nltk; from nltk.corpus import stopwords; from nltk.tokenize import word_tokenize; nltk.download('stopwords'); nltk.download('punkt'); stopwords = set(stopwords.words('english')); tokens = word_tokenize("Natural language processing is a fascinating area of computer science."); filtered = [w for w in tokens if w.lower() not in stopwords]`

Must remember:

- Stopwords = frequent function words with low discriminative power
- Removal reduces vocabulary, improves efficiency, highlights content words
- Apply after tokenisation using NLTK's `stopwords.words('english')`

- Keep stopwords for sentiment, QA, language modelling, and grammar-aware tasks
- Customise lists per domain - add domain noise, preserve negations

Exam trap: Treating stopword removal as a mandatory rule - it is a design decision Removing negation words in sentiment pipelines

NLP-W02-T05 Stemming: Rule-Based Morphological Reduction

Tier A

Definition and why it matters: Stemming reduces inflected or derived word forms to a common stem - a crude base form obtained by stripping suffixes. The goal is vocabulary normalisation: treat related surface forms as one token connect, connected, connecting, connection → connect

Core explanation: When exact word form is unimportant, stemming: Groups morphological variants together
 Use Case: Search engines; Why stemming helps: Query "running" matches documents with "run", "runs"
 Use Case: Information retrieval; Why stemming helps: Higher recall across word variants Use Case: Topic modelling; Why stemming helps: Fewer sparse dimensions from inflectional forms Use Case: Early NLP pipelines; Why stemming helps: Fast, lightweight normalisation Stemmer: Porter Stemmer; Basis: Original Porter rules (1980); Notes: Most widely used; English-focused Stemmer: Snowball Stemmer; Basis: Porter2 / improved rules; Notes: Supports multiple languages Reduces vocabulary size Speeds preprocessing and retrieval NLTK provides two main algorithms: Code pattern: from nltk.stem import PorterStemmer; ps = PorterStemmer(); ps.stem('running') 'run'; ps.stem('studies') 'studi' (over-stemming); ps.stem('better') 'better' (under-stemming)

Must remember:

- Stemming = rule-based reduction to a common stem, ignoring context
- Reduces vocabulary, groups inflections, speeds IR and topic modelling
- NLTK: PorterStemmer() and SnowballStemmer()
- Produces non-words; over/under-stemming are known failure modes
- Prefer stemming when speed matters and slight inaccuracies are acceptable

Example or contrast: Aspect: Output; Stemming: May be non-word; Lemmatisation: Valid dictionary form
 Aspect: Method; Stemming: Rule-based suffix stripping; Lemmatisation: Dictionary + POS lookup
 Aspect: Speed; Stemming: Faster; Lemmatisation: Slower

Exam trap: Problem: Non-words produced; Example: studies - studi (not a valid English word) Problem: Over-stemming; Example: organisation and organ merged despite different meanings

NLP-W02-T06 Lemmatisation: Dictionary-Based Morphological Normalisation

Tier A

Definition and why it matters: Lemmatisation reduces a word to its lemma - the canonical dictionary form - using vocabulary and morphological rules. Unlike stemming, lemmatisation produces valid words and can respect grammatical role running (verb) → run

Core explanation: Stemming blindly strips suffixes. Lemmatisation asks: What part of speech is this word, and what is its base form? Word: playing; Context: verb ("They are playing"); Lemma: run - play Word: playing; Context: noun ("The playing was excellent"); Lemma: unchanged - playing Code: n; Part of Speech: Noun (default) Code: v; Part of Speech: Verb Code: a; Part of Speech: Adjective Code: r; Part of Speech: Adverb This POS sensitivity makes lemmatisation valuable for text classification, information extraction, question answering, and semantic analysis POS-aware lemmatisation dramatically improves results - always specify POS when available spaCy performs tokenisation, POS tagging, and lemmatisation in one pipeline: spaCy does not offer stemming - it treats lemmatisation as the production-grade approach Code pattern: import nltk; from nltk.stem import WordNetLemmatizer; nltk.download('wordnet'); nltk.download('omw-1.4'); lemmatizer = WordNetLemmatizer(); lemmatizer.lemmatize('running') 'running' (default POS = noun); lemmatizer.lemmatize('running', pos='v') 'run'; lemmatizer.lemmatize('studies') 'study' Code pattern: import spacy; nlp = spacy.load('en_core_web_sm'); doc = nlp("The children were playing happily."); for token in doc; print(token.text, token.lemma); The the | children child | were be | playing play | happily happily

Must remember:

- Lemmatisation → valid dictionary lemma using vocabulary + POS

- More accurate than stemming; handles better → good, verb playing → play
- NLTK: WordNetLemmatizer with POS codes n/v/a/r
- spaCy: integrated pipeline; no separate stemming option
- Use lemmatisation for production and semantic tasks; stemming for speed-critical IR

Example or contrast: Choose Stemming: Speed is critical; Choose Lemmatisation: Word meaning matters
 Choose Stemming: Exploratory analysis; Choose Lemmatisation: Clean, interpretable tokens needed
 Choose Stemming: Slight errors acceptable; Choose Lemmatisation: Production-grade NLP systems

Exam trap: Calling `lemmatize('running')` without POS - defaults to noun, returns `running` not `run`
 Expecting lemmatisation to be as fast as stemming - dictionary lookup is slower

NLP-W02-T07 Building a Text Preprocessing Pipeline

Tier B

Definition and why it matters: Individual preprocessing steps - cleaning, tokenisation, stopword removal, lemmatisation - are rarely applied in isolation. Production NLP systems chain them into a reusable pipeline that ensures:

Core explanation: Consistency - same transformations on training and inference data
 Concept flow: Raw Text - Text Normalisation: lowercase - Noise Removal: regex - Tokenisation - Stopword Removal - Stemming or Lemmatisation - Optional Phrase Handling - Processed Tokens
 Step: Normalise case first; Rationale: Ensures consistent matching downstream
 Step: Clean before tokenise; Rationale: Prevents HTML fragments becoming tokens
 Step: Tokenise before stopwords; Rationale: Stopword lists operate on tokens
 Step: Lemmatise after stopwords; Rationale: Fewer tokens to lemmatise (efficiency)
 Reproducibility - documented, ordered steps
 Scalability - batch processing across datasets and tasks
 Task: Sentiment analysis; Pipeline adjustment: Keep negation words (not, never); minimal cleaning
 Incorrect step ordering can destroy information or produce inconsistent tokens
 Task: Topic modelling; Pipeline adjustment: Aggressive stopword removal; lemmatisation
 Task: Named Entity Recognition; Pipeline adjustment: Minimal preprocessing - preserve capitalisation and punctuation
 Task: Search engines; Pipeline adjustment: Stemming often preferred over lemmatisation for speed
 Example: "Natural Language Processing is an exciting field of AI!" → ['natural', 'language', 'process', 'exciting', 'field', 'ai']
 A pipeline must be task-aware, data-aware, and flexible - the template above is a general starting point, not a universal solution
 Code pattern: `import re; import nltk; from nltk.tokenize import word_tokenize; from nltk.corpus import stopwords; from nltk.stem import WordNetLemmatizer; nltk.download('punkt'); nltk.download('stopwords'); nltk.download('wordnet')`

Must remember:

- Pipelines chain preprocessing for consistency, reproducibility, and scale
- Order: normalise → clean → tokenise → stopwords → stem/lemmatise
- Task-specific variants: sentiment keeps negations; NER keeps minimal changes
- NLTK pipeline combines regex cleaning, wordtokenize, stopword filter, WordNetLemmatizer
- Always validate pipeline output on representative raw text before modelling

Exam trap: Tokenising before cleaning - HTML tags become spurious tokens
 Removing stopwords before sentiment tasks - loses negation

Week 2 Exam Lens

Likely questions

- Define and explain Text Preprocessing and Cleaning: Foundations for NLP; include the mechanism and one limitation.
- Compare Text Preprocessing and Cleaning: Foundations for NLP with Tokenisation: Splitting Text into Processable Units and state when each is preferred.
- Apply Stemming: Rule-Based Morphological Reduction to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Mistakes to avoid: Treating cleaning and preprocessing as identical - exams may ask which step removes HTML vs which reduces "running" to "run" Assuming split on whitespace is sufficient - fails on "don't", "U.K.", URLs Replacing \s+ with empty string instead of single space - destroys word boundaries

Week 3: POS Tagging and Named Entity Recognition

Week roadmap: Structural Text Analysis: POS Tagging and Named Entity Recognition → Part-of-Speech Tagging: Assigning Grammatical Categories → POS Tagging with spaCy → Visualising POS Tags and Dependency Trees with spaCy → POS Tagging with NLTK → POS Tagging with Flair → 5 more topics.

Coverage: 11 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W03-T01 Structural Text Analysis: POS Tagging and Named Entity Recognition

Tier A

Definition and why it matters: This module shifts from preprocessing to structural analysis - understanding how words function grammatically and which tokens refer to real-world entities Technique: Part-of-Speech (POS) Tagging; Layer: Syntactic; Question Answered: What grammatical role does each word play?

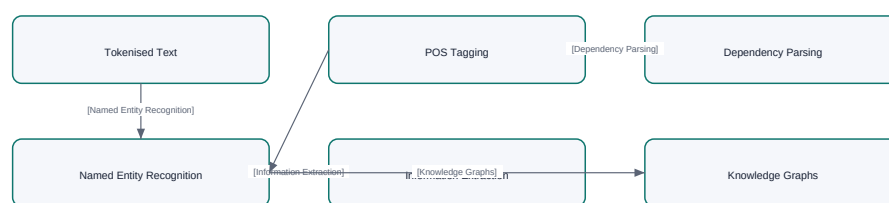
Core explanation: Preprocessing normalises surface form; structural analysis reveals relationships: POS tags expose syntax - nouns vs verbs, determiners vs auxiliaries Library: spaCy; Role in Module: Production-grade POS and NER with visualisation Library: NLTK; Role in Module: Traditional modular pipeline; educational baseline Library: Flair; Role in Module: Neural sequence taggers; accuracy-focused alternative NER maps text to ontology categories - Google as ORG, not just NN Modern models learn implicit syntax, but explicit tags remain interpretable and useful for rule-based post-processing A comparative framework helps choose the right tool for learning, production, or maximum accuracy

Must remember:

- Module focus: syntactic structure (POS) and semantic entities (NER)
- POS → grammatical roles; NER → persons, orgs, locations, dates, money
- Both underpin IE, QA, knowledge graphs, and parsing pipelines
- Implementations: spaCy (production), NLTK (fundamentals), Flair (accuracy)
- Library choice depends on speed vs accuracy vs educational goals

Exam trap: Confusing POS tagging with NER - POS assigns grammatical categories; NER identifies real-world entity types Assuming POS is a dictionary lookup - tags are context-dependent (book as verb vs noun)

Structural Text Analysis: POS Tagging and Named Entity Recognition



Vector reconstruction of the source Mermaid visual

Concept map: Structural Text Analysis: POS Tagging and Named Entity Recognition

NLP-W03-T02 Part-of-Speech Tagging: Assigning Grammatical Categories

Tier B

Definition and why it matters: Part-of-Speech (POS) tagging assigns a grammatical category to each token in a sentence - noun, verb, adjective, adverb, preposition, determiner, pronoun, and finer-grained subtypes Token: Natural; POS Tag: JJ; Category: Adjective

Core explanation: POS tags enable and improve: Dependency parsing - subject-verb-object relationships Sentence: "I will book a ticket."; book tag: VB (verb); Role: Action Sentence: "Read this book."; book tag: NN (noun); Role: Object Concept flow: Word: book - Context? - Verb - Noun Information extraction - noun phrases as entity candidates Text normalisation - POS-aware lemmatisation Framework: NLTK / Penn Treebank; Tag Set: Penn POS; Example Tags: NN, NNP, VBZ, DT, JJ Question answering - identifying wh-words and focus entities Framework: spaCy; Tag Set: Universal POS + fine-grained; Example Tags: NOUN, VERB, DET +

token.tag Even transformer models learn implicit syntactic patterns; explicit POS tags remain interpretable and useful for hybrid pipelines Framework: Flair; Tag Set: Flair / OntoNotes-style; Example Tags: NNP, VBZ, DT POS tagging is not dictionary lookup - the same word receives different tags in different contexts: Disambiguation requires surrounding context - a core challenge that modern taggers address with statistical or neural models Different frameworks use different tag sets: Always consult the tag set documentation when interpreting output

Must remember:

- POS tagging assigns grammatical categories to each token
- Reveals syntactic structure; supports parsing, IE, QA, and lemmatisation
- Highly context-sensitive - same word, different tags in different sentences
- Penn Treebank tags (NN, VBZ, DT) common in NLTK; spaCy adds Universal POS
- Not a dictionary lookup - requires contextual disambiguation

Exam trap: Treating POS as fixed per word - context determines tag Confusing POS with NER - Apple as NN (noun) vs ORG (organisation entity)

NLP-W03-T03 POS Tagging with spaCy

Tier A

Definition and why it matters: spaCy provides an end-to-end NLP pipeline: tokenisation, POS tagging, dependency parsing, and NER in a single `nlp()` call. For production workloads, spaCy balances speed and accuracy with a streamlined API

Core explanation: Code pattern: `import spacy; nlp = spacy.load('encorewebstm');` `doc = nlp("I will book a ticket.");` for token in doc:; `print(f"{token.text:12} {token.pos:5} {token.tag}")` Token: I; Universal POS (pos): PRON; Fine-grained (tag): PRP Token: will; Universal POS (pos): AUX; Fine-grained (tag): MD Token: book; Universal POS (pos): VERB; Fine-grained (tag): VB Token: a; Universal POS (pos): DET; Fine-grained (tag): DT Token: ticket; Universal POS (pos): NOUN; Fine-grained (tag): NN Universal POS (pos): PUNCT Property: Tokenisation; Detail: Automatic inside `nlp()` Property: Model; Detail: `encorewebstm` (small), md, lg variants Property: Speed; Detail: Optimised for production batch processing Property: Output; Detail: Token text, POS, tag, lemma, dependency in one pass `token.pos` - Universal POS category (human-readable) `token.tag` - Detailed Penn Treebank-style tag Document intelligence APIs - extract noun phrases for indexing Chatbot intent pipelines - verb/noun patterns for slot filling Pre-annotation - bootstrap NER training data with POS-filtered candidates

Must remember:

- spaCy: `nlp(text)` → tokens with `token.pos` and `token.tag`
- Tokenisation included; no separate step needed
- Context resolves ambiguity (book verb vs noun)
- Production-ready: fast, integrated pipeline
- Install `encorewebstm` before use; practise on varied sentences

Example or contrast: Code pattern: `doc1 = nlp("I will book a ticket.")` book - VERB; `doc2 = nlp("Read this book.")` book - NOUN spaCy's transition-based parser uses word embeddings and context to resolve ambiguity automatically - no manual pipeline assembly required

Exam trap: Forgetting to install and load the language model (`python -m spacy download encorewebstm`) Confusing pos (Universal) with tag (Penn fine-grained)

NLP-W03-T04 Visualising POS Tags and Dependency Trees with spaCy

Tier A

Definition and why it matters: POS tags alone list grammatical categories. Dependency parsing adds syntactic relationships - which token modifies which, and what role each plays (subject, object, auxiliary)

Core explanation: spaCy's `displaCy` renders both POS tags and dependency arcs visually - invaluable for debugging pipelines and communicating NLP results Concept flow: book VERB - I PRON: nsubj - will AUX: aux - ticket NOUN: dobj - a DET: det Relation: nsubj; Meaning: Nominal subject; Example: I - book Relation: aux; Meaning: Auxiliary verb; Example: will - book Relation: dobj; Meaning: Direct object; Example: ticket

- book The dependency visualisation shows: Relation: det; Meaning: Determiner; Example: a - ticket Tokens with POS labels beneath each word Directed arcs linking head tokens to dependents Relation labels - nsubj (nominal subject), dobj (direct object), aux (auxiliary), det (determiner) Sentence: "I will book a ticket." Scenario: Pipeline debugging; Benefit: Spot wrong attachments quickly Scenario: Stakeholder demos; Benefit: Non-technical audience grasps structure Scenario: Annotation QA; Benefit: Verify parser output before training custom models Scenario: Teaching; Benefit: Illustrate subject-verb-object patterns Sentence: "Read this book." read (VERB) - root book (NOUN) - dobj of read this (DET) - det of book Code pattern: from spacy import displacy; doc = nlp("I will book a ticket."); displacy.render(doc, style='dep', jupyter=True) Same word book - noun in this sentence, verb in the previous example. Visualisation makes the contrast immediate

Must remember:

- displaCy.render(doc, style='dep') visualises POS + dependency arcs
- Dependency relations: nsubj, aux, dobj, det link tokens syntactically
- Same word gets different structure in different sentences (book verb vs noun)
- Essential for debugging, demos, and understanding parser behaviour
- NER visualisation uses style='ent' - different from dependency view

Exam trap: Using style='ent' for POS - use style='dep' for dependency/POS trees; 'ent' is for NER Assuming visualisation replaces evaluation - pretty graphs don't guarantee correctness

NLP-W03-T05 POS Tagging with NLTK

Tier A

Definition and why it matters: NLTK implements POS tagging as an explicit multi-step pipeline: tokenise first, then tag. This transparency helps learners understand each stage - unlike spaCy's single-call abstraction

Core explanation: Required NLTK downloads: Tag: NN; Meaning: Noun, singular; Example: language Tag: NNP; Meaning: Proper noun, singular; Example: NLTK Tag: VBZ; Meaning: Verb, present 3sg; Example: is Tag: DT; Meaning: Determiner; Example: a The averaged perceptron tagger is a classical ML tagger - a lightweight neural precursor useful for understanding non-transformer tagging Tag: JJ; Meaning: Adjective; Example: powerful Tag: IN; Meaning: Preposition; Example: for nltk.postag() returns a list of (token, tag) tuples using the Penn Treebank tag set Look up any tag programmatically: Full reference: nltk.help.upenn_tagset() without arguments lists all tags Code pattern: import nltk; from nltk.tokenize import word_tokenize; nltk.download('punkt'); nltk.download('averagedperceptron_tagger'); nltk.download('tagsetsjson') Code pattern: text = "NLTK is a powerful library for natural language processing."; tokens = word_tokenize(text); tagged = nltk.postag(tokens); [('NLTK', 'NNP'), ('is', 'VBZ'), ('a', 'DT'), ('powerful', 'JJ'), ...] Code pattern: nltk.help.upenn_tagset('VBZ'); verb, present tense, 3rd person singular

Must remember:

- NLTK POS: word_tokenize() → nltk.postag() → Penn Treebank tags
- Download punkt + averagedperceptron_tagger + tagsetsjson
- Common tags: NN, NNP, VBZ, DT, JJ, IN
- Lookup: nltk.help.upenn_tagset('VBZ')
- Modular pipeline ideal for learning; spaCy preferred for production speed

Example or contrast: Aspect: Pipeline; NLTK: Manual: tokenise - tag; spaCy: Automatic in nlp() Aspect: Tagger; NLTK: Averaged perceptron; spaCy: Transition-based NN Aspect: Speed; NLTK: Slower; spaCy: Faster

Exam trap: Forgetting averagedperceptron_tagger download - tagging fails silently or errors Confusing NN (common noun) with NNP (proper noun)

NLP-W03-T06 POS Tagging with Flair

Tier A

Definition and why it matters: Flair is a PyTorch-based NLP framework offering pre-trained neural sequence taggers for POS tagging, NER, sentiment analysis, and more. Models use contextual embeddings (bi LSTM-CRF architectures) to achieve high accuracy at the cost of slower inference compared to spaCy

Core explanation: Code pattern: from flair.data import Sentence; from flair.models import SequenceTagger; tagger = SequenceTagger.load('pos'); sentence = Sentence("Flair is a great library for natural language process-

ing."); `tagger.predict(sentence)`; `print(sentence.totaggedstring())` Output assigns Penn-style tags: Flair/NNP, is/VBZ, a/DT, etc Aspect: Architecture; Flair: BiLSTM-CRF + contextual embeddings; NLTK: Averaged perceptron; spaCy: Transition-based NN Aspect: API; Flair: Load model - predict; NLTK: Tokenise - tag; spaCy: `nlp(text)` Aspect: Preprocessing; Flair: Built into Sentence; NLTK: Manual; spaCy: Built into pipeline Aspect: Speed; Flair: Moderate; NLTK: Slowest; spaCy: Fastest Aspect: Accuracy; Flair: High on benchmarks; NLTK: Lower; spaCy: High; slight trade-off for speed Flair tag definitions are documented on Hugging Face (Flair POS tag set reference) Flair provides language-specific POS models (e.g., pos-fast, German POS taggers). Tag sets may align with Universal Dependencies or language-specific conventions Maximum accuracy on specialised or ambiguous text Research benchmarks where F1 score matters more than latency Domain text (legal, medical) where contextual embeddings help disambiguation For most production POS needs at scale, spaCy remains the default choice

Must remember:

- Flair: PyTorch sequence taggers with contextual embeddings
- `SequenceTagger.load('pos') → tagger.predict(sentence) → totaggedstring()`
- Higher accuracy, moderate speed; multilingual models available
- Simple API: create Sentence, load model, predict
- Compare same text across NLTK, spaCy, and Flair to observe tag differences

Exam trap: Forgetting `tagger.predict(sentence)` - model load alone does not tag Passing a string instead of Sentence object - wrap text in `Sentence(text)`

NLP-W03-T07

Named Entity Recognition: Identifying Real-World Entities

Tier A

Definition and why it matters: Named Entity Recognition (NER) identifies and classifies real-world entities mentioned in text - mapping spans of tokens to predefined categories NER moves beyond syntax (POS) to semantic meaning: who, what organisation, where, when, and how much

Core explanation: Label: PERSON; Category: People (real or fictional); Examples: Sundar Pichai, Alice Label: ORG; Category: Organisations; Examples: Google, Apple, UN Label: GPE / LOC; Category: Geopolitical entities / locations; Examples: India, UK, Washington Label: DATE / TIME; Category: Temporal expressions; Examples: December 2026, Monday Label: MONEY; Category: Monetary amounts; Examples: USD 1 billion, 500 Span: Sundar Pichai; Entity Type: PERSON Span: Google; Entity Type: ORG Example: "Sundar Pichai is the CEO of Google." Challenge: Entity ambiguity; Example: Washington - person, location, or organisation? Challenge: Boundary detection; Example: Elon Musk as one span vs two tokens Challenge: Polysemy; Example: Apple - fruit (not ORG) vs company (ORG) Challenge: Context dependence; Example: Domain-specific entity types NER is highly context-dependent - accuracy requires surrounding text, not isolated token lookup

Must remember:

- NER classifies text spans into entity types: PERSON, ORG, GPE, DATE, MONEY
- Semantic layer - distinct from grammatical POS tagging
- Powers IE, search, knowledge graphs, QA, document analysis
- Context-dependent; ambiguity (Washington, Apple) is a core challenge
- NER builds on syntactic analysis but answers different questions than POS

Example or contrast: Concept flow: Named Entity Recognition - Information Extraction - Search & Indexing - Knowledge Graph Construction - Question Answering - Document Analysis / Resume Parsing Aspect: Layer; POS Tagging: Syntactic / grammatical; NER: Semantic / real-world Aspect: Output; POS Tagging: Noun, verb, adjective; NER: PERSON, ORG, GPE, DATE

Exam trap: Equating NN (noun) with named entity - all entities are typically nouns, but not all nouns are entities Assuming one-word entities only - multi-token spans (New York, Sundar Pichai) are common

NLP-W03-T08

Named Entity Recognition with spaCy

Tier A

Definition and why it matters: spaCy provides production-grade NER integrated into its core pipeline. A single `nlp()` call returns entities accessible via `doc.ents` with labels, spans, and human-readable descriptions

Core explanation: Code pattern: `import spacy; nlp = spacy.load('encorewebstm'); text = "Apple is looking at buying a UK startup for USD 1 billion. Elon Musk said in December 2026."; doc = nlp(text); for ent in doc.ents; print(ent.text, ent.label, spacy.explain(ent.label))` Entity: Apple; Label: ORG; Description: Companies, agencies, institutions Entity: UK; Label: GPE; Description: Countries, cities, states Entity: USD 1 billion; Label: MONEY; Description: Monetary values Entity: Elon Musk; Label: PERSON; Description: People, including fictional Entity: December 2026; Label: DATE; Description: Absolute or relative dates Concept flow: Apple ORG - UK GPE - USD 1 billion MONEY - Elon Musk PERSON - December 2026 DATE Strength: Accuracy; Detail: Strong on standard English news and web text Strength: Speed; Detail: Fastest among NLTK, spaCy, Flair for large corpora Strength: Integration; Detail: Same pipeline as POS and dependency parsing `ent.text - span text` Strength: Customisation; Detail: Train `EntityRuler` or fine-tune NER component `ent.label - entity type string` `spacy.explain(label) - human-readable definition` Colour-coded entity spans render in Jupyter or browser - each entity type gets a distinct highlight Financial news monitoring - extract ORG and MONEY from filings CRM enrichment - PERSON and ORG from email threads Compliance scanning - GPE and DATE in contract clauses Code pattern: `from spacy import displacy; displacy.render(doc, style='ent', jupyter=True)`

Must remember:

- spaCy NER: `doc = nlp(text) → iterate doc.ents`
- Labels: ORG, GPE, MONEY, PERSON, DATE + `spacy.explain()`
- Visualise: `displacy.render(doc, style='ent')`
- Industry default for production NER - fast and accurate
- Practise on varied text; consider custom models for specialised domains

Exam trap: Accessing entities via `doc.ents` not `doc.entities` Forgetting `ent.label` (string) vs `ent.label` (hash ID)

NLP-W03-T09 Named Entity Recognition with NLTK

Tier A

Definition and why it matters: NLTK implements NER through a multi-step classical pipeline - fundamentally different from spaCy's integrated neural approach:

Core explanation: Apply POS tagging Tag: PERSON; Meaning: People `Chunk` named entities with `nechunk()` Tag: ORG; Meaning: Organisations This modular design exposes each stage for learning, but produces lower accuracy than modern neural taggers Tag: GPE; Meaning: Geopolitical entities Tag: LOCATION; Meaning: Non-GPE locations Tag: FACILITY; Meaning: Buildings, airports, highways `nechunk()` returns a nested Tree structure where entity subtrees are labelled (e.g., GPE, PERSON, ORG) Extract named entities programmatically: NLTK's `maxentnechunker` supports a limited set of entity types: Code pattern: `import nltk; from nltk.tokenize import wordtokenize; from nltk.tag import postag; from nltk.chunk import nechunk; nltk.download('punkt'); nltk.download('averagedperceptrontagger'); nltk.download('maxentnechunker'); nltk.download('words')` Code pattern: `text = "Apple is looking at buying a UK startup for USD 1 billion. Elon Musk said in December 2026."; tokens = wordtokenize(text); tagged = postag(tokens); netree = nechunk(tagged)` Educational contexts - understanding chunking and POS-dependent NER Code pattern: `for chunk in netree; if hasattr(chunk, 'label'); print(chunk.label(), chunk.leaves())` Legacy systems - maintaining older rule-based pipelines Fallback - when neural models unavailable (resource-constrained environments) For production NER, spaCy or Flair is strongly preferred

Must remember:

- NLTK NER: `tokenise → postag() → nechunk()`
- Download `maxentnechunker` + `words` + `tagger` resources
- Limited entity types: PERSON, ORG, GPE, LOCATION, FACILITY
- Lower accuracy than spaCy - misses dates, money, multi-token spans
- Useful for learning pipeline mechanics; not recommended for production

Exam trap: On the same benchmark text, NLTK typically detects fewer entities than spaCy: Span: Apple; spaCy: ORG; NLTK (typical): GPE (misclassified)

NLP-W03-T10 Named Entity Recognition with Flair

Tier A

Definition and why it matters: Flair applies biLSTM-CRF neural architectures with contextual string embeddings to NER. The same SequenceTagger framework used for POS tagging loads a dedicated NER checkpoint

Core explanation: Trade-off: higher accuracy potential vs slower inference than spaCy Concept flow: Input Tokens - Contextual Embeddings - BiLSTM - CRF Decoder - Entity Labels per Token Output spans with labels such as ORG, LOC, MONEY, PER, MISC - depending on the model's tag set Unlike model loading alone, `tagger.predict(sentence)` must be called before accessing entities. Omitting this step returns empty results - a common implementation error Contextual embeddings capture surrounding words - critical for disambiguating Apple (company vs fruit) Specialised domains (medical, legal) where maximum F1 matters Research requiring state-of-the-art sequence labelling Batch offline processing where latency is acceptable For real-time tweet streams or high-volume document ingestion, spaCy's speed advantage dominates Code pattern:

```
from flair.data import Sentence; from flair.models import SequenceTagger; tagger = SequenceTagger.load('ner'); sentence = Sentence("Apple is looking at buying a UK startup for USD 1 billion. Elon Musk said in December 2026."); tagger.predict(sentence) Required - predictions not automatic; for entity in sentence.getspans('ner'); print(entity.text, entity.tag)
```

Must remember:

- Flair NER: `SequenceTagger.load('ner') → Sentence(text) → predict() → getspans('ner')`
- BiLSTM-CRF + contextual embeddings for high accuracy
- Must call `predict()` before reading entities
- Strong alternative to spaCy when accuracy speed
- Compare outputs on identical text across all three libraries

Example or contrast: Entity: Apple; spaCy: ORG; Flair: ORG; NLTK: GPE (partial) Entity: UK; spaCy: GPE; Flair: LOC; NLTK: Missed Entity: USD 1 billion; spaCy: MONEY; Flair: Detected; NLTK: Missed

Exam trap: Skipping `predict()` - most common Flair NER bug Using `load('pos')` instead of `load('ner')` - wrong model checkpoint

NLP-W03-T11 Comparing NLTK, spaCy, and Flair for NLP Tasks

Tier A

Definition and why it matters: For POS tagging and NER, library selection depends on the project's primary goal:

Core explanation: Goal: Learn NLP fundamentals; Recommended Library: NLTK Goal: Deploy fast production systems; Recommended Library: spaCy Goal: Maximise accuracy (compute available); Recommended Library: Flair Always match tool to task - no single library dominates all dimensions Methodology: Modular pipeline - tokenise → POS tag → chunk for NER Transparent step-by-step mechanics Excellent for teaching and small-scale experiments Rich corpora and linguistic resources Slower and less accurate on modern benchmarks NER limited entity types and span detection Not ideal for production microservices at scale Concept flow: Millions of Documents - spaCy Pipeline - Tagged + Entities - Low Latency Best for: Coursework, POCs, understanding building blocks Methodology: Efficient transition-based parser powered by shallow feed-forward neural networks and Bloom embeddings that capture word context Fastest processing - critical for millions of documents or real-time streams Integrated POS, NER, dependency parsing in one pass displaCy for visual debugging and demos Industry-standard deployment path Slightly lower accuracy than heaviest deep models (often <1% F1 difference) Best for: Production APIs, real-time tweet processing, document pipelines at scale Concept flow: Primary goal? - NLTK - spaCy - Flair Methodology: BiLSTM-CRF with contextual string embeddings - deep understanding of word meaning from surrounding text. Handles polysemy well State-of-the-art accuracy on standard benchmarks Strong on ambiguous and domain-specific text Simple high-level API Heavier model - slower inference Less ecosystem tooling than spaCy (visualisation, custom pipeline components) Best for: Medical/legal NLP, research papers, offline batch where F1 is paramount Process millions of tweets in real time → spaCy Squeeze maximum accuracy from critical medical records → Flair Build a teaching demo of NER pipeline steps → NLTK NLTK as backup when other libraries fail in constrained environments

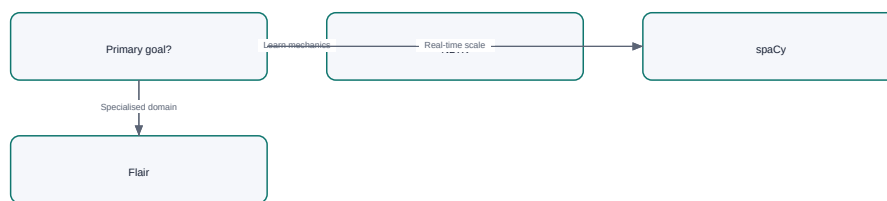
Must remember:

- NLTK: modular, educational, slowest, lowest accuracy - learn fundamentals
- spaCy: fastest, production-ready, displaCy, industry standard
- Flair: highest accuracy, BiLSTM-CRF, slower, best for specialised/research use
- Choose by goal: learn (NLTK), deploy (spaCy), accuracy (Flair)
- Real-time scale → spaCy; critical F1 → Flair; pipeline transparency → NLTK

Example or contrast: Dimension: Primary use; NLTK: Education, research POC; spaCy: Production, industry; Flair: Research, specialised domains Dimension: Speed; NLTK: Slowest; spaCy: Fastest; Flair: Moderate (slowest neural option) Dimension: Accuracy; NLTK: Lower (classical ML); spaCy: High; ~1% below heaviest models; Flair: Highest F1 on benchmarks

Exam trap: Claiming NLTK is obsolete - still valuable for education and linguistics resources Assuming Flair is always better - speed penalty matters at scale

Comparing NLTK, spaCy, and Flair for NLP Tasks



Vector reconstruction of the source Mermaid visual

Concept map: Comparing NLTK, spaCy, and Flair for NLP Tasks

Week 3 Exam Lens

Likely questions

- Define and explain Structural Text Analysis: POS Tagging and Named Entity Recognition; include the mechanism and one limitation.
- Compare Structural Text Analysis: POS Tagging and Named Entity Recognition with POS Tagging with spaCy and state when each is preferred.
- Apply Visualising POS Tags and Dependency Trees with spaCy to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Mistakes to avoid: Confusing POS tagging with NER - POS assigns grammatical categories; NER identifies real-world entity types Treating POS as fixed per word - context determines tag Forgetting to install and load the language model (python -m spacy download en_coreweb_sm)

Week 4: Text Corpora and Corpus Linguistics

Week roadmap: Text Corpora and Corpus Linguistics: Module Overview → What Is a Text Corpus and Why It Matters → Types of Text Corpora → Bias and Representation in Text Corpora → Corpus Exploration Setup with NLTK → Gutenberg Corpus Analysis: Vocabulary and Word Frequency → 1 more topics.

Coverage: 7 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W04-T01 Text Corpora and Corpus Linguistics: Module Overview

Tier C

Definition and why it matters: Every NLP model learns from text data. The quality, composition, and bias of that data directly shape model behaviour - from word embeddings to large language models

Core explanation: This module examines: Concept flow: Text Corpus - Word Frequencies / TF-IDF - Word Embeddings - Topic Models - Language Models - Bias & Fairness Issues Corpus: Gutenberg; Character: Literary classics; Analysis Goals: Vocabulary richness, word frequency, preprocessing effects What text corpora are and how they differ from documents and datasets

Must remember:

- Module covers corpora foundations, types, bias, and hands-on Gutenberg/Brown analysis
- Models learn statistics from corpora - corpus quality determines model quality
- Gutenberg: literary; Brown: genre-balanced modern English
- Explore corpora before training - avoid treating text as a black box
- Bias in data propagates to bias in predictions

Formula / decision rule: Model behaviour = $f(\text{corpus statistics})$

Exam trap: Using corpus, document, and dataset interchangeably - each has a distinct definition Assuming large corpus = good corpus - representation and bias matter more than size alone

NLP-W04-T02 What Is a Text Corpus and Why It Matters

Tier A

Definition and why it matters: A text corpus (plural: corpora) is a large, structured collection of text assembled to study, analyse, and model language. It is more than a single document or an unorganised folder of files - it is carefully collected to represent language usage for a purpose

Core explanation: Machines process zeros and ones - they learn patterns, statistics, and distributions from text: NLP Technique: TF-IDF; Corpus Dependency: Word frequencies across documents NLP Technique: Word embeddings; Corpus Dependency: Word co-occurrence patterns NLP Technique: Topic models (LDA); Corpus Dependency: Document-word distributions NLP Technique: Language models; Corpus Dependency: Sequential token patterns $\text{TF-IDF}(t, d)$ proportional to $\text{count}(t \text{ in } d) \times \log(N)/(\text{df}(t))$ All statistics originate from the corpus. Any change in corpus changes model behaviour Traditional linguistics emphasises grammatical rules. Modern NLP emphasises usage patterns: Bank disambiguates via co-occurrence (river bank vs investment bank), not explicit rules Frequency and context from the corpus drive meaning - not a hand-crafted lexicon Principle: Garbage corpus → garbage model (GIGO)

Must remember:

- Corpus = large, structured text collection for language study and modelling
- Document < Corpus < Dataset (with labels for tasks)
- Corpora provide language; datasets provide supervision
- TF-IDF, embeddings, topic models, LMs all depend on corpus statistics
- Language learning in NLP is statistical - GIGO applies

Formula / decision rule: $\text{TF-IDF}(t, d)$ proportional to $\text{count}(t \text{ in } d) \times \log(N)/(\text{df}(t))$

Example or contrast: Term: Document; Definition: Single unit of text; Example: One book, article, or review Term: Corpus; Definition: Collection of documents; Example: 500 books; 1 million reviews Term: Dataset; Definition: Corpus organised for a specific task, often with labels; Example: IMDb sentiment dataset; SQuAD QA pairs

Exam trap: Calling a single PDF a corpus - a corpus requires multiple documents Confusing corpus (raw language) with dataset (task-specific labels)

NLP-W04-T03 Types of Text Corpora

Tier B

Definition and why it matters: Concept flow: Text Corpora - General-Purpose - Domain-Specific - Raw / Unannotated - Annotated - Task-Specific Datasets

Core explanation: Not all text corpora serve the same purpose. Genre, annotation level, and domain determine which corpus fits which NLP task. Choosing wrongly leads to poor generalisation and misinterpreted model outputs News language differs from conversational text; literary prose differs from product reviews; raw text differs from labelled annotations Corpus: Brown; Composition: Balanced across news, fiction, reviews, etc; Strengths: Diverse vocabulary; multiple writing styles Corpus: Gutenberg; Composition: Literary classics and books; Strengths: Rich vocabulary; long-form narrative Domain: Sentiment; Corpus Example: Movie reviews (IMDb); Task: Sentiment classification Domain: Clinical; Corpus Example: Medical reports; Task: Clinical NLP Domain: Legal; Corpus Example: Contracts, case law; Task: Contract analysis Designed to represent broad language usage across genres and styles Examples: Gutenberg books; Use Cases: Language modelling Limitation: Poor performance on specialised domains (finance, healthcare, legal) without fine-tuning or domain corpora Examples: Wikipedia dumps; Use Cases: Word embeddings, topic modelling Focused on a particular field or application Examples: POS-tagged Brown corpus; Use: Supervised tagging evaluation Properties: Specialised terminology, narrow context, high task relevance Examples: CoNLL NER datasets; Use: NER training and benchmarking Critical rule: A sentiment model trained on movie reviews will not perform well on medical text Plain text with no linguistic or task labels Task: Sentiment analysis; Dataset: IMDb reviews Task: Question answering; Dataset: SQuAD Suited for unsupervised methods that learn from co-occurrence and distribution alone Task: Text classification; Dataset: AG News, etc Enriched with linguistic or task-specific labels: Sentiment labels (positive / negative / neutral) Used for supervised learning and evaluation

Must remember:

- Five corpus types: general-purpose, domain-specific, raw, annotated, task-specific
- Brown = balanced genres; Gutenberg = literary classics
- Domain corpora essential for finance, health, legal NLP
- Raw corpora for unsupervised methods; annotated for supervised tagging/NER
- Match corpus type to task - wrong corpus → poor generalisation

Example or contrast: Type: General-purpose; Labels: Usually none; Diversity: High; Best For: Foundational NLP, exploration Type: Domain-specific; Labels: Varies; Diversity: Low (within domain); Best For: Specialised applications Type: Raw; Labels: None; Diversity: Varies; Best For: LM, embeddings, topic modelling

Exam trap: Training a medical NER model on Gutenberg literary text Using task-specific datasets for general language modelling - too narrow

NLP-W04-T04 Bias and Representation in Text Corpora

Tier A

Definition and why it matters: A text corpus is written by humans, selected by humans, and collected from specific sources. It naturally reflects:

Core explanation: Social norms of the source community Bias Type: Source bias; Description: Each source has its own lens; Example: News vs social media vs academic text Bias Type: Representation bias; Description: Who appears, how often; Example: Under-represented demographics; absent dialects Bias Type: Historical bias; Description: Old texts reflect past norms; Example: Outdated terminology; stereotypical role language Bias Type: Annotation bias; Description: Human labelers inject subjectivity; Example: Ambiguous cases simplified; cultural framing in labels Concept flow: Source Selection - Corpus - Representation Gaps - Historical Norms - Annotator Judgement - NLP Model - Skewed Predictions Manifestation: Skewed predictions; Example: Model performs worse on under-represented groups NLP models learn directly from corpora - they do not invent bias; they absorb it from training data Manifestation: Stereotypes in embeddings; Example: doctor closer to man; nurse closer to woman Model bias ← Corpus bias Manifestation: Unequal performance; Example: Higher error rates on dialects absent from training data Bias is a data problem before it is a model problem Strategy:

Audit; Action: Explore corpus demographics and term frequencies Strategy: Diversify; Action: Combine multiple sources to balance representation Word embeddings capture historical co-occurrence patterns from text - models optimise statistical regularities, not fairness Strategy: Evaluate per group; Action: Measure performance across subpopulations This is not a bug in the algorithm alone; it reflects patterns in the corpus Strategy: De-bias embeddings; Action: Post-processing or adversarial training (advanced) Before training or deploying any NLP model, ask: Where does this corpus come from? (Source, time period, collection method) Who does it represent well? (Demographics, domains, languages) Who might it miss? (Minority dialects, non-Western contexts, rare entities) Understanding data is as important as understanding model architecture

Must remember:

- Corpora reflect human choices - not neutral ground truth
- Bias types: source, representation, historical, annotation
- Models absorb corpus bias → skewed predictions and stereotyped embeddings
- Ask: where from, who represented, who missed - before deployment
- Bias is a data problem first; understanding corpus blaming model alone

Formula / decision rule: Model bias ← Corpus bias

Exam trap: Claiming "the model is biased" without examining training data first Assuming larger corpora eliminate bias - scale can amplify dominant viewpoints

NLP-W04-T05 Corpus Exploration Setup with NLTK

Tier A

Definition and why it matters: The Gutenberg corpus contains literary classics - novels, poetry, and historical texts Each file ID corresponds to one classic work

Core explanation: Corpus analysis reveals what data is actually available - structure, size, tokenisation, and genre composition. Skipping exploration treats text as a black box, hiding vocabulary gaps, preprocessing needs, and domain mismatch until models fail silently in production How many documents and tokens exist? Is text pre-tokenised or raw? What genres or authors are represented? Concept flow: Download Corpus - List fileids / categories - Load words / sentences - Compute size stats - Deep analysis Step: List contents; Function: `gutenberg.fileids()`; Output: Available works Step: Word tokens; Function: `gutenberg.words(fileid)`; Output: Token list Step: Sentences; Function: `gutenberg.sents(fileid)`; Output: Sentence list Step: Categories; Function: `brown.categories()`; Output: Genre labels NLTK provides pre-tokenised words and sentences - ready for frequency analysis without manual tokenisation The Brown corpus represents modern American English balanced across multiple genres (news, fiction, religion, etc.). Deep analysis follows in a dedicated note; here we confirm availability: Pre-tokenised NLTK corpora include punctuation and capitalisation from original texts. Raw counts differ from cleaned counts - always document whether statistics apply to raw or preprocessed text Code pattern: `import nltk; from nltk.corpus import gutenberg, brown; nltk.download('gutenberg'); nltk.download('brown')` Code pattern: `words = gutenberg.words('carroll-alice.txt'); sents = gutenberg.sents('carroll-alice.txt'); len(words) e.g., 34,110 tokens; len(sents) e.g., 1,703 sentences` Code pattern: `brown.categories()`; `['adventure', 'belleslettres', 'editorial', 'fiction', 'government', 'hobbies', 'humor', 'learned', 'lore', 'mystery', 'news', ...]`

Must remember:

- Explore corpora before modelling - avoid black-box assumptions
- NLTK: gutenberg and brown with `nltk.download()`
- Gutenberg: literary classics; `words()` and `sents()` pre-tokenised
- Brown: genre-balanced modern American English
- Raw corpus tokens include punctuation and capitalisation

Exam trap: Forgetting `nltk.download()` before accessing corpora Confusing word count with unique vocabulary size

Corpus Exploration Setup with NLTK



Vector reconstruction of the source Mermaid visual

Concept map: Corpus Exploration Setup with NLTK

NLP-W04-T06

Gutenberg Corpus Analysis: Vocabulary and Word Frequency

Tier A

Definition and why it matters: Using a single literary text (Alice's Adventures in Wonderland), this analysis explores:

Core explanation: Corpus size (tokens and sentences) Effect: Lowercasing; Impact: Reduces vocabulary inflation (The vs the) Effect of preprocessing on counts Effect: `isalpha()` filter; Impact: Removes punctuation tokens and numbers Vocabulary richness (type-token ratio) Word frequency distributions Content words after stopword removal Raw tokens preserve original capitalisation and punctuation - reflecting the source text, not model-ready form Normalise for statistical analysis (distinct from modelling preprocessing): Type-Token Ratio (TTR): Concept flow: Rank 1", "Rank 2", "Rank 3", "...", "Rank N - 1642, 872, 702, 50, 10 $TTR = (|types|)/(|tokens|) = (2569)/(27333) \approx 0.094$ TTR measures lexical diversity - higher values suggest richer vocabulary relative to text length. Literary texts often show higher TTR than repetitive genres TTR provides intuition, not absolute judgement - compare within similar text lengths and genres Top tokens are function words (stopwords) - low semantic content, high frequency After stopword removal: Code pattern: `from nltk.corpus import gutenberg; fileid = 'carroll-alice.txt'; words = gutenberg.words(fileid); sents = gutenberg.sents(fileid); len(words) 34,110 total tokens; len(sents) 1,703 sentences` Content words reveal thematic signals - character names, plot vocabulary - useful for topic modelling, summarisation, and sentiment analysis Few words dominate (head); a long tail of rare words follows. This power-law pattern appears across virtually all natural language corpora Code pattern: `cleanwords = [w.lower() for w in words if w.isalpha()]; len(cleanwords) 27,333 (down from 34,110)` Matplotlib bar charts of `freq.mostcommon(20)` for raw vs stopword-filtered distributions make the shift from function words to content words visually clear Code pattern: `vocab = set(cleanwords); len(vocab) 2,569 unique words (types)` Code pattern: `from collections import Counter; freq = Counter(cleanwords); freq.mostcommon(10); [('the', 1642), ('to', 872), ('and', 702), ...]` Code pattern: `from nltk.corpus import stopwords; stops = set(stopwords.words('english')); contentwords = [w for w in cleanwords if w not in stops]; Counter(contentwords).mostcommon(10); [('said', ...), ('alice', ...), ('queen', ...), ...]`

Must remember:

- Gutenberg Alice: ~34K raw tokens → ~27K alphabetic lowercase tokens
- Vocabulary: 2,569 unique types; TTR ≈ 0.094 (lexical diversity measure)
- Raw freq dominated by the, to, and; content words emerge after stopword removal
- Few words dominate; long tail of rare words (Zipfian pattern)
- Preprocessing for analysis != preprocessing for modelling - document which applies

Formula / decision rule: $TTR = (|types|)/(|tokens|) = (2569)/(27333) \approx 0.094$

Exam trap: Reporting raw token count without noting punctuation inclusion Confusing types (unique words) with tokens (total count)

NLP-W04-T07

Brown Corpus Analysis: Cross-Genre Language Comparison

Tier A

Definition and why it matters: The Brown corpus differs fundamentally from Gutenberg:

Core explanation: Aspect: Era; Gutenberg: Classic literature; Brown: Modern American English Aspect: Composition; Gutenberg: Single-author books; Brown: Balanced across genres Aspect: Purpose; Gutenberg:

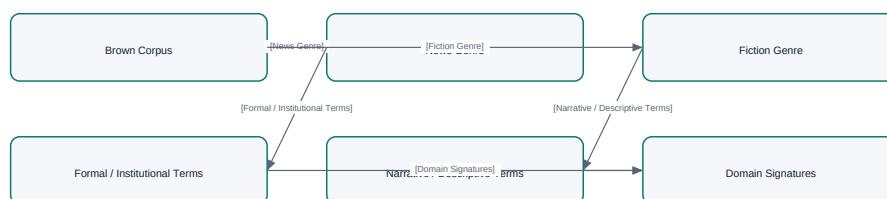
Literary language study; Brown: Representative language usage Aspect: Design; Gutenberg: Author/style-specific; Brown: Multi-genre sampling Genre: News; Tokens (clean): ~11,151; Unique types: ~1,488; TTR: ~0.133 Genre: Fiction; Tokens (clean): ~8,141; Unique types: ~1,161; TTR: ~0.143 Brown enables quantitative comparison of how language varies across domains Each category represents a genre of English - news, fiction, government prose, etc Genre: News; Top content words (examples): fulton, county, grand, jury, investigation, said Genre: Fiction; Top content words (examples): would, said, one, could, like, man, back Token count alone does not explain lexical richness - compare vocabulary metrics next Preprocess: keep alphabetic tokens only Fiction often shows higher lexical diversity (TTR) despite fewer total tokens. News language is more standardised and repetitive - institutional terms recur (said, county, jury) Concept flow: Brown Corpus - News Genre - Fiction Genre - Formal / Institutional Terms - Narrative / Descriptive Terms - Domain Signatures News exhibits institutional/formal vocabulary; fiction uses narrative/descriptive terms - clear domain signatures Domain-specific high-frequency words may lack semantic value: Code pattern: from nltk.corpus import brown; brown.categories(); ['adventure', 'belleslettres', 'editorial', 'fiction', 'government',; 'hobbies', 'humor', 'learned', 'lore', 'mystery', 'news', 'religion',; 'reviews', 'romance', 'sciencefiction', ...]; len(brown.words()) ~1,161,192 total tokens (entire corpus) Custom stopwords lists improve task-specific analysis - said disappears from top-N lists after addition Matplotlib bar charts comparing mostcommon(20) for news vs fiction (after stopwords removal) with rotated x-labels (plt.xticks(rotation=45)) reveal genre-specific frequency profiles Code pattern: newswords = brown.words(categories='news'); fictionwords = brown.words(categories='fiction'); len(newswords) e.g., ~46,000+ tokens (varies by preprocessing); len(fictionwords) e.g., ~68,000+ tokens Same language, different statistics - English words behave differently by genre High-frequency words are domain-specific after stopwords removal Code pattern: def preprocess(words); return [w.lower() for w in words if w.isalpha()]; newsclean = preprocess(newswords); fictionclean = preprocess(fictionwords); newsvocab = set(newsclean); fictionvocab = set(fictionclean); newstr = len(newsvocab) / len(newsclean); fictionttr = len(fictionvocab) / len(fictionclean) TTR and token counts must be interpreted together - not in isolation Custom stopwords refine analysis for specific use cases Code pattern: from collections import Counter; from nltk.corpus import stopwords; stops = set(stopwords.words('english')); def topcontentwords(words, n=10); filtered = [w for w in words if w not in stops]; return Counter(filtered).mostcommon(n) Code pattern: stops.add('said') common in news but uninformative for topic analysis

Must remember:

- Brown: ~1.16M tokens, balanced across 15+ genres of modern American English
- News vs fiction: different token counts, TTR, and content-word profiles
- News → institutional terms; fiction → narrative vocabulary
- TTR: fiction often higher lexical diversity; news more repetitive
- Custom stopwords refine genre analysis; visualise with frequency bar charts

Exam trap: Comparing raw token counts without alphabetic preprocessing Assuming higher token count = richer vocabulary - TTR may disagree

Brown Corpus Analysis: Cross-Genre Language Comparison



Vector reconstruction of the source Mermaid visual

Concept map: Brown Corpus Analysis: Cross-Genre Language Comparison

Week 4 Exam Lens

Likely questions

- Define and explain What Is a Text Corpus and Why It Matters; include the mechanism and one limitation.
- Compare What Is a Text Corpus and Why It Matters with Bias and Representation in Text Corpora and state when each is preferred.
- Apply Corpus Exploration Setup with NLTK to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: Model behaviour = $f(\text{corpus statistics})$; TF-IDF(t, d) proportional to $\text{count}(t \text{ in } d) \times \log(N)/(\text{df}(t))$; Model bias $\leftarrow$ Corpus bias; TTR = $(|\text{types}|)/(|\text{tokens}|) = (2569)/(27333) \approx 0.094$

Mistakes to avoid: Using corpus, document, and dataset interchangeably - each has a distinct definition
Calling a single PDF a corpus - a corpus requires multiple documents
Training a medical NER model on Gutenberg literary text

Week 5: Traditional Word Representations

Week roadmap: Traditional Word Embeddings: Module Overview → One-Hot Encoding for Text → Implementing One-Hot Encoding in Python → Advantages and Limitations of One-Hot Encoding → Bag of Words (BoW) → Implementing Bag of Words with scikit-learn → 4 more topics.

Coverage: 10 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W05-T01 Traditional Word Embeddings: Module Overview

Tier C

Definition and why it matters: Natural language processing models are mathematical engines. They operate on tensors of floating-point values - not on characters, sentences, or meaning in the human sense. Before any classifier, clustering algorithm, or neural network can process text, that text must be converted into a numerical representation. This conversion is called vectorization or word embedding

Core explanation: Consider a cloud-based spam filter at scale: millions of emails arrive per hour. The pipeline cannot "read" each message; it must transform subject lines and body text into feature vectors that a logistic regression model or gradient-boosted tree can score in milliseconds

Must remember:

- NLP models require numerical input; vectorization bridges human language and machine computation
- Statistical NLP converts words to vectors so mathematical operations can be applied
- One-hot encoding, BoW, TF-IDF, and n-grams are the foundational sparse techniques
- The core trade-off is sparsity (memory, dimensionality) versus semantic retention
- Rule-based preprocessing precedes vectorization; they solve different problems
- Dense embeddings (next module) address the semantic gap that sparse methods leave open
- Real-world pipelines (spam filters, search ranking, document classifiers) depend on choosing the right representation for the task

Exam trap: Dimension: Vector size; Sparse methods (OHE, BoW, TF-IDF): Vocabulary size (10,000+); Dense methods (Word2Vec, GloVe): Fixed small dimension (50-300) Dimension: Semantic similarity; Sparse methods (OHE, BoW, TF-IDF): None - all words orthogonal; Dense methods (Word2Vec, GloVe): Captured - similar words cluster

NLP-W05-T02 One-Hot Encoding for Text

Tier A

Definition and why it matters: One-hot encoding (OHE) is the simplest way to represent text numerically. It treats every unique word in a vocabulary as an isolated category - much like encoding a color field with values red, blue, green in a tabular dataset The key idea: a word is either present in a document (1) or absent (0). No partial credit, no frequency count, no semantic relationship between words

Core explanation: Given a corpus of documents, extract every unique word across all sentences. This collection is the vocabulary Sentence: I love NLP Sentence: I love AI Sentence: NLP is the future Concept flow: Corpus of Sentences - Tokenize - Build Vocabulary - unique words - Create Binary Matrix - One row per sentence, one column per word Vocabulary (unique words): AI, I, NLP, future, is, love - six words total Each sentence becomes a row; each vocabulary word becomes a column. Cell values indicate whether that word appears in that sentence (1 = present, 0 = absent) "I love NLP" and "I love AI" share three active dimensions (I, love, and either NLP or AI) Repeated words within a sentence still register as 1 - OHE does not count frequency For vocabulary size $|V|$ and document d , the one-hot vector x_d in $(0, 1)^{|V|}$ satisfies: $x_i = \begin{cases} 1 & \text{if word } w_i \text{ appears in } d \\ 0 & \text{otherwise} \end{cases}$ At most $|V|$ entries can be 1 per sentence (fewer if the sentence is shorter than the vocabulary) OHE works well when: The vocabulary is small (dozens to low hundreds of words) Only presence matters, not frequency or order Interpretability is valued - each dimension has a clear label In cloud ML contexts, OHE appears in feature stores for categorical fields (e.g., product category tags in a recommendation pipeline) and as a baseline text representation before investing in embedding infrastructure

Must remember:

- One-hot encoding represents each word as a binary indicator in a fixed-size vocabulary vector
- Vocabulary = set of all unique words in the corpus

- Each sentence becomes a row of 0s and 1s - 1 where the word is present, 0 otherwise
- OHE is the simplest vectorization method but ignores frequency, order, and semantics
- Works best for small vocabularies and presence-only tasks
- Distinct from BoW (which counts) and TF-IDF (which weights by rarity)

Formula / decision rule: $|V|$; , the one-hot vector

Exam trap: Confusing OHE with BoW - OHE marks presence (0/1); BoW counts frequency (0, 1, 2, ...). "I love love" would be [0,1,0,0,2] in BoW but [0,1,0,0,0,1] in OHE Forgetting vocabulary ordering - columns must stay in a consistent order across all documents; shuffling vocabulary indices breaks the representation

NLP-W05-T03 Implementing One-Hot Encoding in Python

Tier A

Definition and why it matters: Understanding the matrix on paper is one step; implementing it reveals the pipeline every NLP system follows: tokenize → build vocabulary → encode. Manual implementation clarifies what libraries like scikit-learn's CountVectorizer automate under the hood Concept flow: Documents List - Join and Split - Tokenize - Set - Unique Words - Sort - Stable Vocabulary - Zero Vector per Document - Flip 0 to 1 for Present Tokens - DataFrame / Matrix

Core explanation: Code pattern: documents = ["I love NLP",; "I love AI",; "NLP is the future";] Join all sentences, split on whitespace, deduplicate with a set, then sort for reproducibility: Operation: " ".join(documents); Purpose: Flatten corpus into one string Operation: split(); Purpose: Tokenize on whitespace Operation: set(...); Purpose: Remove duplicates Operation: sorted(...); Purpose: Deterministic column order Sentence: I love NLP; Vector: [0, 1, 1, 0, 0, 1] Sentence: NLP is the future; Vector: [0, 0, 1, 1, 1, 0] For each document: Tokenize the sentence Create a zero vector of length len(vocab) Tool: scikit-learn; Class: CountVectorizer(binary=True); Notes: binary=True enforces 0/1 instead of counts For each token present in the vocabulary, set the corresponding index to 1 Tool: pandas; Class: pd.getdummies(); Notes: Works for token-level, not document-level Append the vector to the result list Tool: Keras; Class: keras.utils.to_categorical(); Notes: Common for label encoding, adaptable for tokens Code pattern: vocab = sorted(set(" ".join(documents).split())); ['AI', 'I', 'NLP', 'future', 'is', 'love'] The DataFrame makes the matrix human-readable: rows are sentences, columns are vocabulary words, values are 0 or 1 For large-scale pipelines (e.g., AWS SageMaker feature engineering), pre-built vectorizers handle out-of-vocabulary words, n-grams, and sparse matrix storage automatically Code pattern: onehotvectors = []; for doc in documents;; tokens = doc.split(); vector = [0] len(vocab); for token in tokens;; if token in vocab;; vector[vocab.index(token)] = 1; onehotvectors.append(vector) Code pattern: import pandas as pd; df = pd.DataFrame(onehotvectors, columns=vocab, index=documents)

Must remember:

- OHE implementation: tokenize corpus → build sorted vocabulary → create zero vectors → set 1 for present tokens
- set() removes duplicates; sorted() ensures reproducible column order
- Output is a list of lists (or sparse matrix) with one row per document
- Pandas DataFrame aids visualization with sentences as index and words as columns
- Production systems use CountVectorizer(binary=True) or equivalent for efficiency
- Case handling and tokenization strategy must be decided before vocabulary construction

Formula / decision rule: $O(n)$

Exam trap: Case sensitivity - "NLP" and "nlp" are different tokens unless lowercased during preprocessing Using list.index() in a loop - works for demos but is $O(n)$ per lookup; production code uses a dict mapping word → index

NLP-W05-T04 Advantages and Limitations of One-Hot Encoding

Tier A

Definition and why it matters: One-hot encoding is the "Hello World" of text vectorization. Its strengths are transparency and simplicity; its weaknesses become critical at scale. Choosing OHE without understanding these trade-offs leads to wasted memory, poor model performance, and false confidence in semantic understanding Related words are orthogonal - their dot product is zero:

Core explanation: Advantage: Simplicity; Explanation: Easy to understand, implement, and debug - no training required Advantage: No information loss on presence; Explanation: If a word appears, it is recorded exactly; nothing is averaged or compressed away Advantage: Interpretability; Explanation: Each dimension maps to a known word; feature importance is directly readable Advantage: No training data dependency; Explanation: Vocabulary is built from any corpus instantly - no GPU, no epochs In practice, OHE suits small, controlled vocabularies: intent labels in a chatbot router, product SKU categories in an e-commerce classifier, or keyword flags in a rule-augmented ML pipeline Concept flow: One-Hot Encoding - Simple and interpretable - Exact presence tracking - High sparsity - memory waste - No semantics - orthogonal words - No frequency or order For a vocabulary of 10,000 words, each sentence vector has at most a handful of 1s and 9,995+ zeros. This wastes memory and slows computation Sparsity ratio $\approx 1 - (\text{words per sentence})/(|V|)$ A 10-word sentence in a 10,000-word vocabulary is 99.9% zeros Sentences of different lengths produce vectors of the same fixed size (vocabulary length), but the number of active (non-zero) dimensions varies. Without padding or aggregation strategies, comparing raw sentence vectors across documents of very different lengths can mislead distance-based algorithms "I love love love NLP" and "I love NLP" produce identical vectors - OHE cannot distinguish emphasis or repetition Every new unique word adds a dimension. Open-vocabulary domains (social media, medical notes) make the matrix impractically wide

Must remember:

- OHE advantages: simple, interpretable, no training, preserves word presence exactly
- OHE limitations: extreme sparsity, no semantic similarity, no frequency, no word order
- "Dog" and "puppy" are orthogonal under OHE - the model cannot learn they are related
- A 10,000-word vocabulary means ~9,999 zeros per sentence vector
- OHE suits small vocabularies and presence-only tasks; not large-scale semantic NLP
- BoW and TF-IDF share OHE's semantic blindness but add frequency and weighting respectively

Formula / decision rule: Sparsity ratio $\approx 1 - (\text{words per sentence})/(|V|)$; dog * puppy = 0

Example or contrast: Property: Values; One-Hot: 0 or 1; Bag of Words: Integer counts; TF-IDF: Float weights Property: Frequency; One-Hot: Ignored; Bag of Words: Captured; TF-IDF: Captured + weighted Property: Semantics; One-Hot: None; Bag of Words: None; TF-IDF: None

Exam trap: "OHE is lossless" - true for presence, false overall. Frequency, order, and meaning are all discarded Using OHE for large vocabularies - exam questions may ask when OHE fails; answer: high dimensionality and no semantic similarity

NLP-W05-T05 Bag of Words (BoW)

Tier A

Definition and why it matters: Bag of Words treats a document as a multiset (bag) of its words - like shaking a Scrabble tile bag and counting what fell out. Grammar, syntax, and word order are discarded entirely. What remains is how many times each word appears The name captures the metaphor: tip a document into a bag, shake it, and count the tiles. "The cat sat on the mat" and "mat the on sat cat the" produce identical bags

Core explanation: BoW answers two questions for each word in the vocabulary: Does this word appear in the document? How many times? This is a step beyond one-hot encoding, which only answers the first question Despite its simplicity, BoW remains useful in cloud ML pipelines for: Spam detection - keywords like "OTP", "lottery", "urgent" appear with characteristic frequencies Topic routing - support tickets classified by dominant term counts Baseline models - fast to build, easy to benchmark before investing in embeddings BoW is a strong baseline: if a complex model cannot beat BoW on a keyword-driven task, the added complexity is not justified

Must remember:

- Bag of Words represents a document as word frequency counts over a fixed vocabulary
- Word order and grammar are completely ignored; only multiplicity (count) is preserved
- BoW extends one-hot encoding by recording frequency, not just presence
- Effective for keyword-driven tasks like spam detection and simple classification
- Does not capture semantics - related words remain unrelated in vector space

- Serves as a fast, interpretable baseline in production ML pipelines

Example or contrast: Document: The cat sat on the mat Document: The dog sat on the log Document: Cats and dogs are enemies

Exam trap: "Bag of words preserves multiplicity but not order" - a classic exam phrasing; multiplicity means repeat counts are kept Confusing BoW with TF-IDF - BoW uses raw counts; TF-IDF scales counts by inverse document frequency

NLP-W05-T06 Implementing Bag of Words with scikit-learn

Tier A

Definition and why it matters: Manual BoW implementation mirrors one-hot encoding but increments counters instead of flipping bits. In production, CountVectorizer from scikit-learn handles tokenization, vocabulary building, counting, and sparse matrix storage in a single fittransform call Concept flow: Corpus - Count Vectorizer - fittransform - Sparse Matrix 3 x 12 - Convert to Dense Array - Pandas DataFrame

Core explanation: Code pattern: from sklearn.feature_extraction.text import CountVectorizer; corpus = ["The cat sat on the mat", "The dog sat on the log", "Cats and dogs are enemies"]; Code pattern: vectorizer = CountVectorizer(); X = vectorizer.fittransform(corpus) Word: cat; Count: 1 Word: mat; Count: 1 fittransform performs two operations: Word: on; Count: 1 fit - learns the vocabulary from the corpus Word: sat; Count: 1 transform - converts each document to a count vector Output shape: (3, 12) - 3 documents, 12 unique tokens in vocabulary The result is a compressed sparse row (CSR) matrix - efficient storage when most values are zero Parameter: binary; Default: False; Effect: Set True for one-hot-style presence only BoW vector for "The cat sat on the mat": Parameter: lowercase; Default: True; Effect: Converts all tokens to lowercase Parameter: stopwords; Default: None; Effect: Remove common words ('english' available) Parameter: ngramrange; Default: (1, 1); Effect: Use (1, 2) for unigrams + bigrams Parameter: maxfeatures; Default: None; Effect: Limit vocabulary to top-N frequent terms Parameter: min_df / max_df; Default: 1 / 1.0; Effect: Filter rare or overly common terms The word the appears twice - the distinguishing feature of BoW over one-hot encoding Code pattern: import numpy as np; X.toarray() For a document classification API on Azure ML, maxfeatures=10000 and min_df=2 are common starting points to control dimensionality Code pattern: import pandas as pd; df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out(), index=corpus)

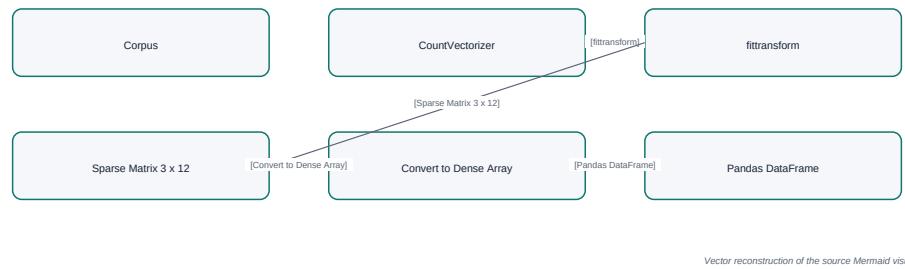
Must remember:

- CountVectorizer from scikit-learn implements BoW via fittransform
- Output is a sparse matrix: rows = documents, columns = vocabulary, values = word counts
- the appearing twice in a sentence yields count 2, not 1
- Convert to DataFrame with get_feature_names_out() for readable inspection
- Key parameters: binary, stopwords, ngramrange, maxfeatures, min_df
- Always fit on training data only; transform test data with the learned vocabulary

Example or contrast: Format: CSR sparse matrix; When to Use: Default; efficient for high-dimensional text Format: Dense array (.toarray()); When to Use: Debugging, small vocabularies, visualization Format: DataFrame; When to Use: Human-readable inspection

Exam trap: Forgetting fit before transform on new data - vocabulary must be learned on training data only; applying a test corpus to fittransform leaks future vocabulary Not lowercasing consistently - CountVectorizer lowercases by default; custom preprocessing that preserves case creates mismatches

Implementing Bag of Words with scikit-learn



Concept map: Implementing Bag of Words with scikit-learn

NLP-W05-T07

Advantages and Limitations of Bag of Words

Tier A

Definition and why it matters: Bag of Words is deceptively powerful for keyword-driven tasks and deceptively weak for anything requiring structure or meaning. Knowing both sides prevents over-engineering (using transformers for spam detection) and under-engineering (using BoW for machine translation)

Core explanation: Advantage: Simple to implement; Explanation: One CountVectorizer call in scikit-learn Advantage: Computationally efficient; Explanation: No training; linear in corpus size Advantage: Surprisingly effective for keyword tasks; Explanation: Spam, topic detection, intent routing Advantage: Interpretable features; Explanation: Each dimension is a word; feature weights are directly explainable Advantage: Frequency signal; Explanation: Unlike OHE, repeated keywords increase the count Concept flow: Bag of Words - Good: spam, topic, keyword tasks - Bad: translation, summarization, negation - Shared weakness: no semantics, high sparsity These two sentences produce identical BoW vectors: "The man bit the dog" The meaning is opposite; the representation is the same. Any task where word order determines semantics (negation, question vs statement, subject-verb-object) fails with BoW Like OHE, BoW vectors have dimension equal to vocabulary size. A 50,000-word vocabulary means 50,000-dimensional vectors that are ~99% zeros. Memory and compute scale poorly without sparse matrix formats "Excellent" and "outstanding" are unrelated dimensions. The model must learn their equivalence purely from co-occurrence patterns in labeled data - with no structural hint Every new unique token (typos, hashtags, product names) expands the matrix. Open-domain text (Twitter, customer reviews) creates unstable, high-dimensional features

Must remember:

- BoW advantages: simple, fast, interpretable, effective for keyword-driven tasks like spam detection
- BoW limitations: no word order, no semantics, high sparsity, vocabulary explosion
- "The man bit the dog" and "the dog bit the man" are identical under BoW
- Frequency counts distinguish BoW from one-hot encoding
- TF-IDF improves BoW by downweighting common words; dense embeddings address semantics
- Choose BoW as a baseline; upgrade when order, negation, or meaning matter

Example or contrast: Spam emails share characteristic vocabulary patterns: High counts of: OTP, lottery, winner, urgent, click, free Low counts of: normal conversational words

Exam trap: "BoW is better than OHE because it counts" - true for frequency, but both lose order and semantics Using BoW for sentiment with negation - "not good" and "good" share the word good; BoW cannot distinguish them without n-grams

NLP-W05-T08

TF-IDF: Term Frequency-Inverse Document Frequency

Tier A

Definition and why it matters: Bag of Words treats every word equally - the and mitochondria both contribute to the vector. But common words carry little discriminative power, while rare, domain-specific terms are highly informative TF-IDF corrects this imbalance by asking two questions:

Core explanation: How often word x appears in a single document d : $TF(x, d) = \text{count of } x \text{ in } d$ A word appearing 5 times in a document is more locally important than one appearing once How rare word x is across all N documents in the corpus: $IDF(x) = \log(N)/(DF(x))$ where $DF(x)$ = number of documents containing word x Word in every document $\rightarrow DF = N \rightarrow \log(N/N) = \log(1) = 0 \rightarrow$ no weight Word in one document $\rightarrow DF = 1$

→ $\log(N/1)$ → maximum weight The log dampens the effect so that very rare words do not dominate excessively
 Document classification - news categorization, legal document routing Information retrieval - search engines rank documents by query term TF-IDF overlap Similarity matching - cosine similarity on TF-IDF vectors for FAQ deduplication in cloud support systems

Must remember:

- TF-IDF = Term Frequency x Inverse Document Frequency
- TF measures how often a word appears in one document
- $IDF = \log(N / DF)$ - rare words get higher weight; universal words get near-zero weight
- Automatically downweights common words like the, is, and
- Boosts domain-specific rare terms like mitochondria in specialized corpora
- Improves on BoW for classification and retrieval but still lacks semantic understanding

Formula / decision rule: appears in a single document; $TF(x, d) = \text{count of } x \text{ in } d$

Example or contrast: Document: The car is fast Document: The car is parked Concept flow: Word x in Document d - TF: count in d - IDF: $\log N / DF$ - Multiply - TF-IDF Score

Exam trap: Confusing TF and IDF - TF is per-document frequency; IDF is corpus-wide rarity. Exam questions often swap them Forgetting the log in IDF - without log, rare words dominate too aggressively

NLP-W05-T09 Implementing TF-IDF in Python

Tier A

Definition and why it matters: Implementing TF-IDF by hand requires computing term counts, document frequencies, and log ratios for every word-document pair. scikit-learn's `TfidfVectorizer` encapsulates this entire pipeline - the same `fittransform` pattern as `CountVectorizer`, but output values are floats, not integers Concept flow: Corpus - `TfidfVectorizer` - `fittransform` - Sparse Matrix - float - DataFrame for inspection

Core explanation: Code pattern: `from sklearn.feature_extraction.text import TfidfVectorizer; import pandas as pd; corpus = [ "The car is fast", "The race car is fast", "The car is parked" ]` Code pattern: `tfidf = TfidfVectorizer(); X = tfidf.fittransform(corpus)` Document: The car is fast; car score: 0.46 Document: The car is parked; car score: 0.41 Output: CSR sparse matrix, dtype=float, shape (3, 6) - 3 documents, 6 vocabulary terms Unlike BoW (integer counts), TF-IDF produces floating-point weights reflecting both local frequency and global rarity Key observation - same word, different scores: The word car receives a different TF-IDF weight in each document because: Parameter: `sublinear_tf`; Effect: Use $1 + \log(TF)$ instead of raw TF - dampens high frequencies Local term frequency differs (race car has car alongside another distinctive term) Parameter: `smooth_idf`; Effect: Adds 1 to DF to avoid division by zero for unseen terms The relative importance of car vs other words in each document differs Parameter: `norm`; Effect: 'l2' (default) - normalizes vectors for cosine similarity Words appearing in all documents (like the, is) receive lower weights due to low IDF Parameter: `use_idf`; Effect: Set False to get raw counts (equivalent to BoW) Parameter: `gram_range`; Effect: Include bigrams/trigrams for partial order capture Code pattern: `df = pd.DataFrame(X.toarray()); columns=tfidf.get_feature_names_out(); index=corpus;` In a document search microservice: fit TF-IDF on the indexed document corpus Transform incoming queries with the same vectorizer Compute cosine similarity between query vector and document vectors Return top-k matches This pattern powers FAQ matching in Azure Bot Service and similar retrieval systems without neural models

Must remember:

- `TfidfVectorizer` from scikit-learn implements TF-IDF via `fittransform`
- Output is a float sparse matrix, not integer counts
- Same word gets different TF-IDF scores in different documents
- Common words (the, is) receive low weights; distinctive words receive high weights
- Fit on training data only; transform new documents with learned vocabulary and IDF
- Default L2 normalization makes vectors ready for cosine similarity comparisons

Formula / decision rule: $1 + \log(TF)$

Example or contrast: Property: Output type; `CountVectorizer`: Integer counts; `TfidfVectorizer`: Float weights Property: Common word handling; `CountVectorizer`: No adjustment; `TfidfVectorizer`: Automatic

downweighting Property: Underlying steps; CountVectorizer: Tokenize + count; TfidfVectorizer: Tokenize + count + TF-IDF scaling

Exam trap: Expecting integer output - TF-IDF values are floats; BoW values are integers Fitting on test data - same leakage risk as BoW; fit only on training corpus

NLP-W05-T10

Advantages and Limitations of TF-IDF

Tier A

Definition and why it matters: TF-IDF is often the strongest sparse vectorization method - it improves on BoW without requiring neural network training. But it inherits fundamental limitations of all count-based methods. Understanding where TF-IDF stops is as important as knowing where it shines "Car" and "automobile" remain orthogonal features. "Happy" and "joyful" have no structural relationship. The model must discover synonymy from label co-occurrence alone

Core explanation: Common words (the, is, and, of) appear in nearly every document, so their IDF approaches zero. TF-IDF reduces their influence without an explicit stopwords removal step Word: the; Typical IDF Behavior: Near 0 - appears everywhere Word: mitochondria; Typical IDF Behavior: High - appears in few biology documents Word: refund; Typical IDF Behavior: Moderate-high - domain-specific to support tickets Concept flow: TF-IDF - Downweights common words - Boosts rare domain terms - No training required - No semantics - High dimensionality Rare but meaningful terms receive high weights. In a biology paper corpus, mitochondria scores far higher than cell (common even within the domain) or the (common everywhere) TF-IDF + linear SVM or logistic regression remains competitive for: Email routing in enterprise helpdesk systems Patent or legal document classification on cloud ML platforms No GPU training, no epochs. Each feature is a word with a float weight - explainable via top-weighted terms per document Vocabulary size equals feature count. A 100,000-term vocabulary produces 100,000-dimensional vectors. Sparse storage mitigates memory, but compute for dense operations remains costly Like BoW, TF-IDF discards sequence. "Not good" and "good" share the feature good unless n-grams are added Words not seen during fit are silently dropped at transform time. Rapidly evolving domains (social media, product catalogs) require periodic vocabulary refresh IDF values depend entirely on the training corpus. A term rare in one corpus may be common in another - TF-IDF weights are not transferable across domains without refitting Upgrade to dense or contextual embeddings when: Synonymy and paraphrase matter (semantic search) Word order and negation are critical (sentiment, NLI) Transfer learning from large pretrained models is available (BERT, sentence-transformers)

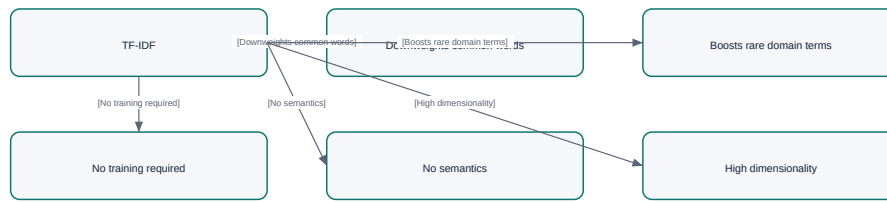
Must remember:

- TF-IDF advantages: auto-downweights common words, boosts domain terms, no training, strong classification baseline
- TF-IDF limitations: no semantics, high dimensionality, no word order, corpus-dependent IDF
- mitochondria scores high in biology corpora; the scores near zero
- TF-IDF is the strongest sparse method but still treats words as independent features
- N-grams can partially recover word order within TF-IDF pipelines
- Dense embeddings (Word2Vec, BERT) address the semantic gap TF-IDF cannot close

Example or contrast: Property: Values; OHE: 0/1; BoW: Integers; TF-IDF: Floats Property: Frequency; OHE: No; BoW: Yes; TF-IDF: Yes (weighted) Property: Stopword handling; OHE: Manual; BoW: Manual; TF-IDF: Automatic (partial)

Exam trap: "TF-IDF captures semantics" - false. It captures statistical importance, not meaning "Stopword removal is unnecessary with TF-IDF" - mostly true but not absolute; explicit removal can still improve results

Advantages and Limitations of TF-IDF



Vector reconstruction of the source Mermaid visual

Concept map: Advantages and Limitations of TF-IDF

Week 5 Exam Lens

Likely questions

- Define and explain One-Hot Encoding for Text; include the mechanism and one limitation.
- Compare One-Hot Encoding for Text with Implementing One-Hot Encoding in Python and state when each is preferred.
- Apply Advantages and Limitations of One-Hot Encoding to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $|V|$; , the one-hot vector; $O(n)$; Sparsity ratio $\sim 1 - (\text{words per sentence})/(|V|)$; $\text{dog} * \text{puppy} = 0$; appears in a single document; $\text{TF}(x, d) = \text{count of } x \text{ in } d$; $1 + \log(\text{TF})$

Mistakes to avoid: Dimension: Vector size; Sparse methods (OHE, BoW, TF-IDF): Vocabulary size (10,000+); Dense methods (Word2Vec, GloVe): Fixed small dimension (50-300) Confusing OHE with BoW - OHE marks presence (0/1); BoW counts frequency (0, 1, 2, ...). "I love love" would be [0,1,0,0,0,2] in BoW but [0,1,0,0,0,1] in OHE Case sensitivity - "NLP" and "nlp" are different tokens unless lowercased during preprocessing

Week 6: Modern Word Embeddings

Week roadmap: Dense Word Embeddings: Module Overview → Word2Vec: Learning Word Embeddings → Implementing Word2Vec with Gensim → GloVe: Global Vectors for Word Representation → Implementing GloVe with Gensim → Visualizing Word Embeddings → 1 more topics.

Coverage: 7 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W06-T01 Dense Word Embeddings: Module Overview

Tier C

Definition and why it matters: Sparse methods (one-hot, BoW, TF-IDF) treat every word as an isolated symbol. The vectors for car and automobile share no dimensions and have zero similarity - even though humans know they mean nearly the same thing. This semantic gap limits every downstream task that depends on word relatedness

Core explanation: Issue: Orthogonality; Example: dog and puppy have dot product = 0 Issue: Dimensionality; Example: 10,000-word vocabulary - 10,000-dim vectors, 99.9% zeros

Must remember:

- Dense embeddings map words to low-dimensional continuous vectors (50-300 dims)
- Similar contexts → similar vectors (distributional hypothesis)
- Solves the semantic gap: car $\sim$ automobile in vector space
- Vector arithmetic captures relationships: king - man + woman $\sim$ queen
- Word2Vec (neural) and GloVe (co-occurrence matrix) are the two main static methods
- This module transitions from frequency counting to semantic representation

Formula / decision rule: $v_{\text{king}} = [0.2, -0.5, 0.8, \dots]$ in $\mathbb{R}^d$; $v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \sim v_{\text{queen}}$

Exam trap: Confusing "embedding" with only dense vectors - TF-IDF is also called a word embedding in the broad sense Assuming all embeddings are contextual - Word2Vec and GloVe produce one fixed vector per word (static)

NLP-W06-T02 Word2Vec: Learning Word Embeddings

Tier A

Definition and why it matters: Word2Vec learns word meanings by training a shallow neural network on a simple task: predict a missing word from its neighbors (or predict neighbors from a word). To solve this prediction efficiently, the network compresses word meanings into dense vectors in its hidden layer. These hidden-layer weights become the embeddings Architecture: CBOW (Continuous Bag of Words); Input: Context words; Output: Target word; Task: Fill in the blank

Core explanation: Continuous Bag of Words averages context word vectors and predicts the center word Example sentence: "The cat sat on the mat" Input (context): the, cat, on, mat Output (target): sat Like a fill-in-the-blanks game: given surrounding words, predict the missing center word CBOW is faster to train because it predicts one word from many context inputs averaged together Skip-gram takes the center word and predicts each surrounding context word Input (target): sat Output (context): the, cat, on, mat Skip-gram is slower but often performs better on rare words because it generates more training examples per word Each word is initially assigned a random vector The network adjusts vectors to minimize prediction error Words that share contexts (e.g., cat and dog both follow the) move closer in vector space After training, the hidden-layer weight matrix rows are the word embeddings $\mathbf{v}_w$ in $\mathbb{R}^d$, d in $[50, 300]$ Pretrained Word2Vec models power: Semantic similarity in recommendation systems (similar product descriptions) Feature initialization for downstream NLP models Vocabulary expansion in search (find related terms) In Gensim, pretrained models like word2vec-google-news-300 provide 3 million words in 300 dimensions - ready for cosine similarity queries without training from scratch

Must remember:

- Word2Vec learns dense embeddings via a shallow neural network trained on word prediction
- CBOW: context words → predict target word (fill in the blank)

- Skip-gram: target word $\rightarrow$ predict context words (opposite direction)
- Similar contexts produce similar vectors (distributional hypothesis)
- CBOW is faster; Skip-gram handles rare words better
- Embeddings live in the hidden layer weights; typical dimension: 50-300

Formula / decision rule: v_w in $\mathbb{R}^d$, d in $[50, 300]$

Example or contrast: Criterion: Direction; CBOW: Context - target; Skip-gram: Target - context Criterion: Speed; CBOW: Faster; Skip-gram: Slower Criterion: Rare words; CBOW: Less effective; Skip-gram: Better

Exam trap: Confusing CBOW and Skip-gram direction - CBOW: many $\rightarrow$ one; Skip-gram: one $\rightarrow$ many "Word2Vec is a deep network" - it is a shallow (single hidden layer) network

NLP-W06-T03 Implementing Word2Vec with Gensim

Tier A

Definition and why it matters: Conceptual understanding of CBOW and Skip-gram is necessary; training a model on a real corpus reveals how hyperparameters control context window, dimensionality, and vocabulary filtering. Gensim provides a production-ready Word2Vec implementation Concept flow: Corpus - Preprocess - lowercase, tokenize - Word2Vec model - Trained Embeddings - Query: similarity, analogy

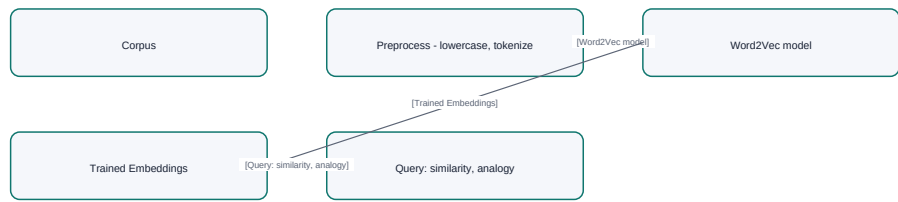
Core explanation: Code pattern: `pip install gensim; from gensim.models import Word2Vec` Code pattern: `corpus = [ "I love machine learning"; "Machine learning is fascinating"; "I love dogs"; more sentences; ]`; Preprocess: `lowercase + tokenize; tokenizedcorpus = [sentence.lower().split() for sentence in corpus]` Parameter: `sentences`; Value: `tokenized corpus`; Meaning: Training data Parameter: `vectorsize`; Value: `50`; Meaning: Embedding dimensionality (d) Parameter: `window`; Value: `3`; Meaning: Context window: 3 words before and after the target Parameter: `mincount`; Value: `1`; Meaning: Ignore words appearing fewer than this many times Parameter: `workers`; Value: `4`; Meaning: Number of CPU threads for parallel training Gensim expects a list of lists - each inner list is a tokenized sentence Window example: for the word learning in "I am learning machine learning love": Window = 3 looks at 3 words before and 3 words after Approach: Train from scratch; When: Domain-specific corpus (medical, legal, internal docs) Context includes: am, machine, love, etc Approach: Load pretrained; When: General English (Google News 300, fastText) Approach: Fine-tune; When: Start pretrained, continue training on domain data Similarity quality depends on: Corpus size - small corpora produce noisy similarities Hyperparameters - window size, vector dimension, mincount Domain relevance - a general corpus won't capture specialized terminology Code pattern: `model = Word2Vec(; sentences=tokenizedcorpus,; vectorsize=50,; window=3,; mincount=1,; workers=4; )` On a small demo corpus, machine may be similar to learning (co-occurrence) but also spuriously to unrelated words - this is expected with insufficient data For cloud deployments, pretrained embeddings reduce training cost; custom training is justified when domain vocabulary diverges significantly from general English Code pattern: `model.wv['machine']; array([0.02, -0.15, 0.33, ...])` - 50-dimensional Code pattern: `model.wv.mostsimilar('machine', topn=3); [('learning', 0.85), ('dog', 0.42), ('love', 0.38)]`

Must remember:

- Gensim Word2Vec trains embeddings from a tokenized corpus (list of lists)
- Key hyperparameters: `vectorsize` (dims), `window` (context radius), `mincount`, `workers`
- `model.wv['word']` returns the embedding vector; `mostsimilar()` finds nearest neighbors
- Preprocess with lowercase and whitespace tokenization before training
- Similarity quality scales with corpus size and domain coherence
- Pretrained models are preferred for general English; train custom for specialized domains

Exam trap: Not lowercasing - Machine and machine become separate vocabulary entries Passing raw strings instead of tokenized lists - Gensim requires `List[List[str]]`

Implementing Word2Vec with Gensim



Vector reconstruction of the source Mermaid visual

Concept map: Implementing Word2Vec with Gensim

NLP-W06-T04

GloVe: Global Vectors for Word Representation

Tier A

Definition and why it matters: Word2Vec learns from local context windows - it sees a few neighboring words at a time. GloVe (Global Vectors) takes a different approach: build a global co-occurrence matrix counting how often every word pair appears together across the entire corpus, then factorize this matrix to produce word vectors. GloVe combines the strengths of count-based methods (global statistics) and prediction-based methods (dense vectors).

Core explanation: Aspect: Training signal; Word2Vec: Local context windows; GloVe: Global co-occurrence matrix. Aspect: Architecture; Word2Vec: Shallow neural network; GloVe: Matrix factorization. Aspect: Scope; Word2Vec: Neighborhood around each word; GloVe: Entire corpus statistics. Aspect: Pretrained availability; Word2Vec: Google News, etc; GloVe: Wikipedia, Twitter, Common Crawl. Concept flow: Entire Corpus - Co-occurrence Matrix - Matrix Factorization - Word Vectors. For vocabulary of size $|V|$, construct a $|V| \times |V|$ matrix X where: $X(i,j)$ = number of times word j appears in the context of word i . This matrix captures global word-word relationships - not just immediate neighbors. Matrix factorization decomposes X into lower-dimensional word vectors w_i and w_j such that: $w_i^T w_j \approx \log X(i,j)$. Model: glove-wiki-gigaword-100; Dimensions: 100; Corpus: Wikipedia + Gigaword. Model: glove-twitter-25; Dimensions: 25; Corpus: Twitter data. Model: glove-840B-300d; Dimensions: 300; Corpus: Common Crawl (840B tokens). Pretrained GloVe models are ideal for quick prototyping - download and query without training.

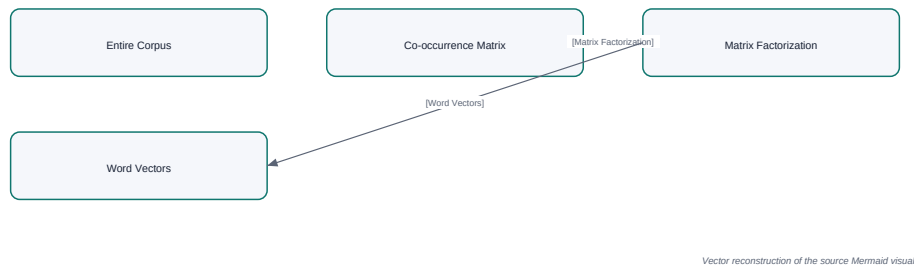
Must remember:

- GloVe = Global Vectors for Word Representation
- Builds a global word co-occurrence matrix, then factorizes it into dense vectors
- Ratio of co-occurrence probabilities encodes semantic relationships
- Combines count-based global statistics with dense vector output
- Pretrained models available (Wikipedia, Twitter, Common Crawl)
- Static embeddings - one vector per word, no context sensitivity

Formula / decision rule: $|V|$; $|V| \times |V|$

Example or contrast: Criterion: Training data efficiency; GloVe: Uses global counts (efficient); Word2Vec: Requires many window passes. Criterion: Rare words; GloVe: Good (global counts help); Word2Vec: Skip-gram better for rare words. Criterion: Pretrained models; GloVe: Widely available (Gensim, spaCy); Word2Vec: Widely available.

Exam trap: "GloVe is a neural network" - it uses matrix factorization, not backpropagation through a prediction task. Confusing local vs global - Word2Vec = local windows; GloVe = global co-occurrence.

GloVe: Global Vectors for Word Representation

Concept map: GloVe: Global Vectors for Word Representation

NLP-W06-T05**Implementing GloVe with Gensim****Tier A**

Definition and why it matters: Training GloVe from scratch requires building a co-occurrence matrix over a large corpus - computationally expensive. Gensim provides pretrained GloVe models via its downloader API, enabling immediate similarity and analogy queries without training. Concept flow: pip install gensim - Download Pretrained Model - Load KeyedVectors - similarity / mostsimilar

Core explanation: Code pattern: pip install gensim; import gensim.downloader as api; Download pretrained model (one-time); glove_model = api.load("glove-twitter-25") Model Name: glove-twitter-25; Dimensions: 25; Training Data: Twitter Model Name: glove-wiki-gigaword-50; Dimensions: 50; Training Data: Wikipedia + Gigaword Model Name: glove-840B-300d; Dimensions: 300; Training Data: Common Crawl (840B tokens) For production semantic search, glove-wiki-gigaword-100 or glove-840B-300d offer better quality than the small Twitter model Cosine similarity ranges from -1 to 1; values near 1 indicate strong semantic relatedness Gensim supports analogy queries via mostsimilar: This implements the vector arithmetic: $v(\text{king}) - v(\text{man}) + v(\text{woman}) \approx v(\text{queen})$ How it works: positive words are added to the query vector negative words are subtracted The model returns words whose vectors are closest to the resulting vector Code pattern: glove_model.similarity("king", "queen"); ~0.92 - high similarity (royalty, gender-related concepts) Code pattern: glove_model.similarity("king", "car"); ~0.67 - lower similarity (unrelated domains) Code pattern: glove_model.mostsimilar(positive=["woman", "king"], negative=["man"], topn=1); [(queen, 0.85)]

Must remember:

- Gensim api.load() downloads and loads pretrained GloVe models
- similarity(w1, w2) returns cosine similarity between two word vectors
- mostsimilar(positive=[...], negative=[...]) solves analogy tasks via vector arithmetic
- king - man + woman ≈ queen is the canonical analogy example
- Pretrained models avoid training cost; choose dimensionality based on task requirements
- Handle out-of-vocabulary words explicitly in production code

Formula / decision rule: $v(\text{king}) - v(\text{man}) + v(\text{woman}) \approx v(\text{queen})$

Example or contrast: Code pattern: glove_model.mostsimilar(positive=["woman", "man"], negative=["king"], topn=1); Returns a word in the direction of "commoner" or similar Approach: Pretrained GloVe; Code: api.load("glove-wiki-gigaword-100"); Training Time: None (download only); Quality: High for general English Approach: Train Word2Vec; Code: Word2Vec(sentences=...); Training Time: Minutes to hours; Quality: Depends on corpus

Exam trap: Using a tiny model for serious tasks - glove-twitter-25 (25 dims) is for demos; production needs 100-300 dims Words not in vocabulary - KeyError if the word was not in training data; handle with try/except or model.keytoindex check

NLP-W06-T06**Visualizing Word Embeddings****Tier A**

Definition and why it matters: Word embeddings live in 50-300 dimensions - impossible to visualize directly. Dimensionality reduction projects these high-dimensional vectors onto a 2D plane, revealing clusters of semantically related words. Seeing king, queen, man, woman grouped together makes the abstract concept

of embedding space concrete Concept flow: Word List - Lookup Embedding Vectors - PCA to 2D - Scatter Plot

Core explanation: Without visualization: Vectors are abstract number arrays; With visualization: Clusters reveal semantic groups Without visualization: Similarity is a cosine score; With visualization: Proximity on a plot is intuitive Without visualization: Hard to debug bad embeddings; With visualization: Outliers and noise become visible Method: PCA; Speed: Fast; Preserves: Global variance; Best for: Quick exploration Method: t-SNE; Speed: Slow; Preserves: Local neighborhoods; Best for: Cluster visualization In ML experimentation (e.g., comparing custom-trained embeddings vs pretrained), 2D plots provide a quick sanity check before deploying to production Method: UMAP; Speed: Moderate; Preserves: Both local and global; Best for: Publication-quality plots Cluster: Royalty/gender; Words: king, queen, man, woman; Why close: Shared gender and authority attributes Each vector has dimension equal to the model's vectorsize (e.g., 25 for glove-twitter-25) Cluster: Food; Words: apple, orange, fruit; Why close: Culinary co-occurrence Cluster: Transport; Words: car, bus; Why close: Vehicle domain PCA (Principal Component Analysis) finds the two directions of maximum variance in the high-dimensional data and projects onto them PCA is preferred for quick inspection; t-SNE better reveals tight clusters at the cost of global structure distortion Words in the same semantic domain cluster together; unrelated domains occupy different regions of the plot Distance $\sim$ dissimilarity - but only approximately; PCA loses information Code pattern: `import gensim.downloader as api; glove_model = api.load("glove-twitter-25"); words = ["king", "queen", "man", "woman", "apple", "orange", "fruit", "car", "bus";] Cluster tightness - reflects consistency of co-occurrence patterns in training data Code pattern: vectors = [glove_model[word] for word in words] Outliers - words with unusual usage patterns or polysemy may appear misplaced Code pattern: from sklearn.decomposition import PCA; pca = PCA(n_components=2); reduced = pca.fit_transform(vectors) Code pattern: import matplotlib.pyplot as plt; plt.figure(figsize=(10, 8)); plt.scatter(reduced[:, 0], reduced[:, 1]); for i, word in enumerate(words): plt.annotate(word, (reduced[i, 0], reduced[i, 1])); plt.title("Word Embeddings in 2D (PCA)"); plt.xlabel("PC1"); plt.ylabel("PC2")`

Must remember:

- High-dimensional embeddings (50-300D) require dimensionality reduction for visualization
- PCA projects vectors to 2D by preserving maximum variance directions
- Semantic clusters emerge: royalty words together, food words together, transport words together
- Pipeline: load model → extract vectors → PCA → scatter plot with labels
- t-SNE is an alternative that better preserves local neighborhoods
- 2D plots are for exploration; use cosine similarity in original space for production

Exam trap: Treating 2D distances as exact - PCA/t-SNE distort high-dimensional relationships; cosine similarity in original space is authoritative Too few words - clusters are meaningless with 3-4 words; use 20+ for meaningful patterns

NLP-W06-T07 Static vs Contextual Word Embeddings

Tier A

Definition and why it matters: Human language is understood through meaning; machines process numbers. Word embeddings bridge this gap by converting each word into a mathematical vector - a list of real numbers that encodes semantic properties

Core explanation: The numbers are not arbitrary: they capture the essence of the word so that mathematical operations reveal relationships humans recognize intuitively Property: Has authority; king: 1; queen: 1 Property: Gender (0=male, 1=female); king: 0; queen: 1 Property: Is rich; king: 1; queen: 1 Model: Custom toy model; Typical Dimensions: 3-8 Model: GloVe; Typical Dimensions: 100-300 Model: Word2Vec; Typical Dimensions: 300-500 Imagine encoding words by listing observable properties: Model: BERT-base; Typical Dimensions: 768 Model: BERT-large; Typical Dimensions: 1024 Model: OpenAI text-embedding-small; Typical Dimensions: 1536 Add more properties (has moustache, war experience, trained in cooking) and the vectors grow in dimensionality. Different people might choose different properties - the scheme is arbitrary, but the principle is universal: encode meaning as numbers along meaningful dimensions Real models (Word2Vec, GloVe, BERT) learn hundreds of latent properties automatically - we do not know what each dimension represents, but we know the vectors work More dimensions capture finer-grained meaning but increase compute and storage cost $v(\text{king}) - v(\text{man}) + v(\text{woman}) \sim v(\text{queen})$ Intuition: subtract "maleness" from

"king" (leaving royalty/crown), add "femaleness" $\rightarrow$ arrive at "queen" Verification with property vectors: Concept flow: man - woman - king - queen $\text{begin}(\text{bmatrix})101\text{end}(\text{bmatrix}) - \text{begin}(\text{bmatrix})0.100.2\text{end}(\text{bmatrix}) + \text{begin}(\text{bmatrix})0.110.2\text{end}(\text{bmatrix}) = \text{begin}(\text{bmatrix})111\text{end}(\text{bmatrix}) \sim= v(\text{queen})$ $v(\text{Delhi}) - v(\text{India}) + v(\text{Russia}) \sim= v(\text{Moscow})$ Property: Vectors per word; Static (Word2Vec, GloVe): 1 Remove "India" (leaving "capital-of"), apply to "Russia" $\rightarrow$ "Moscow" Property: Context sensitivity; Static (Word2Vec, GloVe): None The difference between two word vectors encodes a relationship: Property: Polysemy handling; Static (Word2Vec, GloVe): Poor $r(\text{gender}) = v(\text{woman}) - v(\text{man})$ Property: Training; Static (Word2Vec, GloVe): Corpus-wide co-occurrence or prediction Transposing this relationship to royalty: Property: Speed; Static (Word2Vec, GloVe): Fast lookup $v(\text{king}) + r(\text{gender}) \sim= v(\text{queen})$ Similarly, $v(\text{Delhi}) - v(\text{India})$ encodes the "capital city" relationship, which transposes to Moscow - Russia Property: Vectors per word; Contextual (BERT, GPT): One per occurrence Definition: Each word has exactly one fixed vector regardless of context Property: Context sensitivity; Contextual (BERT, GPT): Full Property: Polysemy handling; Contextual (BERT, GPT): Excellent Property: Training; Contextual (BERT, GPT): Massive neural networks Property: Speed; Contextual (BERT, GPT): Slower (requires full sentence) Context level: dish alone; Embedding shifts toward: Ambiguous (food or utensil) The word bank has one vector whether the sentence discusses a river bank or a financial bank. The model cannot distinguish meanings without surrounding context Context level: rice dish; Embedding shifts toward: Rice-based foods (biryani, risotto, kheer) Definition: The vector for a word changes based on its surrounding context in the sentence Context level: Indian rice dish with spices and meat; Embedding shifts toward: Biryani Concept flow: dish - rice dish - Indian rice dish - biryani Consider: "I ate the dish last night." Each additional context word shifts the embedding through relationship vectors in space: The final contextual embedding for dish in this sentence is near biryani, not risotto or pizza Example: Two people with different food preferences:

Must remember:

- Word embeddings convert words to numerical vectors that encode meaning
- Embeddings are created by encoding properties as dimensions; real models learn latent properties automatically
- Vector arithmetic captures relationships: king - man + woman $\sim=$ queen
- Relationship vectors (gender, capital-city) transpose across word pairs
- Static embeddings: one fixed vector per word (Word2Vec, GloVe); fail on polysemy
- Contextual embeddings: vectors change with sentence context (BERT, GPT); resolve ambiguity
- "dish" $\rightarrow$ "Indian rice dish with spices and meat" $\rightarrow$ biryani demonstrates contextual resolution
- BERT-base: 768 dims; GPT embeddings: up to 1536 dims

Formula / decision rule: king $\rightarrow [0.2, -0.5, 0.8, \dots]$; $v_(\text{king}) = [1, 0, 1]$

Example or contrast: Criterion: Vectors per word; Static: 1 (fixed); Contextual: Context-dependent Criterion: Polysemy; Static: Cannot resolve; Contextual: Resolves via context Criterion: Model examples; Static: Word2Vec, GloVe; Contextual: BERT, GPT, ELMo

Exam trap: "All embeddings are contextual" - Word2Vec and GloVe are static; only one vector per word Confusing dimensions with properties - we know dimension counts (768 for BERT) but not what each dimension means

Week 6 Exam Lens

Likely questions

- Define and explain Word2Vec: Learning Word Embeddings; include the mechanism and one limitation.
- Compare Word2Vec: Learning Word Embeddings with Implementing Word2Vec with Gensim and state when each is preferred.
- Apply GloVe: Global Vectors for Word Representation to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $v_{\text{king}} = [0.2, -0.5, 0.8, \dots]$ in $\mathbb{R}^d$; $v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}$; v_w in $\mathbb{R}^d$, d in $[50, 300]$; $|V|$; $|V| \times |V|$; $\text{king} \rightarrow [0.2, -0.5, 0.8, \dots]$; $v_{\text{king}} = [1, 0, 1]$

Mistakes to avoid: Confusing "embedding" with only dense vectors - TF-IDF is also called a word embedding in the broad sense Confusing CBOW and Skip-gram direction - CBOW: many $\rightarrow$ one; Skip-gram: one $\rightarrow$ many Not lowercasing - Machine and machine become separate vocabulary entries

Week 7: Sequential Modeling and Recurrent Networks

Week roadmap: Sequential Modelling: Module Overview → Sequence-to-Sequence (Seq2Seq) Modelling → Recurrent Neural Networks (RNNs) → RNN Architecture Patterns: Many-to-One, One-to-Many, Many-to-Many → The Vanishing and Exploding Gradient Problem → Long Short-Term Memory (LSTM) Networks → 1 more topics.

Coverage: 7 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W07-T01 Sequential Modelling: Module Overview

Tier C

Definition and why it matters: Previous vectorization methods - TF-IDF, feedforward neural networks - treat text as a fixed-size feature vector. Two fundamental problems make this inadequate for real language understanding: Fixed input size - vocabulary-sized vectors cannot naturally handle a 3-word sentence and a 1000-word paragraph differently

Core explanation: Method: BoW / TF-IDF; Input constraint: Vector length = vocabulary size (fixed) Method: Feedforward NN; Input constraint: Input layer has fixed number of neurons

Must remember:

- Two problems with prior methods: fixed input size and loss of word order
- BoW treats "the dog bit the man" and "the man bit the dog" as identical
- Sequential models process variable-length text token by token
- Temporal order is preserved via hidden state memory
- "not good" != "good" - negation requires sequence awareness
- This module covers Seq2Seq, RNN, LSTM, GRU, and vanishing gradients

Exam trap: "TF-IDF handles variable length" - the vector size is fixed (vocabulary length); only the number of non-zero entries varies Confusing sequential models with n-grams - n-grams capture local order in a fixed window; RNNs capture order across the full sequence

NLP-W07-T02 Sequence-to-Sequence (Seq2Seq) Modelling

Tier A

Definition and why it matters: Many NLP tasks transform an input sequence into an output sequence of different length and content: English sentence → French sentence (translation)

Core explanation: Concept flow: Token 1 - Token 2 - Token 3 - Context Vector - Decoder - Output 1 - Output 2 - Output 3 Consumes the input sequence one token at a time Progressively updates its internal state Compresses the entire input meaning into a single fixed-size vector: the context vector (or final hidden state) Analogy: reading an entire book and forming one mental summary Receives the context vector as its starting point Generates the output sequence one token at a time Each generated token feeds back as input for the next step Analogy: taking that mental summary and writing it in a different language Input length: Short (5 words); Challenge: Context vector is sufficient Input length: Long (500 words); Challenge: Model forgets the beginning by the time it reaches the end Input length: Very long (5000 words); Challenge: Severe information loss Concept flow: Long Input Sequence - Encoder - Single Context Vector - bottleneck - Decoder - Output may miss early information Architecture: Many-to-one; Input: Sequence; Output: Single label; Example: Sentiment analysis Architecture: One-to-many; Input: Single input; Output: Sequence; Example: Image captioning Architecture: Many-to-many (synced); Input: Sequence; Output: Sequence (aligned); Example: POS tagging The encoder must compress everything about a long input - grammar, tone, facts, entities - into a single fixed-size context vector Architecture: Seq2Seq (many-to-many); Input: Sequence; Output: Sequence (unaligned); Example: Translation This bottleneck is the fundamental limitation of basic Seq2Seq and motivates attention mechanisms (covered in later modules), which allow the decoder to look back at all encoder states rather than relying on one compressed vector Seq2Seq is the general many-to-many case where input and output lengths need not match

Must remember:

- Seq2Seq converts input sequences to output sequences in a different domain

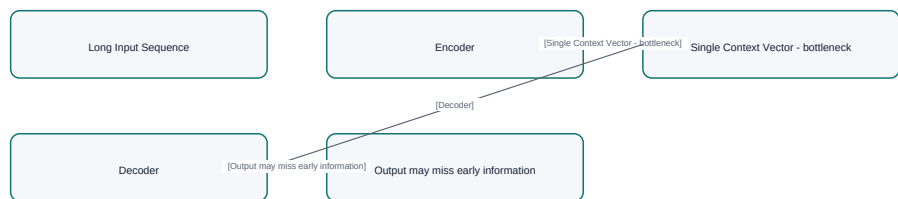
- Encoder-decoder architecture: encoder compresses input $\rightarrow$ context vector $\rightarrow$ decoder generates output
- Applications: translation, summarization, speech recognition
- Encoder processes one token at a time; decoder generates one token at a time
- Information bottleneck: single context vector cannot capture very long inputs
- Attention mechanisms (future topic) address the bottleneck by allowing selective lookback

Formula / decision rule: $h_1; h_2$

Example or contrast: Input: "I am fine" Step: 1; Encoder reads: I; Hidden state updates: h_1 Step: 3; Encoder reads: fine; Hidden state updates: h_3 (context vector)

Exam trap: Confusing encoder and decoder roles - encoder reads input; decoder generates output "Context vector captures everything perfectly" - false for long sequences; this is the information bottleneck

Sequence-to-Sequence (Seq2Seq) Modelling



Vector reconstruction of the source Mermaid visual

Concept map: Sequence-to-Sequence (Seq2Seq) Modelling

NLP-W07-T03

Recurrent Neural Networks (RNNs)

Tier A

Definition and why it matters: A standard feedforward neural network maps input X directly to output Y - each input is processed independently with no memory of previous inputs. Language requires memory: the meaning of "bank" depends on whether the previous word was "river" or "financial" RNNs introduce a feedback loop: the hidden state from the previous time step feeds into the current time step, creating persistent memory across the sequence

Core explanation: At each time step t : Input: X_t (current token) and $H(t-1)$ (previous hidden state / memory) Symbol: X_t ; Meaning: Input at current time step Symbol: $H(t-1)$; Meaning: Previous memory (hidden state) Symbol: H_t ; Meaning: Updated memory Symbol: $W(hh)$, $W(xh)$, $W(hy)$; Meaning: Weight matrices (shared across all time steps) Symbol: $\tanh$; Meaning: Activation function (bounds values to $[-1, 1]$) Output: Y_t (current prediction) and H_t (updated hidden state) $Y_t = W(hy) H_t + b_y$ Concept flow: X_0 - RNN - Y_0 - X_1 - RNN - Y_1 - X_2 - RNN - Y_2 As the RNN reads word by word, it updates this bucket Property: Same weights at every step; Implication: Position-invariant processing rules New information is added; old information may fade Property: Parameter efficiency; Implication: Few weights regardless of sequence length At step t , the network decides based on: Property: Generalization; Implication: Rules learned at position 3 apply at position 300 Current input X_t Current context H_t (being computed) Past context $H(t-1)$ A 5-word sentence produces a chain of 5 identical RNN cells: Each cell uses the same weight matrices ($W(xh)$, $W(hh)$, $W(hy)$) - the model learns one general rule for processing language regardless of position in the sequence This is analogous to applying the same reading comprehension strategy to every word in a sentence Basic Seq2Seq encoders are RNNs. The final hidden state H_T serves as the context vector for the decoder. For short sequences, this works well. For long sequences, early information in H_0 , H_1 , gets overwritten by later inputs - the memory bucket overflows This limitation leads to LSTM and GRU architectures (covered next)

Must remember:

- RNNs add a feedback loop: previous hidden state $H(t-1)$ feeds into current step
- $H_t = \tanh(W(hh) H(t-1) + W(xh) X_t + b_h)$
- Hidden state H_t is the network's memory bucket
- Unrolled RNN = chain of identical cells sharing weights

- Same weight matrices at every time step - position-invariant processing
- Vanilla RNNs handle short context but fail on long-range dependencies (vanishing gradients)

Formula / decision rule: directly to output; $X \rightarrow H \rightarrow Y$

Example or contrast: Feedforward NN: $X - H - Y$; RNN: $X_t + H_{(t-1)} - H_t - Y_t$ Feedforward NN: No memory between inputs; RNN: Hidden state carries past context Feedforward NN: Fixed input size; RNN: Variable sequence length

Exam trap: "RNN has different weights at each step" - false; weights are shared (tied) across time steps
Confusing H_t and Y_t - H_t is internal memory; Y_t is the output prediction

NLP-W07-T04 RNN Architecture Patterns: Many-to-One, One-to-Many, Many-to-Many

Tier A

Definition and why it matters: A single RNN cell can be wired differently depending on the task. The naming convention describes how many input time steps and how many output time steps the architecture uses: Input tokens - RNN - Output tokens

Core explanation: Concept flow: Many-to-One - Sentiment Analysis - One-to-Many - Image Captioning - Many-to-Many - Translation - Synced Many-to-Many - Video Labeling
Input: sequence of tokens $\rightarrow$ Output: single label
Component: Input; Description: Multiple words (sequence)
Component: RNN; Description: Processes all tokens, final hidden state summarizes
Component: Output; Description: One prediction (class label, score)
Component: Input; Description: One vector (image features, seed token)
Component: RNN; Description: Generates tokens sequentially from initial state
Component: Output; Description: Sequence of words
Example - Sentiment Analysis: Input: "The movie is terrible" Like reading an entire review to decide thumbs up or down
Production uses: product review scoring, ticket urgency classification, social media brand monitoring
Input: single token/vector $\rightarrow$ Output: sequence of tokens
Component: Input; Description: Source language sentence
Component: Encoder-decoder RNN; Description: Compresses input, generates output
Component: Output; Description: Target language sentence
Example - Image Captioning: Input: image feature vector (from CNN)
Output: "A cat sat on a sofa" The RNN looks at the photo representation and describes it word by word
Production uses: automated alt-text generation, video thumbnail descriptions
Input: sequence $\rightarrow$ Output: sequence (lengths may differ)
Component: Input; Description: One token per time step
Component: RNN; Description: Produces one output per input step
Example - Machine Translation: Input: English sentence (5 words)
Output: French sentence (7 words)
Input and output lengths are not required to match
Production uses: Google Translate (historically), text summarization, chatbot response generation
Input: sequence $\rightarrow$ Output: sequence (aligned one-to-one)
Example - Video Frame Labeling / Subtitle Generation: Task: Classify a document; Architecture: Many-to-one; Why: Whole document - one category
Input: video frames [F1, F2, F3, F4]
Task: Generate text from image; Architecture: One-to-many; Why: Single image - word sequence
Output: labels [L1, L2, L3, L4] (running, jumping, running, sitting)
Task: Translate or summarize; Architecture: Many-to-many (Seq2Seq); Why: Variable-length in and out
Each input time step maps to exactly one output time step
Task: Tag each word; Architecture: Synced many-to-many; Why: One label per token
Production uses: frame-level action recognition, phoneme-level speech alignment

Must remember:

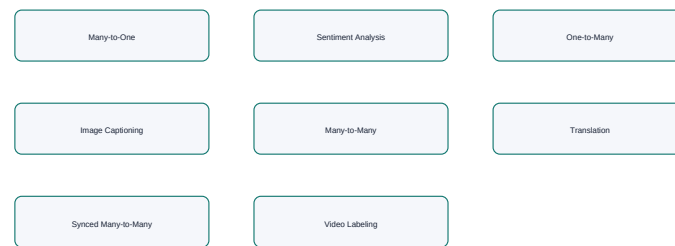
- Four RNN patterns: many-to-one, one-to-many, many-to-many, synced many-to-many
- Many-to-one: sequence input $\rightarrow$ single output (sentiment analysis)
- One-to-many: single input $\rightarrow$ sequence output (image captioning)
- Many-to-many: sequence $\rightarrow$ different-length sequence (translation, summarization)
- Synced many-to-many: one output per input step (POS tagging, frame labeling)
- Architecture choice depends on input/output cardinality, not the RNN cell itself

Formula / decision rule: Input tokens $\rightarrow$ RNN $\rightarrow$ Output tokens; [F_1, F_2, F_3, F_4]

Example or contrast: Architecture: Many-to-one; Input: Sequence; Output: 1 label; Input:Output ratio: N:1; Example: Sentiment analysis
Architecture: One-to-many; Input: 1 vector; Output: Sequence; Input:Output ratio: 1:N; Example: Image captioning
Architecture: Many-to-many; Input: Sequence; Output: Sequence; Input:Output ratio: N:M (variable); Example: Translation

Exam trap: Confusing many-to-many with synced many-to-many - Seq2Seq allows different lengths; synced requires 1:1 alignment "Sentiment analysis is one-to-many" - false; it is many-to-one (many words → one label)

RNN Architecture Patterns: Many-to-One, One-to-Many, Many-to-Many



Vector reconstruction of the source Mermaid visual

Concept map: RNN Architecture Patterns: Many-to-One, One-to-Many, Many-to-Many

NLP-W07-T05

The Vanishing and Exploding Gradient Problem

Tier A

Definition and why it matters: Humans maintain context over long passages effortlessly. Consider a paragraph that begins "I grew up in France" and ends with "I speak fluent " - the answer is obviously "French". A vanilla RNN struggles with this: by the time it reaches the last word, it has forgotten "France" from the beginning. The root cause is how gradients flow backward through time during training.

Core explanation: In theory, the hidden state $H(t-1)$ provides memory across the full sequence. In practice, vanilla RNNs: Handle short-term context well: "The clouds are in the " → sky. Concept flow: Y_T - loss - H_T gradient - H_{T-1} - ... - H_0 gradient. Fail at long-term context: connecting "France" (word 1) to "fluent" (word 100). This is RNN amnesia - only the last few words influence predictions. Training an RNN requires propagating the error backward from the last time step to the first: To update weights affecting an early word, the gradient must travel through every intermediate time step. Problem: Vanishing; Analogy: Whisper gets quieter through 10 people; Effect: Early words have no influence. By the chain rule, the gradient at time step 0 involves multiplying gradients from all subsequent steps: Problem: Exploding; Analogy: Whisper becomes a scream; Effect: Training diverges (NaN). $(dL)/(dH_0) = (dL)/(dH_T) \prod_{t=1}^T \frac{(dH_t)(dH(t-1))}{(dH_t)(dH(t-1))}$ Problem: Exploding gradients; Solution: Gradient clipping; Mechanism: Cap gradient magnitude at a threshold (e.g., 5.0). Each $\frac{(dH_t)(dH(t-1))}{(dH_t)(dH(t-1))}$ involves the weight matrix $W(hh)$. Problem: Vanishing gradients; Solution: LSTM / GRU; Mechanism: Gated cell state preserves gradient flow. If $|W(hh)| < 1$ (e.g., 0.9): Problem: Vanishing gradients; Solution: Attention; Mechanism: Direct connections to all positions (Transformers). $0.9 \times 0.9 \times \dots$ (100 times) $\approx 0.00003 \approx 0$. The gradient shrinks to zero → weights at the beginning of the sequence stop updating → the network cannot learn from the distant past. If $|W(hh)| > 1$ (e.g., 1.1): The gradient grows exponentially → weights become NaN → training crashes. If $|g|$ threshold: $g \leftarrow (\text{threshold})/(|g|) \cdot g$. Simple and effective for exploding gradients - does not help vanishing. Instead of whispering through a chain, LSTM provides a cell state highway where information flows with minimal distortion - like giving each person a walkie-talkie instead of whispering.

Must remember:

- Vanilla RNNs forget long-range context due to vanishing gradients in BPTT
- Chain rule multiplies gradients across all time steps
- Weights 1 → exploding (gradient → NaN)
- Gradient clipping fixes exploding gradients only
- LSTM/GRU solve vanishing via gated cell state (information highway)
- "I grew up in France ... I speak fluent " is the canonical long-context test

Formula / decision rule: H_{t-1} ; $(dL)/(dH_0) = (dL)/(dH_T) \prod_{t=1}^T \frac{(dH_t)(dH(t-1))}{(dH_t)(dH(t-1))}$

Example or contrast: Model: Vanilla RNN; Can connect "France" to "fluent "?: No - gradient vanishes over ~100 steps. Model: LSTM / GRU; Can connect "France" to "fluent "?: Yes - cell state preserves long-range info. Model: Transformer; Can connect "France" to "fluent "?: Yes - attention connects any two positions directly.

Exam trap: Confusing vanishing and exploding - vanishing: weights $\rightarrow 0$, no learning; exploding: weights $\rightarrow \text{NaN}$, crash "Gradient clipping fixes vanishing gradients" - false; clipping only caps large gradients (exploding)

NLP-W07-T06 Long Short-Term Memory (LSTM) Networks

Tier A

Definition and why it matters: Vanilla RNNs suffer from short-term memory - gradients vanish over long sequences. LSTM (Long Short-Term Memory) solves this by separating short-term processing (hidden state) from long-term storage (cell state) and using gates to control what information flows, stays, or gets discarded. Think of LSTM not as a simple neuron but as a logic chip with selective memory management.

Core explanation: Store: Hidden state H_t ; Role: Short-term working memory; Analogy: Current thought. Store: Cell state C_t ; Role: Long-term storage highway; Analogy: Persistent notebook. Concept flow: X_t - Forget Gate - Input Gate - Output Gate - C_{t-1} - Cell State Highway - C_t - H_t . The cell state runs through the entire chain with minimal transformation - gradients flow backward through it largely unchanged, solving the vanishing gradient problem. Information flows straight down the cell state path with few interactions. This "superhighway" allows gradients to propagate backward without vanishing. Decides what information to throw away from the cell state. Uses sigmoid activation $\rightarrow$ output in $[0, 1]$ 0 = forget completely; 1 = keep everything. Example: When the subject changes from singular to plural, forget the old singular subject information $f_t = \sigma(W_f [H(t-1), X_t] + b_f)$. Concept flow: Input X_t - Forget: what to discard - Input: what to store - Update Cell State - Output: what to reveal - Hidden State H_t . Decides what new information to write into the cell state. Sigmoid: what values to update. Tanh: candidate new values. Example: Store the new gender of a plural subject $i_t = \sigma(W_i [H(t-1), X_t] + b_i)$. $\tilde{C}_t = \tanh(W_C [H(t-1), X_t] + b_C)$. Decides what to output based on the current cell state. Example: Output a verb conjugated for the new plural subject $o_t = \sigma(W_o [H(t-1), X_t] + b_o)$. $H_t = o_t \odot \tanh(C_t)$. Receive input X_t and previous states $H(t-1)$, $C(t-1)$. Forget gate removes irrelevant old information. Input gate adds relevant new information. Cell state is updated. Output gate filters cell state to produce H_t . Understanding LSTM gate equations in full detail is valuable for research but not required to use LSTMs effectively in practice. Frameworks like PyTorch (`nn.LSTM`) and TensorFlow (`tf.keras.layers.LSTM`) handle the internals; practitioners focus on architecture design, hyperparameters, and data.

Must remember:

- LSTM separates short-term (hidden state) and long-term (cell state) memory
- Three gates: forget (discard), input (store), output (reveal)
- Cell state is a gradient highway - solves vanishing gradients
- Forget/input gates use sigmoid; candidate values use tanh
- More parameters than vanilla RNN but handles long sequences effectively
- Frameworks abstract gate internals; focus on architecture and data for applied use

Formula / decision rule: H_t ; C_t

Example or contrast: Criterion: Long-term memory; Vanilla RNN: Poor (vanishing gradients); LSTM: Strong (cell state highway). Criterion: Gates; Vanilla RNN: None; LSTM: Forget, input, output. Criterion: Parameters; Vanilla RNN: Fewer; LSTM: More (4 gate weight matrices).

Exam trap: Confusing hidden state and cell state - C_t is long-term storage; H_t is short-term output. "LSTM has no vanishing gradient problem" - the cell state mitigates it; very extreme cases can still struggle.

NLP-W07-T07 Gated Recurrent Unit (GRU)

Tier A

Definition and why it matters: LSTM solved the long-term memory problem but at a cost: four gate weight matrices, separate cell and hidden states, and significant computational overhead. GRU (Gated Recurrent Unit) achieves comparable performance with a simplified architecture - merging the cell and hidden states and reducing three LSTM gates to two.

Core explanation: Combines LSTM's forget and input gates into one decision: "How much of the past should I keep, and how much new information should I let in?" Value: Near 0; Behavior: Discard past, accept new information. Value: Near 1; Behavior: Keep past, ignore new information. Analogy: "Should I update my memory now?" $z_t = \sigma(W_z [H(t-1), X_t])$. Decides how much of the past information to ignore when computing the new candidate state. Analogy: "Drop the previous context for a moment - does this new word make sense

on its own?" The reset gate is useful when a new topic begins and prior context would mislead the prediction. Compute update gate z_t and reset gate r_t from $H(t-1)$ and X_t . Compute candidate hidden state $\tilde{H}_t$ (using reset gate to optionally ignore past). Interpolate between old and new state using update gate: Application: Tweet sentiment analysis; Typical Choice: GRU (short sequences, small data) $H_t = (1 - z_t) \odot H(t-1) + z_t \odot \tilde{H}_t$. Application: Product review classification; Typical Choice: GRU. The update gate controls the blend: mostly old memory or mostly new information. Application: Machine translation (production); Typical Choice: LSTM or Transformer (long sequences). Application: Time-series anomaly detection; Typical Choice: GRU (speed matters for real-time). In cloud ML pipelines, GRU-based models deploy faster and cost less per inference - a meaningful advantage at scale when LSTM's marginal accuracy gain does not justify the overhead.

Must remember:

- GRU is a simplified LSTM: 2 gates instead of 3, no separate cell state
- Update gate: blends past and new information (replaces forget + input gates)
- Reset gate: optionally ignores past when computing new candidate state
- Faster to train, fewer parameters, often matches LSTM on smaller datasets
- Rule of thumb: start with GRU; switch to LSTM if underperforming on long/complex sequences
- Transformers have largely superseded both in modern NLP, but GRU/LSTM remain relevant for constrained environments

Formula / decision rule: $z_t = \sigma(W_z [H_{t-1}, X_t])$; $r_t = \sigma(W_r [H_{t-1}, X_t])$

Example or contrast: Component: Memory stores; LSTM: Cell state + hidden state; GRU: Hidden state only (merged). Component: Gates; LSTM: 3 (forget, input, output); GRU: 2 (update, reset). Component: Parameters; LSTM: More; GRU: Fewer.

Exam trap: "GRU has a separate cell state" - false; GRU merges cell and hidden state. "GRU is always worse than LSTM" - false; often comparable, especially on smaller datasets.

Week 7 Exam Lens

Likely questions

- Define and explain Sequence-to-Sequence (Seq2Seq) Modelling; include the mechanism and one limitation.
- Compare Sequence-to-Sequence (Seq2Seq) Modelling with Recurrent Neural Networks (RNNs) and state when each is preferred.
- Apply RNN Architecture Patterns: Many-to-One, One-to-Many, Many-to-Many to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $h_1; h_2$; directly to output; $X \rightarrow H \rightarrow Y$; Input tokens $\rightarrow$ RNN $\rightarrow$ Output tokens; $[F_1, F_2, F_3, F_4]$; H_{t-1} ; $(dL)/(dH_0) = (dL)/(dH_T) \prod_{t=1}^T \frac{(dH_t)}{(dH_{t-1})}$

Mistakes to avoid: "TF-IDF handles variable length" - the vector size is fixed (vocabulary length); only the number of non-zero entries varies. Confusing encoder and decoder roles - encoder reads input; decoder generates output. "RNN has different weights at each step" - false; weights are shared (tied) across time steps.

Week 8: The Transformer Revolution

Week roadmap: From Sequential Models to Transformers: The Motivation → Attention Is All You Need: Origins of the Transformer → Transformer Architecture: Self-Attention, Math, and Positional Encoding → Popular Transformer Model Families: Encoders, Decoders, and Seq2Seq → Hugging Face: The Open ML Hub for Models, Data, and Apps → Hugging Face Tokens and Model Implementation in Colab.

Coverage: 6 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W08-T01 From Sequential Models to Transformers: The Motivation

Tier A

Definition and why it matters: Recurrent Neural Networks (RNNs) and their improved variant, Long Short-Term Memory (LSTM) networks, were designed to handle sequences by maintaining hidden state across time steps. LSTMs address the vanishing gradient problem better than plain RNNs and can retain information over longer spans - a genuine advance over earlier sequence models

Core explanation: However, both architectures share a structural constraint: sequential processing. To compute the representation of the 100th token, the model must first process tokens 1 through 99. Each step depends on the previous one, so the computation graph unfolds strictly in order
 Concept flow: Token 1 - Token 2 - Token 3 - ... - Token 100 - Token 1 - Token 2 - Token 100 - Token 3
 Modern GPUs excel at massive parallel matrix operations - the same kind of work that powers training on cloud ML platforms (AWS SageMaker, Google Vertex AI, Azure ML). Sequential models underutilize this hardware: Only one time step runs at a time per sequence
 Batch parallelism helps across sequences, but not within a single long document
 Training on large corpora (millions of sentences) becomes prohibitively slow
 Limitation: Processing order; LSTM / RNN: Sequential (token by token); Transformer: Parallel (all tokens at once)
 For a 100-word sentence, the bottleneck is manageable. For a document with 1,000-10,000 words, sequential dependency makes both training time and inference latency unacceptable at production scale
 Limitation: GPU utilization; LSTM / RNN: Low within a sequence; Transformer: High - matrix ops across all positions
 Even with LSTM gating, information from early tokens tends to fade by the time the model reaches token 100.
 Real NLP tasks - coreference resolution ("it" referring to "animal"), document-level classification, long-form summarization - require linking distant words. LSTMs partially mitigate this but do not eliminate it
 Limitation: Long-range links; LSTM / RNN: Indirect, decay over distance; Transformer: Direct via attention weights
 Limitation: Training speed on large data; LSTM / RNN: Slow; Transformer: Much faster at scale
 Transformers abandon recurrence entirely. Instead, they rely on a mechanism that lets any token attend to any other token in a single parallel pass: Process the entire sentence (or chunk) at once
 Each word can directly "look at" every other word and compute a relevance score
 No waiting for prior tokens to finish processing
 This design unlocks full GPU parallelism and direct long-range connections - the foundation of BERT, GPT, Gemini, Claude, and virtually every modern NLP system deployed in production

Must remember:

- LSTMs fix vanishing gradients but retain sequential processing as a core bottleneck
- Sequential models cannot efficiently use GPU parallelism within a single long sequence
- Long documents (1,000+ words) expose both speed and long-distance dependency failures in RNN/LSTM architectures
- Transformers replace recurrence with attention: every token can relate to every other token in one step
- This parallel design is the prerequisite for training on internet-scale data and building modern LLMs
- The shift from LSTM-era NLP to Transformer-era NLP is driven by hardware efficiency and direct long-range modeling, not just incremental accuracy gains

Exam trap: Trap: "LSTMs solve all long-sequence problems." LSTMs improve memory but still process sequentially and still struggle with very long dependencies compared to attention
 Trap: Confusing "parallel batching" (processing many sequences at once) with "parallel within-sequence processing" - only Transformers achieve the latter

NLP-W08-T02

Attention Is All You Need: Origins of the Transformer

Tier A

Definition and why it matters: Before 2017, state-of-the-art sequence modeling relied heavily on Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs) for tasks like machine translation. These architectures processed text step-by-step or through local convolution windows - workable, but slow and limited in capturing global context efficiently

Core explanation: In 2017, researchers at Google Brain published "Attention Is All You Need" (Vaswani et al.). The paper's central proposal was radical in its simplicity: Concept flow: Input Sequence - Self-Attention Layers - Feed-Forward Networks - Output Representations - RNN / LSTM - CNN encoders Discard RNNs and CNNs for sequence transduction. Use self-attention as the sole mechanism for relating tokens Self-attention allows each position in a sequence to compute a weighted combination of representations from all other positions. No hidden state needs to propagate sequentially from left to right. The model learns which tokens matter for interpreting each token - in parallel Model family: BERT; Role: Language understanding; Transformer variant: Encoder-only Model family: GPT-3 / GPT-4; Role: Text generation; Transformer variant: Decoder-only The Transformer architecture enabled training on massive corpora that were impractical under sequential models: Model family: Gemini, Claude; Role: General-purpose LLMs; Transformer variant: Decoder-only (scaled) Full Wikipedia dumps Model family: T5, BART; Role: Translation, summarization; Transformer variant: Encoder-decoder Reddit comment archives Stack Overflow Q&A Broad web-scale crawls Training became orders of magnitude faster because attention layers map cleanly to parallel GPU kernels. The result was not just faster experiments - it unlocked models large enough to exhibit emergent language understanding The paper directly enabled the modern large language model ecosystem: Every major cloud NLP API - OpenAI, Google AI, Anthropic, Azure OpenAI - rests on Transformer derivatives trained on web-scale data Transfer learning at scale: Pre-train once on huge data, fine-tune for downstream tasks (sentiment, NER, QA) Infrastructure alignment: Transformers match the hardware trend toward tensor-core GPUs and TPU pods Open ecosystem: Architectures and weights published on Hugging Face accelerated adoption across startups and enterprises Real-world example: A fintech company routing support tickets previously used bag-of-words + logistic regression. After BERT-based fine-tuning (enabled by the Transformer paper's architecture), accuracy on intent classification jumped while the same cloud GPU budget supported larger batch sizes during training

Must remember:

- Published in 2017 by Google Brain; title: "Attention Is All You Need."
- Proposed replacing RNNs and CNNs with pure self-attention for sequence modeling
- Self-attention computes relationships between all token pairs in parallel
- Enabled training on web-scale datasets (Wikipedia, Reddit, Stack Overflow) much faster than before
- Directly birthed BERT, GPT, Gemini, Claude, and the modern LLM industry
- The original architecture combined encoder and decoder; later models often use only one side
- Cloud-scale NLP today is fundamentally a consequence of this architectural shift

Exam trap: Trap: Crediting LSTMs as the foundation of GPT - GPT is decoder-only Transformer, not LSTM-based Trap: Thinking "Attention Is All You Need" introduced attention for the first time - attention existed in seq2seq models before; the novelty was using only self-attention, removing recurrence entirely

NLP-W08-T03

Transformer Architecture: Self-Attention, Math, and Positional Encoding

Tier A

Definition and why it matters: Consider the sentence: Concept flow: it - animal - street - tired

Core explanation: Stage: Input embeddings; Purpose: Convert token IDs to dense vectors Stage: Positional encoding; Purpose: Inject order information (see below) Stage: Encoder self-attention; Purpose: Tokens attend to all tokens in the source Stage: Feed-forward network (FFN); Purpose: Non-linear transformation per position Stage: Decoder masked attention; Purpose: Tokens attend only to prior decoder positions Stage: Encoder-decoder attention; Purpose: Decoder queries attend to encoder outputs Stage: Softmax; Purpose: Convert logits to probability distribution over vocabulary Symbol: Q (Query); Meaning: "What am I looking for?"; Intuition: The token asking for context Symbol: K (Key); Meaning: "What do I contain?"; Intuition: Labels/tags used for matching Symbol: V (Value); Meaning: "What information do I pass?"; Intuition: Actual content aggregated after matching Symbol: dk; Meaning: Key dimension; Intuition: Scaling factor prevents

dot products from growing too large "Shifted right" in decoder inputs means the decoder receives previously generated tokens as input during training (teacher forcing), shifted one position so it predicts the next token at each step Scaled dot-product attention is defined as: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}}) V$ Softmax converts raw scores into a probability distribution (values between 0 and 1 that sum to 1), determining how much each value vector contributes The scaling by $\sqrt{d_k}$ stabilizes gradients when dimensionality is high - a detail often tested in exams Word: The; Sentence 1 position: 1; Sentence 2 position: 1 (different role) One attention head captures one type of relationship. Multi-head attention runs several heads in parallel, each learning different patterns: Word: dog; Sentence 1 position: 2; Sentence 2 position: 5 Head 1: syntactic structure (subject-verb agreement) Word: bit; Sentence 1 position: 3; Sentence 2 position: 3 Head 2: semantic similarity (synonyms, related concepts) Head 3: coreference (pronouns $\rightarrow$ antecedents) Word: man; Sentence 1 position: 5; Sentence 2 position: 2 Outputs from all heads are concatenated and projected. This replaces sequential processing with parallel specialized attention - directly addressing the GPU efficiency problem Because all tokens are processed simultaneously, the raw embedding of "The dog bit the man" is identical to "The man bit the dog" without additional information. Word order is lost Solution: Add a positional signal to each embedding using sine and cosine waves of different frequencies. Each position receives a unique encoding vector Example for two sentences with the same words in different order: The model can now distinguish "dog bit man" from "man bit dog" because positional encodings differ even when word embeddings repeat

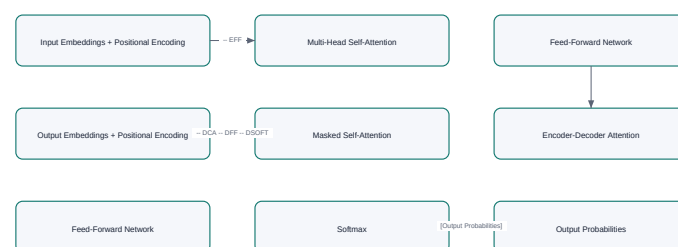
Must remember:

- Self-attention: each token scores its relationship with every other token; high scores drive the final representation
- Coreference ("it" $\rightarrow$ "animal") is a canonical example of attention-based disambiguation
- Encoder: bidirectional self-attention + FFN; Decoder: masked self-attention + cross-attention + FFN + softmax
- Attention formula: $\text{softmax}(QK^T / \sqrt{d_k}) V$ with Query, Key, Value roles
- Multi-head attention runs parallel specialized heads (grammar, semantics, coreference) and concatenates results
- Positional encoding (sin/cos waves) injects order because parallel processing discards sequence information by default
- Scaling by $\sqrt{d_k}$ prevents softmax saturation in high dimensions

Formula / decision rule: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}}) V$; d_k

Exam trap: Trap: Saying self-attention is unidirectional in the encoder - encoder self-attention is fully bidirectional; only the decoder uses masking for autoregressive generation Trap: Forgetting $\sqrt{d_k}$ in the attention formula - omitting it is a common exam error

Transformer Architecture: Self-Attention, Math, and Positional Encoding



Vector reconstruction of the source Memmald visual

Concept map: Transformer Architecture: Self-Attention, Math, and Positional Encoding

NLP-W08-T04

Popular Transformer Model Families: Encoders, Decoders, and Seq2Seq

Tier A

Definition and why it matters: The original 2017 Transformer combined an encoder (reads and understands input) and a decoder (generates output). Modern production models rarely use both stacks unchanged. Instead, practitioners pick the half that matches the task - or use the full encoder-decoder for transformation tasks

Core explanation: Concept flow: Encoder - Decoder - Encoder-Only: BERT, RoBERTa - Decoder-Only: GPT, LLaMA, Gemini - Encoder-Decoder: T5, BART - Original - Modern Model: BERT; Notes: Bidirectional Encoder Representations from Transformers Model: RoBERTa; Notes: Robustly optimized BERT (more data, no NSP) Model: DistilBERT; Notes: Smaller, faster distilled BERT Encoder-only models process the entire input bidirectionally - every token sees every other token. They excel at understanding, not generation Model: GPT (Generative Pre-trained Transformer); Notes: The T stands for Transformer Model: LLaMA; Notes: Meta's open-weight decoder family Model: Gemini; Notes: Google's multimodal decoder stack Text classification (spam, intent, topic) Named Entity Recognition (NER) Question answering (with fine-tuning) Real-world use: A cloud support platform fine-tunes DistilBERT on ticket text to route "billing" vs "technical" vs "cancellation" requests - low latency on CPU-friendly deployments Key property: Sees context from both left and right simultaneously (bidirectional) Decoder-only models generate text autoregressively - each new token is predicted from all prior tokens only (left context). They cannot see future tokens during generation Model: T5 (Text-to-Text Transfer Transformer); Notes: Frames all tasks as text-to-text Model: BART; Notes: Denoising autoencoder pre-training Open-ended text generation Chat and dialogue Creative writing, summarization (via prompting) Real-world use: GitHub Copilot and ChatGPT-style assistants are decoder-only models scaled to billions of parameters and aligned with human feedback Key property: Causal / unidirectional attention - only past tokens inform the next prediction These retain both stacks: the encoder builds a rich representation of the source; the decoder generates the target sequence while attending to the encoder Any task mapping one sequence to another Real-world use: Google Translate pipelines and enterprise document summarization APIs often use encoder-decoder Transformers fine-tuned per language pair

Must remember:

- Original Transformer = encoder + decoder; modern models often use one side or both depending on task
- Encoder-only (BERT family): bidirectional readers - classification, sentiment, NER, QA
- Decoder-only (GPT family): left-to-right writers - generation, chat, code; T = Transformer
- Encoder-decoder (T5, BART): translators - translation, summarization, seq2seq
- Choose architecture by task: understand → encoder; generate → decoder; transform sequence → encoder-decoder
- DistilBERT and similar variants trade minimal accuracy for speed - common in production edge deployments

Example or contrast: Aspect: Direction; Encoder-Only: Bidirectional; Decoder-Only: Unidirectional (left-to-right); Encoder-Decoder: Encoder: bidirectional; Decoder: causal Aspect: Primary task; Encoder-Only: Understanding / classification; Decoder-Only: Generation; Encoder-Decoder: Sequence transformation Aspect: Examples; Encoder-Only: BERT, RoBERTa, DistilBERT; Decoder-Only: GPT, LLaMA, Gemini; Encoder-Decoder: T5, BART

Exam trap: Trap: Using GPT (decoder-only) for bidirectional NER without adaptation - GPT sees only left context unless fine-tuned with special prompting; BERT is the natural choice for token-level understanding Trap: Forgetting that T in GPT = Transformer, not "trained" or "text" alone

NLP-W08-T05

Hugging Face: The Open ML Hub for Models, Data, and Apps

Tier B

Definition and why it matters: Hugging Face (huggingface.co) is a collaborative platform where the machine learning community publishes and consumes models, datasets, and applications. It began as an aggregator for open-source Transformer weights and evolved into a full ecosystem for AI developers - comparable to GitHub for code, but centered on ML artifacts

Core explanation: As of recent counts, the hub hosts over 2 million models, spanning NLP, vision, audio, and multimodal systems. Major contributors include Meta (Facebook), Mistral, Baidu, Cohere, and thousands of independent researchers Concept flow: Hugging Face Hub - Models: Pre-trained weights + cards - Datasets: Curated training/eval data - Spaces: Hosted demo apps - Buckets: Dataset storage - Documentation & Enterprise Pricing The core value proposition: download and run state-of-the-art models in a few lines of Python Colab / Kaggle: One-click notebook links from model pages Local Python: Copy pipeline or AutoModel snippets into scripts Enterprise: Hosted inference endpoints with pricing tiers Real-world use: A startup building a document QA product prototypes with bert-base-uncased from Hugging Face, then swaps to

a domain fine-tune (legal-bert) without changing pipeline code - only the model ID changes Hugging Face also hosts high-value datasets with documentation and loading scripts: Reasoning traces synthesized by frontier models Domain-specific medical, legal, and financial text Each dataset page describes schema, license, size, and a `load_dataset()` code example - enabling reproducible training pipelines on cloud VMs A newer offering: managed storage for private or team datasets, analogous to cloud object storage integrated with the hub workflow. Teams training custom models on proprietary data can keep artifacts in one ecosystem Create a free account (email + password) Browse trending models on the homepage or search by name (e.g., "BERT", "Mistral")

Must remember:

- Hugging Face is the primary open hub for sharing and running ML models, datasets, and demo apps
- Models hub: 2M+ checkpoints with cards, usage code, and browser testing widgets
- Datasets hub: documented corpora loadable via datasets library
- Spaces: host interactive Gradio/Streamlit apps for public demos
- Buckets: storage for custom datasets within the platform
- Typical workflow: search model → read card → copy Python snippet → run in Colab or cloud VM
- Always verify license, limitations, and domain fit before production deployment

Example or contrast: Each model page typically includes: Model card: Description, training data, intended use, limitations Variations: Related checkpoints (uncased, cased, fine-tuned variants)

Exam trap: Trap: Assuming all Hugging Face models are free for commercial use - always check the license on the model card (Apache 2.0, MIT, CC, research-only, etc.) Trap: Confusing the website widget results with guaranteed production accuracy - browser demos use default checkpoints and may differ from fine-tuned deployments

NLP-W08-T06

Hugging Face Tokens and Model Implementation in Colab

Tier A

Definition and why it matters: Many models on Hugging Face are public and download without authentication. However, gated models, private checkpoints, and higher rate limits require an access token. Best practice: configure the token once per notebook session so any model download succeeds without embedding secrets in source code

Core explanation: Log in at huggingface.co Open profile menu → Access Tokens (huggingface.co/settings/tokens) Click Create new token Assign a descriptive name (e.g., colab-research) Click Create token at the bottom Copy the token immediately - it may not be shown again in full Token management options after creation: Edit permissions (read vs write) Refresh or revoke Delete compromised tokens Security rule: Never commit tokens to Git, share them in screenshots, or hardcode them in notebook cells visible to others Colab Secrets keep credentials out of notebook source: Concept flow: Create token on HF settings - Add HFTOKEN to Colab Secrets - Grant notebook access - `userdata.get` in code - Download models via `transformers` Click the key icon (Secrets) in the left sidebar Add new secret: Name: HFTOKEN (exact spelling matters for lookup) Value: paste your token Toggle Notebook access to grant the runtime permission Retrieve the secret in code: Store in a variable - do not print the token after loading Copy the usage snippet from a model page (e.g., bert-base-uncased): Checkpoint: The model identifier string (e.g., bert-base-uncased) points to weights + config on the hub. `from_pretrained(checkpoint)` downloads and caches locally Not every model requires a token, but loading HFTOKEN upfront avoids failures on gated checkpoints Code pattern:

```
from google.colab import userdata; hftoken = userdata.get('HFTOKEN')
```

 Code pattern:

```
from transformers import pipeline; fillmask = pipeline("fill-mask", model="bert-base-uncased"); fillmask("Hello I'm a [MASK] model.");
```

 Top prediction: "fashion" (among role, new, super, fine, etc.); `fillmask("The cat sat on the [MASK].");` Top predictions: floor, bed, couch, sofa, ground

Must remember:

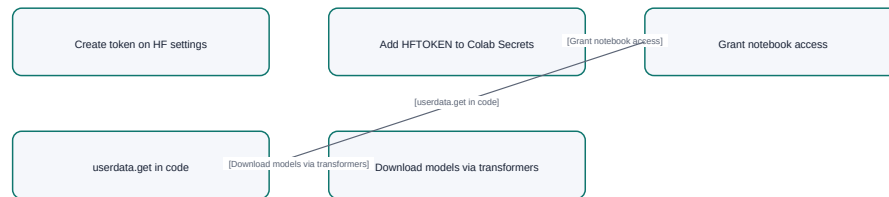
- Generate HF access tokens at Settings → Access Tokens; copy once at creation
- Store as HFTOKEN in Colab Secrets; load via `userdata.get('HFTOKEN')`
- Never expose tokens in source code, commits, or printed output
- `pipeline()` offers high-level task APIs (fill-mask, sentiment, translation)
- Seq2Seq: `AutoTokenizer + AutoModelForSeq2SeqLM.from_pretrained(checkpoint) + generate()`

- Checkpoint = hub model ID linking tokenizer weights and architecture
- Pre-load token even for public models - gated models will fail without it

Example or contrast: For translation (encoder-decoder), the pattern uses AutoTokenizer and AutoModel ForSeq2SeqLM: Component: checkpoint; Role: Hub model ID - selects weights and vocabulary Component: AutoTokenizer; Role: Text - token IDs (subword segmentation)

Exam trap: Trap: Hardcoding hf... tokens in notebook cells - use Colab Secrets or environment variables
 Trap: Mismatching tokenizer and model checkpoints - always pair them from the same repo

Hugging Face Tokens and Model Implementation in Colab



Vector reconstruction of the source Mermaid visual

Concept map: Hugging Face Tokens and Model Implementation in Colab

Week 8 Exam Lens

Likely questions

- Define and explain From Sequential Models to Transformers: The Motivation; include the mechanism and one limitation.
- Compare From Sequential Models to Transformers: The Motivation with Attention Is All You Need: Origins of the Transformer and state when each is preferred.
- Apply Transformer Architecture: Self-Attention, Math, and Positional Encoding to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}}) V$; d_k

Mistakes to avoid: Trap: "LSTMs solve all long-sequence problems." LSTMs improve memory but still process sequentially and still struggle with very long dependencies compared to attention Trap: Crediting LSTMs as the foundation of GPT - GPT is decoder-only Transformer, not LSTM-based Trap: Saying self-attention is unidirectional in the encoder - encoder self-attention is fully bidirectional; only the decoder uses masking for autoregressive generation

Week 9: Natural Language Understanding with BERT

Week roadmap: BERT and Contextual Embeddings: Beyond Static Word Vectors → BERT Architecture and Pre-Training Objectives → Applications of BERT in Production NLP → BERT Variants: RoBERTa, ALBERT, DistilBERT, and Domain Fine-Tunes.

Coverage: 4 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W09-T01 BERT and Contextual Embeddings: Beyond Static Word Vectors

Tier A

Definition and why it matters: When BERT processes "I deposited money at the bank near the river," the representation of bank incorporates deposited, money, and river. The same word form in a financial headline gets a different internal representation because its neighbors differ. This directly addresses limitations of Word2Vec-era pipelines used in early cloud search and recommendation systems.

Core explanation: Before BERT, dominant word representations - Word2Vec, GloVe, fastText - assigned one fixed vector per word type. The word "bank" received the same embedding whether the sentence discussed a riverbank or Bank of America. This is the polysemy problem: one surface form, multiple meanings. Static embeddings average or collapse senses into a single point in vector space, limiting downstream task accuracy. Approach: Word2Vec / GloVe; "bank" (river): Same vector; "bank" (financial): Same vector. Approach: BERT; "bank" (river): Context-dependent vector; "bank" (financial): Different vector. Concept flow: Pre-train on Wikipedia: MLM + NSP - Fine-tune on labeled task - Distill to smaller model - Production: sentiment, QA, NER - Mobile / low-latency APIs. BERT is an encoder-only Transformer stack pre-trained on large text corpora. It revolutionized Natural Language Understanding (NLU) by producing contextual embeddings - the vector for each token depends on all surrounding tokens. Training BERT from scratch requires enormous compute (Wikipedia-scale data, many GPU-days). Transfer learning reuses pre-trained weights: Pre-training: Learn general language structure (self-supervised on unlabeled text). Fine-tuning: Add a task-specific head; train on labeled data for classification, QA, NER, etc. Distillation: Compress a large teacher model into a smaller student (e.g., DistilBERT) for mobile or real-time inference. Real-world example: An e-commerce platform fine-tunes BERT on 50k product reviews instead of training a classifier on bag-of-words from scratch - achieving higher F1 on aspect sentiment with less labeled data. Classification: [CLS] token embedding → softmax over labels. Question answering: Predict start/end spans in passage. NER: Per-token labels from contextual representations. All benefit from bidirectional context unavailable to left-to-right models at pre-training time.

Must remember:

- Static embeddings (Word2Vec, GloVe): one vector per word regardless of context
- BERT: Bidirectional Encoder Representations from Transformers - encoder-only stack
- Contextual embeddings change per occurrence based on surrounding tokens
- Polysemy ("bank" river vs financial) motivated the shift from static to contextual models
- Transfer learning: pre-train once, fine-tune for classification, QA, NER; distill for speed
- BERT is the foundation model for modern NLU pipelines before the GPT chat era dominated generation tasks

Exam trap: Trap: Calling BERT embeddings "static" - they are contextual; only the pre-training objective is fixed, not the runtime vectors. Trap: Confusing BERT with GPT - BERT is encoder-only and bidirectional; GPT is decoder-only and unidirectional.

NLP-W09-T02 BERT Architecture and Pre-Training Objectives

Tier A

Definition and why it matters: BERT is a stack of Transformer encoder blocks - the encoder half of the original "Attention Is All You Need" architecture, without the decoder. GPT occupies the decoder side; BERT occupies the encoder side.

Core explanation: Concept flow: Encoder Stack - Decoder Stack - BERT: Encoder only - GPT: Decoder only - FullTransformer - Split Variant: BERT-base; Encoder layers: 12; Typical use: Default for fine-tuning experiments. Variant: BERT-large; Encoder layers: 24; Typical use: Higher capacity; more compute. Concept flow: The cat - [MASK] - on the mat - Predict: sat. BERT was not trained for translation or autoregressive

generation. It was pre-trained to understand language via two self-supervised tasks on Wikipedia (and Books Corpus) Goal: Force the model to use bidirectional context to predict hidden words Take a sentence: "The cat sat on the mat" Randomly mask ~15% of tokens (e.g., hide sat → "The cat [MASK] on the mat") Model predicts the original token using both left and right context Repeat across millions of sentences with different masks each epoch Why "bidirectional": Unlike GPT (left-to-right only), BERT sees sat's neighbors on both sides when filling the blank. The label for sat depends on cat, on, and mat simultaneously Training detail (often tested): Of selected tokens, 80% are replaced with [MASK], 10% with random words, 10% unchanged - reducing train/serve mismatch Goal: Teach inter-sentence relationships - critical for QA, natural language inference, and summarization Task: MLM; What the model learns: Token-level meaning from full context Split text into sentence pairs: Task: NSP; What the model learns: Discourse / sentence-pair coherence Sentence A: "The man went to the store" Sentence B: "He bought milk" Training examples include: IsNext: B truly follows A in the original document NotNext: B is a random sentence from elsewhere Model predicts whether B is the actual next sentence Why it matters: Question-passage pairs, premise-hypothesis pairs, and multi-sentence inputs require knowing whether two spans are related - not just whether individual words co-occur BERT inputs concatenate segments with special tokens: [CLS] sentenceA [SEP] sentenceB [SEP] [CLS] - classification aggregate used for sentence-level tasks [SEP] - separator between segments Segment embeddings + token embeddings + positional embeddings are summed before entering the encoder stack

Must remember:

- BERT = stack of Transformer encoders only (no decoder)
- BERT-base: 12 layers; BERT-large: 24 layers
- Pre-trained on Wikipedia-scale text, not for translation/generation
- MLM: mask tokens; predict using bidirectional context → contextual embeddings
- NSP: predict if sentence B follows sentence A → sentence-pair understanding
- Input uses [CLS], [SEP], token, segment, and positional embeddings
- RoBERTa later dropped NSP; BERT original recipe includes both MLM and NSP

Exam trap: Trap: Saying BERT is trained left-to-right - MLM is bidirectional; only the decoder in GPT is strictly causal Trap: Confusing MLM with autoregressive LM - MLM predicts masked positions with full context; GPT predicts next token with left context only

BERT Architecture and Pre-Training Objectives



Vector reconstruction of the source Mermaid visual

Concept map: BERT Architecture and Pre-Training Objectives

NLP-W09-T03

Applications of BERT in Production NLP

Tier A

Definition and why it matters: BERT produces deep contextual representations for every token plus a sentence-level [CLS] embedding. After pre-training, a lightweight task-specific layer is added and fine-tuned on labeled data - far less data and compute than training from scratch

Core explanation: Because BERT sees full bidirectional context, it outperforms bag-of-words, TF-IDF, and static embedding classifiers on nuanced text Concept flow: BERT Pre-trained Encoder - Text Classification Head - Token Classification Head - Span Prediction Head - Pair Classification Head - Sentiment / Spam / Intent - Entity Extraction - Reading Comprehension - Entailment / Contradiction Application: Sentiment analysis; Label examples: positive, negative, neutral Application: Spam detection; Label examples: spam, not spam Application: Intent recognition; Label examples: bookflight, cancelreservation, trackorder Application:

Topic labeling; Label examples: sports, finance, politics Application: Ticket routing; Label examples: billing, technical, account Assign a single label (or multi-label set) to an entire document or sentence Mechanism: Fine-tune BERT with [CLS] output → linear layer → softmax Real-world use: Gmail-style spam filtering and Zendesk-style support ticket categorization on cloud NLP APIs often use fine-tuned BERT or DistilBERT backends Extract structured entities from unstructured text - persons, organizations, locations, dates, medical terms Example: "Apple announced a new office in Austin" → ORG: Apple, LOC: Austin Mechanism: Fine-tune with a token classification head - each subword token receives a BIO tag (Begin, Inside, Outside) Real-world use: Healthcare pipelines extract drug names and dosages from clinical notes; legal tech extracts party names and jurisdictions from contracts Given a question and a context passage, predict the start and end span of the answer in the passage Context: "The Transformer was published in 2017 by Google Brain." Question: "Who published the Transformer?" Answer span: "Google Brain" Mechanism: Two logits per token (start probability, end probability); highest-scoring valid span wins Real-world use: Enterprise search over internal wikis; customer support bots retrieving policy clauses Determine the logical relationship between a premise and a hypothesis: Entailment: hypothesis must be true if premise is true Contradiction: hypothesis cannot be true if premise is true Neutral: neither entailed nor contradicted Premise: "It rained all day." Hypothesis: "The ground is wet." → Entailment Mechanism: Sentence pair input [CLS] premise [SEP] hypothesis [SEP] → classification head Real-world use: Fact-checking pipelines, contract compliance checking (does clause A imply obligation B?)

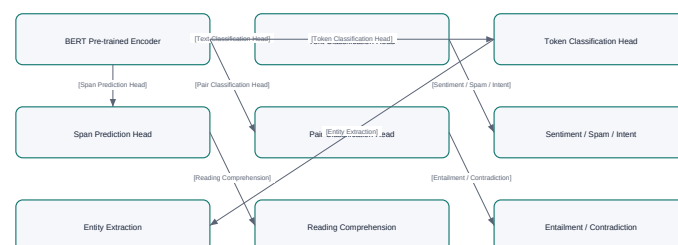
Must remember:

- BERT fine-tuning adds a small head on pre-trained encoders for downstream tasks
- Text classification: sentiment, spam, intent, topic - [CLS] embedding → label
- NER: token-level BIO tags for persons, orgs, locations, domain entities
- QA: predict answer start/end spans in context passages
- NLI: classify premise-hypothesis as entailment, contradiction, or neutral
- All applications leverage bidirectional contextual representations from MLM pre-training
- Production systems (search, support, healthcare, legal) deploy fine-tuned BERT variants via Hugging Face pipelines or custom serving

Example or contrast: Task: Text classification; Output granularity: One label per document; Typical head: [CLS] + linear Task: NER; Output granularity: Label per token; Typical head: Token-wise linear Task: QA; Output granularity: Span (start, end); Typical head: Two span logits

Exam trap: Trap: Using BERT for open-ended story generation without a generative decoder - BERT is not autoregressive; use GPT/T5 for free-form generation Trap: Confusing QA extractive (span from passage) with generative QA (model writes answer in own words) - classic BERT QA is extractive

Applications of BERT in Production NLP



Vector reconstruction of the source Mermaid visual

Concept map: Applications of BERT in Production NLP

NLP-W09-T04 BERT Variants: RoBERTa, ALBERT, DistilBERT, and Domain Fine-Tunes

Tier A

Definition and why it matters: The original BERT checkpoint is a starting point, not the final word. Researchers and practitioners have created architectural variants (different pre-training recipes), efficiency variants (smaller/faster), and domain fine-tunes (specialized vocabulary and corpora). Hugging Face lists hundreds of checkpoints when searching "BERT."

Core explanation: Concept flow: Original BERT - Architectural variants: RoBERTa, ALBERT - Efficiency variants: DistilBERT - Domain fine-tunes: BioBERT, Legal-BERT, FinBERT Feature: NSP task; BERT: Yes; RoBERTa: No Feature: Training duration; BERT: Shorter; RoBERTa: Longer Feature: Data volume; BERT: Wikipedia + BooksCorpus; RoBERTa: Larger mix Developed by Facebook AI Feature: Case handling; BERT: uncased & cased variants; RoBERTa: primarily cased Same Transformer encoder architecture as BERT Trained longer on more data (including CC-News, OpenWebText, Stories) Removed Next Sentence Prediction (NSP) - ablations showed NSP added little Case-sensitive (cased checkpoints common) Typically outperforms original BERT on GLUE and related benchmarks Concept flow: Full BERT: Teacher - DistilBERT: Student - Low-latency inference Reduces memory via parameter sharing across layers (same weights reused) Fewer unique parameters → lower GPU memory for same depth Model: FinBERT; Domain: Financial literature; Language / notes: Market reports, earnings calls Trade-off: sometimes slightly lower accuracy vs full BERT-large at equal compute Model: PubMedBERT; Domain: PubMed biomedical abstracts; Language / notes: Clinical NLP, literature mining Use case: Training large-depth models on memory-constrained cloud instances Model: BioBERT / SciBERT; Domain: Biomedical & scientific text; Language / notes: Entity extraction in papers Knowledge distillation from full BERT teacher to smaller student Model: Legal-BERT; Domain: Legal documents; Language / notes: Contracts, case law Student mimics teacher's output probability distributions, not just hard labels Model: German BERT variants; Domain: Medical/legal German; Language / notes: e.g., GermEval-BERT Results: ~40% smaller, ~60% faster, retains ~97% of BERT-base performance Ideal for mobile, edge, and real-time APIs (sentiment on live chat streams) Requirement: Maximum accuracy, GPU budget available; Recommended direction: RoBERTa-large or BERT-large Requirement: Latency-sensitive / CPU inference; Recommended direction: DistilBERT General BERT knows broad English; domain models adapt vocabulary and semantics to specialized text Requirement: Memory-constrained training; Recommended direction: ALBERT Requirement: Financial text; Recommended direction: FinBERT Requirement: Biomedical NER; Recommended direction: PubMedBERT / BioBERT Requirement: Legal clause classification; Recommended direction: Legal-BERT Real-world use: A hospital NLP pipeline uses PubMedBERT for extracting diagnosis codes from notes - general bert-base-uncased underperforms on medical abbreviations and drug names

Must remember:

- BERT spawned architectural, efficiency, and domain-specific variants on Hugging Face
- RoBERTa: longer training, more data, no NSP, case-sensitive, usually beats BERT
- ALBERT: parameter sharing → lower memory footprint; case-sensitive
- DistilBERT: knowledge distillation → 40% smaller, 60% faster, ~97% of BERT performance
- Domain models: FinBERT (finance), PubMedBERT (medicine), Legal-BERT (law), language-specific medical BERTs
- Pick variant by accuracy budget, latency, memory, and domain vocabulary - not all "BERT" checkpoints are equivalent

Exam trap: Trap: RoBERTa "changes the Transformer architecture entirely" - it mainly changes training procedure (no NSP, more data) Trap: DistilBERT numbers - remember 40% smaller, 60% faster, ~97% performance (approximate; common exam fodder)

Week 9 Exam Lens

Likely questions

- Define and explain BERT and Contextual Embeddings: Beyond Static Word Vectors; include the mechanism and one limitation.
- Compare BERT and Contextual Embeddings: Beyond Static Word Vectors with BERT Architecture and Pre-Training Objectives and state when each is preferred.
- Apply Applications of BERT in Production NLP to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Mistakes to avoid: Trap: Calling BERT embeddings "static" - they are contextual; only the pre-training objective is fixed, not the runtime vectors Trap: Saying BERT is trained left-to-right - MLM is bidirectional; only the decoder in GPT is strictly causal Trap: Using BERT for open-ended story generation without a generative decoder - BERT is not autoregressive; use GPT/T5 for free-form generation

Week 10: Text Classification and Sentiment Analysis

Week roadmap: Text Classification: Motivation and Real-World Applications → Sentiment Analysis: Levels, Examples, and Challenges → Sentiment Analysis with NLTK VADER: Rule-Based Lexicon Approach → Sentiment Analysis with BERT via Hugging Face Pipeline → Comparing VADER and BERT for Sentiment Analysis → Sentiment Analysis with Flair → 1 more topics.

Coverage: 7 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W10-T01 Text Classification: Motivation and Real-World Applications

Tier A

Definition and why it matters: Modern organizations generate enormous volumes of unstructured text every minute:

Core explanation: Email and messaging Social media posts and product reviews Support tickets and chat logs Video titles, descriptions, and transcripts Concept flow: Unstructured Text - NLP Feature Extraction: or Neural Encoder - Classifier - Predefined Label Internal documents and knowledge bases Manual reading and labeling cannot keep pace. Text classification automates the assignment of predefined categories or labels to text documents using NLP and machine learning Text classification is the process of mapping an input text document x to a label y from a fixed set: y in $(c_1, c_2, \dots, c_k)$ Variants include multi-class (exactly one label) and multi-label (zero or more labels per document) Binary or multi-class filtering of unwanted email, comments, or messages Detect document language to trigger translation UI ("Translate this page?") - used by browsers and global content platforms Assign themes such as sports, politics, finance to articles for news aggregators and recommendation engines Classify user goal from utterances in chatbots and voice assistants: "Book a flight to Delhi" → bookflight "Cancel my reservation" → cancelbooking Critical for dialog systems on AWS Lex, Google Dialogflow, and Azure Bot Service Classify emotional tone (positive, negative, neutral) in reviews, tweets, and feedback - covered in depth in subsequent notes Text classification is among the highest-ROI NLP applications because it: Automates repetitive triage workflows Enables real-time routing (support queues, moderation pipelines) Powers analytics dashboards (topic trends, sentiment over time) Reduces operational cost vs human-only review Cloud ML context: Models deploy as REST endpoints (SageMaker, Vertex AI) or stream processors (Kafka → classifier → alert) for continuous ingestion

Must remember:

- Unstructured text grows faster than humans can label it - automation is mandatory at scale
- Text classification assigns predefined categories using NLP + ML
- Core applications: spam detection, language ID, topic labeling, intent recognition, sentiment analysis
- Gmail spam filtering is the canonical scale example
- Intent recognition powers chatbots and virtual assistants in cloud dialog platforms
- Text classification delivers direct business value through workflow automation and routing
- Sentiment analysis is a specialized subset of text classification focused on emotional tone

Formula / decision rule: to a label; y in $(c_1, c_2, \dots, c_k)$

Example or contrast: Approach: Manual; How it works: Human reads each document and assigns a label; Limitation: Impossible at scale; slow, expensive, inconsistent Approach: Automated; How it works: Model predicts label from text features; Limitation: Requires training data and model maintenance; scales horizontally on cloud infrastructure Example - Gmail spam filtering: Every incoming email must be classified as spam or not spam in milliseconds. A human-in-the-loop for each message is infeasible; a classifier runs on Google's infrastructure for every user simultaneously

Exam trap: Trap: Treating sentiment analysis as separate from text classification - it is a type of text classification with emotion labels Trap: Assuming text classification only means topic labeling - intent, spam, and language ID are equally valid exam categories

NLP-W10-T02 Sentiment Analysis: Levels, Examples, and Challenges

Tier A

Definition and why it matters: Sentiment analysis is a specialized form of text classification whose goal is to determine the emotional tone or polarity expressed in text. Standard labels: Positive - approval, satisfaction,

enthusiasm

Core explanation: One label for the entire document Entire movie review → positive / negative / neutral Aspect: food; Sentiment: positive Aspect: service; Sentiment: negative Single tweet → polarity of whole post Use case: App store rating trend dashboards aggregating thousands of reviews per day Concept flow: Document Level: One label per review - Sentence Level: Label per sentence - Aspect-Based: Label per feature Each sentence receives its own label within a longer document Multi-sentence product review: sentence 1 positive, sentence 2 negative Use case: Fine-grained monitoring of long-form feedback where different sentences express different opinions The most advanced level: identify what the sentiment targets and assign polarity per aspect Example: "The food was great, but the service was slow." Use case: Restaurant chains and e-commerce platforms track sentiment per feature (battery, camera, delivery speed) rather than one score per review Text: "The plot was unpredictable."; Domain: Thriller movie; Sentiment: Positive (engaging) Human language nuance breaks naive lexicon-based systems. Key challenge categories: Text: "The plot was unpredictable."; Domain: Driving manual; Sentiment: Negative (confusing) Surface words may be positive while intended meaning is negative "Oh great! My flight is delayed again." Words like "great" are positive in isolation; context flips polarity to negative Negators invert polarity; stacked negatives invert again Challenge: Sarcasm; Lexicon difficulty: High; Contextual model advantage: Moderate (still not perfect) "The movie was not bad." → effectively positive Challenge: Negation; Lexicon difficulty: Moderate (rules help); Contextual model advantage: Strong Both "not" and "bad" carry negative valence individually; together they yield positive sentiment Challenge: Domain context; Lexicon difficulty: High; Contextual model advantage: Strong with domain fine-tuning The same adjective carries opposite sentiment by domain Challenge: Polysemy; Lexicon difficulty: High; Contextual model advantage: Strong "The battery life is long." → positive (duration is good) "The wait time was long." → negative (delay is bad) Long is not inherently positive or negative - context determines valence Rule-based lexicons (VADER) encode heuristics for negation, intensifiers, and contrast conjunctions but struggle with sarcasm and domain polysemy. Contextual models (BERT, Flair) encode full-sentence meaning and generally handle complex cases better - at higher compute cost

Must remember:

- Sentiment analysis classifies emotional tone: positive, negative, or neutral
- Three granularity levels: document, sentence, aspect-based (ABSA)
- ABSA assigns polarity per feature ("food" positive, "service" negative)
- Sarcasm inverts surface word polarity - lexicons often fail here
- Negation and double negatives require compositional handling ("not bad" → positive)
- Domain and polysemy make the same word positive or negative depending on context
- Mixed reviews expose limitations of single document-level labels
- Contextual Transformer models generally outperform rule-based lexicons on hard cases

Exam trap: Trap: Labeling mixed reviews as purely positive or negative without specifying granularity level - document-level vs ABSA yields different correct answers Trap: Treating "not bad" as negative because it contains "bad" - double negation flips to positive

NLP-W10-T03

Sentiment Analysis with NLTK VADER: Rule-Based Lexicon Approach

Tier A

Definition and why it matters: VADER is a lexicon-based, rule-based model tuned for social media text. It ships with: Word: good; Example valence: +1.9 (approximate scale in lexicon)

Core explanation: Sentiment analysis can be implemented via: Traditional rule-based / lexicon methods - fast, interpretable, lightweight Modern deep learning (Transformer) methods - context-aware, higher accuracy, heavier compute This note covers the first approach using NLTK's VADER (Valence Aware Dictionary and sEntiment Reasoner) Key: neg; Meaning: Proportion of negative tone Key: neu; Meaning: Proportion of neutral tone Key: pos; Meaning: Proportion of positive tone Key: compound; Meaning: Normalized aggregate score in [-1, 1] The VADER lexicon is a specialized dictionary mapping words (and some phrases) to sentiment intensities For each input sentence, `sia.polarityscores(text)` returns: Typical defaults used in coursework: Concept flow: Input Text - VADER Lexicon: word valences - Rule Engine: caps, !, not, but - compound score - pos / neu / neg Code pattern: `import nltk; from nltk.sentiment import SentimentIntensity`

Analyzer; `nlk.download('vaderlexicon')`; `sia = SentimentIntensityAnalyzer()` VADER goes beyond summing word scores: Capitalization: "GOOD" is more intense than "good" Punctuation: "good!!!" "good" Degree modifiers: "extremely good" "good" Negation: "not good" flips valence Conjunctions: "good but bad" shifts weight toward the clause after but No GPU required - runs on CPU edge devices Code pattern: `scores = sia.polarityscores(sentence); compound = scores['compound']; if compound > 0.05; label = 'positive'; elif compound < -0.05; label = 'negative'; else; label = 'neutral'` Interpretable - inspect lexicon to explain scores Fast enough for high-throughput social media streams Reasonable on informal short text (tweets, reviews) Struggles with sarcasm, deep context, domain polysemy "I usually hate waiting, but this was worth it." → neutral (compound ~0) while humans and BERT often label positive Real-world use: Brand monitoring dashboards polling Twitter firehoses often start with VADER for speed, escalating uncertain cases to Transformer models

Must remember:

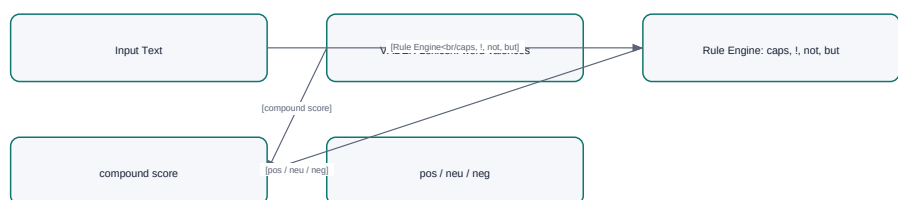
- VADER = Valence Aware Dictionary and sEntiment Reasoner (NLTK); rule + lexicon based
- Returns neg, neu, pos, and compound - classify using compound thresholds (0.05)
- Lexicon assigns valence scores; rules handle caps, punctuation, intensifiers, negation, "but"
- Fast, CPU-friendly, interpretable - ideal for social media monitoring at scale
- Weak on sarcasm and nuanced contrast; mixed sentences may score neutral incorrectly
- Contrasts with BERT: VADER = traditional; BERT = contextual deep learning (next notes)
- Always download vaderlexicon before initializing SentimentIntensityAnalyzer

Formula / decision rule: [-1, 1]

Example or contrast: Sentence: "I love this product."; compound (approx.): +0.67; Label: positive Sentence: "This is the worst experience ever."; compound (approx.): strongly negative; Label: negative Sentence: "The movie was OK. Nothing special."; compound (approx.): moderately negative; Label: negative

Exam trap: Trap: Using pos alone instead of compound for final classification - compound is the standard aggregated score Trap: Forgetting thresholds are configurable - 0.05 / -0.05 are conventions, not universal constants

Sentiment Analysis with NLTK VADER: Rule-Based Lexicon Approach



Vector reconstruction of the source Mermaid visual

Concept map: Sentiment Analysis with NLTK VADER: Rule-Based Lexicon Approach

NLP-W10-T04 Sentiment Analysis with BERT via Hugging Face Pipeline

Tier A

Definition and why it matters: VADER scores individual words and applies rules - it never builds a deep representation of full sentence meaning

Core explanation: BERT applies bidirectional self-attention over the entire sentence. Every token's representation incorporates context from all neighbors, enabling better handling of: Sarcasm and irony (partially) Complex negation ("not uninteresting") Contrast structures ("hate waiting, but worth it") Sentence: "I love this product."; BERT label: positive; Confidence (typical): ~0.999+ Polysemy ("long battery" vs "long wait") Sentence: "This is the worst experience ever."; BERT label: negative; Confidence (typical): ~0.99+ BERT for sentiment uses a fine-tuned classification head on top of pre-trained encoders - weights already adapted on millions of labeled reviews available on Hugging Face Sentence: "The movie was OK. Nothing special."; BERT label: negative; Confidence (typical): ~0.99 (slightly lower) Generate token at huggingface.co/settings/tokens Sentence: "I usually hate waiting, but this was worth it."; BERT label: positive; Confidence (typical): ~0.999+

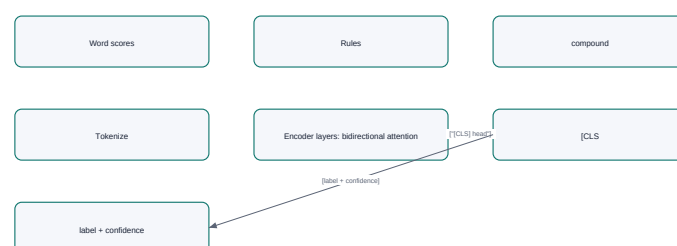
Store as HFTOKEN in Colab Secrets Sentence: "The food was good, but the service was terrible."; BERT label: negative; Confidence (typical): high (~0.9+) Grant notebook access The token enables downloading fine-tuned sentiment checkpoints from the hub Concept flow: Word scores - Rules - compound - Tokenize - Encoder layers: bidirectional attention - [CLS - label + confidence BERT-family models typically accept 512 tokens maximum. Longer inputs are truncated (usually from the end or with a strategy defined by the tokenizer). Without truncation, pipeline calls may error on long documents Contrast with VADER: The contrast sentence ("hate waiting, but worth it") is labeled neutral by VADER (compound ≈ 0) but positive by BERT - illustrating contextual understanding Tokenizer converts text $\rightarrow$ subword IDs BERT encoder produces contextual embeddings [CLS] token embedding passes through a linear classification layer Softmax yields label probabilities; highest wins The Hugging Face sentiment-analysis pipeline wraps tokenizer, model, and post-processing - no manual forward pass required for inference Default checkpoint (e.g., distilbert-base-uncased-finetuned-sst-2-english) is English review sentiment - other languages need multilingual models Code pattern: from transformers import pipeline; sentimentpipeline = pipeline("sentiment-analysis"; truncation=True required for texts 512 tokens;); sentences = ["I love this product."; "This is the worst experience ever.", Labels may appear as POSITIVE/NEGATIVE or LABEL0/LABEL1 depending on checkpoint - check model card Confidence reflects model probability, not calibrated human uncertainty Real-world use: Automated support ticket routing with high accuracy requirements often uses fine-tuned BERT served on GPU endpoints - acceptable latency trade-off for reduced mis-routing cost

Must remember:

- BERT sentiment uses bidirectional context - superior to lexicon methods on negation and contrast
- Hugging Face pipeline("sentiment-analysis") downloads a pre-fine-tuned review classifier
- Set truncation=True for inputs longer than 512 subword tokens
- Output: label + score (confidence); typically binary positive/negative
- BERT correctly labels "hate waiting, but worth it" as positive; VADER often neutral
- Requires HF token for gated models; heavier GPU compute than VADER
- Best for high-accuracy production routing (support tickets, compliance triage)

Exam trap: Trap: Claiming BERT "looks at individual words like VADER" - BERT uses full-sentence contextual representations Trap: Omitting truncation=True on long texts - silent truncation failures or errors on 512 tokens

Sentiment Analysis with BERT via Hugging Face Pipeline



Vector reconstruction of the source Mermaid visual

Concept map: Sentiment Analysis with BERT via Hugging Face Pipeline

NLP-W10-T05 Comparing VADER and BERT for Sentiment Analysis

Tier A

Definition and why it matters: After implementing sentiment analysis with NLTK VADER (rule-based lexicon) and BERT (fine-tuned Transformer), the natural question is: when should each be used?

Core explanation: They differ across speed, hardware, interpretability, accuracy, and deployment context Social media monitoring at high throughput (Twitter/X firehose, Reddit streams) Edge / mobile deployment with no GPU Explainability is required (regulatory, audit trails - "word X scored -2.5") Prototyping before investing in GPU infrastructure Sentence: "I love this product."; VADER: positive; BERT: positive Latency budget is milliseconds on cheap CPU instances Sentence: "This is the worst experience ever."; VADER: negative; BERT: negative Example: Real-time brand mention dashboard classifying 10k tweets/minute on a single cloud

VM without GPU Sentence: "The movie was OK. Nothing special."; VADER: negative; BERT: negative High accuracy is critical - misclassification has real cost Sentence: "I usually hate waiting, but this was worth it."; VADER: neutral; BERT: positive Automated support ticket routing (wrong queue = delayed resolution) Sentence: "The food was good, but the service was terrible."; VADER: negative; BERT: negative Complex language - negation, contrast, informal mixed sentiment GPU budget available (cloud inference endpoints, batch nightly processing) Willing to sacrifice interpretability for F1 score Example: Enterprise helpdesk auto-assigning P1 vs P3 priority from customer email bodies The contrast sentence exposes the core trade-off: VADER's but rule and compound aggregation miss overall positive intent; BERT's contextual encoding captures that the second clause dominates speaker intent Many systems combine both: Run VADER on all incoming text (fast filter) Escalate low-confidence or high-stakes items to BERT Human review for remaining edge cases This optimizes cost vs accuracy on cloud billing - GPU time only where needed

Must remember:

- VADER: fast, CPU, interpretable lexicon+rules - best for social media monitoring
- BERT: slower, GPU-heavy, black box - best for high-accuracy ticket routing and complex sentences
- BERT handles contrast ("hate X but worth it") better; VADER often returns neutral
- VADER explainability = inspect word valences; BERT = no simple dictionary explanation
- Choose by accuracy requirements, hardware budget, latency, and explainability needs
- Hybrid pipelines: VADER first pass, BERT for uncertain/high-value cases
- Neither solves sarcasm perfectly; BERT is materially stronger on negation and context

Example or contrast: Dimension: Approach; VADER (NLTK): Lexicon + hand rules; BERT (Hugging Face): Deep learning, contextual embeddings Dimension: Context; VADER (NLTK): Word-level + rule composition; BERT (Hugging Face): Full-sentence bidirectional attention Dimension: Speed; VADER (NLTK): Very fast; BERT (Hugging Face): Slower

Exam trap: Trap: "BERT is always better" - VADER wins on speed, cost, interpretability for social media at scale Trap: "VADER is not ML" - it is ML-adjacent rule-based NLP; exams contrast it with deep learning, not with "non-computational."

NLP-W10-T06 Sentiment Analysis with Flair

Tier A

Definition and why it matters: Flair is a PyTorch-based NLP library supporting sequence labeling tasks including POS tagging, NER, and sentiment analysis. Like BERT-based pipelines, Flair uses deep learning and contextual embeddings - not hand-written lexicon rules

Core explanation: Flair models often download weights from Hugging Face (may require HF_TOKEN for PyTorch checkpoints) Sentence: "I love this product."; Flair prediction: positive; Confidence: ~99% Sentence: "This is the worst experience ever."; Flair prediction: negative; Confidence: ~100% Package size is large - first install and model download take noticeable time compared to NLTK VADER Sentence: "The movie was OK. Nothing special."; Flair prediction: negative; Confidence: ~99.9% Sentence: "I usually hate waiting, but this was worth it."; Flair prediction: positive; Confidence: ~99% Wrap raw string in Sentence object Sentence: "The food was good, but the service was terrible."; Flair prediction: negative; Confidence: ~99% Load pre-trained TextClassifier with 'sentiment' tag Call classifier.predict(sentence) Concept flow: Raw text - Sentence object - TextClassifier.load sentiment - predict - label + confidence Read predicted label and probability from sentence annotations Flair aligns with BERT (not VADER) on the contrast sentence - labeling "worth it" dominance as positive Code pattern: Install: pip install flair; from flair.data import Sentence; from flair.models import TextClassifier Code pattern: text = "I love Delhi"; sentence = Sentence(text); classifier = TextClassifier.load('sentiment'); classifier.predict(sentence); print(sentence) shows labels and confidence; Example: Sentence[...] - positive (0.9912) Unified API for NER, POS, and sentiment (same library as earlier course modules) Strong accuracy on contextual sentences Pre-trained models readily available Code pattern: sentences = ["I love this product."; "This is the worst experience ever."; "The movie was OK. Nothing special."; "I usually hate waiting, but this was worth it."; "The food was good, but the service was terrible."]; classifier = TextClassifier.load('sentiment') Heavy dependencies (PyTorch + model weights) Slow cold start (download + load) Less ecosystem breadth than Hugging Face transformers for arbitrary checkpoints Real-world use: Research prototypes and coursework pipelines where Flair is already used for NER/POS and sentiment is added

with one consistent API - less common in large-scale production than raw Hugging Face serving

Must remember:

- Flair: PyTorch NLP library with pre-trained sentiment TextClassifier
- Pipeline: `Sentence(text) → TextClassifier.load('sentiment') → predict() → label + confidence`
- Large package; slow download but strong contextual accuracy
- Matches BERT on contrast sentences (positive for "hate waiting, but worth it")
- Same ecosystem as Flair NER/POS from earlier modules - consistent API
- Requires HF token when downloading certain hub-hosted weights
- Trade-off: heavier than VADER/TextBlob; more unified than ad-hoc multi-library stacks

Example or contrast: Library: VADER; Backend: Lexicon + rules; Speed: Fastest; Contextual?: No
Library: TextBlob; Backend: Pattern + TextBlob lexicon; Speed: Fast; Contextual?: Limited Library: Flair;
Backend: PyTorch neural tagger; Speed: Slow (large download); Contextual?: Yes

Exam trap: Trap: Forgetting to wrap strings in `Sentence` before `predict()` - plain strings won't work
Trap: Expecting instant startup - first `load('sentiment')` downloads weights; plan for cache or pre-baked Docker images in production

NLP-W10-T07 Sentiment Analysis with spaCy and TextBlob

Tier A

Definition and why it matters: spaCy is an industrial-strength NLP library focused on efficient pipelines (tokenization, tagging, parsing, NER). Sentiment analysis is not native to core spaCy - it is added via the `spacytextblob` extension, which integrates TextBlob polarity and subjectivity into spaCy Doc objects. This provides a third lightweight alternative alongside VADER (NLTK) and Transformer models (BERT, Flair).

Core explanation: Code pattern: Quiet install of English model + extension; `pip install spacy spacytextblob`; `python -m spacy download en_coreweb_sm`; `import spacy`; `from spacytextblob.spacytextblob import SpacyTextBlob`; `nlp = spacy.load('en_coreweb_sm')`; `nlp.addpipe('spacytextblob')` `en_coreweb_sm` (or larger English pipeline)
Score: Polarity; Measures: Sentiment direction; Range: [-1, 1] Score: Subjectivity; Measures: Opinion vs fact; Range: [0, 1] `spacytextblob` pipe registered on the `nlp` object TextBlob exposes two distinct scores per document: Opinion vs factual content 1 = highly subjective / opinion-based
Important: Subjectivity is independent of polarity - a factual negative news headline can have negative polarity but low subjectivity
Sentence: "I love this product."; VADER: positive; BERT/Flair: positive; spaCy/TextBlob: weak positive (0.62)
Integrates sentiment into existing spaCy pipelines (same Doc as entities, POS tags)
Sentence: Subjectivity score; VADER: No; BERT/Flair: No; spaCy/TextBlob: Yes
Returns subjectivity - useful for filtering opinion reviews vs factual reports
Lighter than BERT/Flair Polarity often underconfident on clearly positive short praise
Poor on contrast clauses compared to contextual models TextBlob backend is older Pattern-based logic - not Transformer-grade
Real-world use: Content moderation pipeline using spaCy NER + TextBlob polarity to flag subjective negative reviews mentioning product defects - subjectivity score helps separate opinion from spec-sheet comparisons
Code pattern: `sentences = [ "I love this product."; "This is the worst experience ever."; "The movie was OK. Nothing special."; "I usually hate waiting, but this was worth it."; "The food was good, but the service was terrible."; ];` for text in sentences:

Must remember:

- spaCy sentiment via `spacytextblob` extension integrating TextBlob into Doc objects
- Install `en_coreweb_sm` + register `spacytextblob` pipe on `nlp`
- Polarity ([-1,1]): negative $\leftrightarrow$ positive emotional tone
- Subjectivity ([0,1]): factual $\leftrightarrow$ opinion-based - unique among methods covered
- Access scores: `doc.polarity`, `doc.subjectivity`
- TextBlob backend: fast, integrated with spaCy NER pipeline, but weaker on contrast sentences
- "hate waiting, but worth it" $\rightarrow$ TextBlob negative; BERT/Flair positive; VADER neutral - classic exam comparison
- Compare all libraries on same test set when choosing a production approach

Formula / decision rule: -1; +1

Example or contrast: Code pattern: `text = "I am feeling very good."; doc = nlp(text); print(doc.polarity)` e.g., 0.90 (positive); `print(doc.subjectivity)` e.g., 0.78 (subjective opinion) Sentence: "I love this product."; `spaCy/TextBlob` polarity: ~0.62 (moderate positive); Other libraries: VADER/BERT/Flair: strong positive Sentence: "This is the worst experience ever."; `spaCy/TextBlob` polarity: ~-1.0 (very negative); Other libraries: All agree: negative

Exam trap: Trap: Treating polarity and subjectivity as the same - `subjectivity != sentiment`; factual text can be objective with zero polarity Trap: Forgetting `nlp.addpipe('spacytextblob')` - without it, `doc.polarity` doesn't exist

Week 10 Exam Lens

Likely questions

- Define and explain Text Classification: Motivation and Real-World Applications; include the mechanism and one limitation.
- Compare Text Classification: Motivation and Real-World Applications with Sentiment Analysis: Levels, Examples, and Challenges and state when each is preferred.
- Apply Sentiment Analysis with NLTK VADER: Rule-Based Lexicon Approach to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: to a label; y in $(c_1, c_2, \dots, c_k)$; $[-1, 1]$; -1 ; $+1$

Mistakes to avoid: Trap: Treating sentiment analysis as separate from text classification - it is a type of text classification with emotion labels Trap: Labeling mixed reviews as purely positive or negative without specifying granularity level - document-level vs ABSA yields different correct answers Trap: Using pos alone instead of compound for final classification - compound is the standard aggregated score

Week 11: Topic Modeling

Week roadmap: Introduction to Topic Modelling → Latent Dirichlet Allocation (LDA) → Key Assumptions of LDA → Implementing Topic Modelling with LDA → Why Alternatives to LDA Are Needed → BERTopic: Semantic Topic Modelling → 2 more topics.

Coverage: 8 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W11-T01 Introduction to Topic Modelling

Tier B

Definition and why it matters: Topic modelling is an unsupervised machine learning technique that discovers hidden themes in a collection of documents without requiring human labels. Instead of reading thousands of texts manually, the algorithm finds groups of words that tend to co-occur and treats each group as a latent "topic." Topic: Tourism; Representative Words: monuments, adventure, hiking, scenic

Core explanation: Modern systems generate text continuously - news feeds, research papers, customer reviews, social media posts, support tickets. Manual categorisation does not scale Topic modelling helps organisations: Concept flow: Large Text Corpus - Topic Modelling - Discovered Topics - Document Clustering - Search Indexing - Recommendations - Trend Analysis Summarise large document collections at a glance Identify recurring themes without predefined labels Group similar documents by latent theme Support downstream tasks such as search, recommendation, and trend monitoring The central question it answers: What are the main themes present in this corpus?

Must remember:

- Topic modelling is unsupervised discovery of latent themes in text corpora
- A topic is a distribution over words, not a single keyword
- Documents can express multiple topics in different proportions
- Unlike classification, no labelled data or predefined categories are required
- Industrial use cases include clustering, EDA, trend tracking, search, and recommendations
- LDA is the most widely used classical algorithm; embedding-based alternatives address its limitations

Example or contrast: Both work with text, but they solve different problems Aspect: Learning type; Text Classification: Supervised; Topic Modelling: Unsupervised Aspect: Labels required; Text Classification: Yes - predefined classes; Topic Modelling: No

Exam trap: Confusing topic modelling with classification - topic modelling does not need labels and does not use predefined classes; exam questions often swap these properties Assuming one topic per document - many algorithms (including LDA) model documents as mixtures of topics, not single assignments

NLP-W11-T02 Latent Dirichlet Allocation (LDA)

Tier A

Definition and why it matters: Latent Dirichlet Allocation (LDA) is the most widely used algorithm for topic modelling because it is:

Core explanation: Probabilistic - outputs interpretable probability distributions, not hard labels Concept flow: Document D - Topic: Technology (70% - Topic: Healthcare (30% - software, system, data, algorithm - patient, treatment, hospital, care Interpretable - topics are described by their top-weighted words Scalable - handles large document collections efficiently LDA answers two questions about a corpus: What topics exist? How is each document composed of those topics? LDA rests on two simple but powerful assumptions: A single document rarely discusses only one subject. A tech-health article might be 70% technology and 30% healthcare. LDA assigns topic proportions (not a single label) to each document Each topic is defined by a set of words with associated probabilities. The technology topic might weight software, system, data, and algorithm highly; the healthcare topic might weight patient, treatment, and hospital For a document with topic proportions (0.07, 0.86, 0.06) across three topics: The document is primarily about topic 1 (86% weight) Topics 0 and 2 contribute minimally Topic labels (e.g., "finance", "sports") must be inferred by inspecting the top words per topic - LDA does not name topics automatically

Must remember:

- LDA is probabilistic, interpretable, and scales to large corpora

- Documents = mixtures of topics; topics = distributions over words
- Output is topic proportions per document, not a single hard assignment
- Topic names are inferred manually from top-weighted words per topic
- K (number of topics) is a hyperparameter set by the practitioner
- LDA is the baseline classical algorithm before embedding-based alternatives like BERTopic

Formula / decision rule: (0.70, 0.30); (0.07, 0.86, 0.06)

Example or contrast: Entity: Document; Composition: 70% technology + 30% healthcare Entity: Technology topic; Composition: software, system, data, algorithm, Entity: Healthcare topic; Composition: patient, treatment, hospital, care,

Exam trap: "LDA assigns one topic per document" - false; it assigns proportions. The highest-proportion topic is often used as a proxy label, but that is a post-hoc simplification Confusing LDA with LSA - both are topic-modelling families, but LDA is generative and probabilistic; LSA uses matrix factorisation (SVD)

NLP-W11-T03 Key Assumptions of LDA

Tier A

Definition and why it matters: Every machine learning model simplifies reality to make inference tractable. LDA's assumptions explain both its strengths and its well-known failure modes. Understanding them is essential for preprocessing decisions and for knowing when to switch to modern alternatives

Core explanation: LDA treats a document as an unordered collection of words - identical to the bag-of-words (BoW) representation. "The dog bit the man" and "The man bit the dog" produce the same input Concept flow: LDA Assumptions - Bag of Words - no word order - Shared topics across corpus - Document = topic mixture - Conditional word independence - Heavy preprocessing required - Poor on short text - Motivation for contextual models Assumption: Bag of words; Practical Impact: Tokenisation, stop-word removal, and normalisation are critical Consequence: No syntactic or sequential context is preserved Assumption: Shared topics; Practical Impact: Topic count K must be chosen carefully for the whole corpus The same set of latent topics appears throughout the entire corpus. A "finance" topic discovered in one document is the same "finance" topic used to explain another Assumption: Topic mixtures; Practical Impact: Works on articles and reports; struggles on tweets and captions Consequence: LDA builds a global topic vocabulary; it does not create document-specific topic sets Assumption: Word independence; Practical Impact: Synonyms and paraphrases are not recognised unless they co-occur Each document can contain multiple topics in varying proportions. A news article might be 60% politics and 40% economics Consequence: LDA is well-suited to long, multi-theme documents but poorly suited to very short texts that express a single idea Once a topic is chosen, each word is drawn independently from that topic's word distribution. Co-occurrence is modelled at the topic level, not at the word-sequence level Consequence: Phrases and collocations ("New York", "machine learning") are not natively captured Because LDA ignores context, the signal comes entirely from which words appear and how often. Noisy tokens (stop words, punctuation, casing variants) dilute topic structure. A clean BoW representation directly improves topic coherence A tweet with 15 words provides too few co-occurrence statistics for LDA to infer a meaningful topic mixture. Models like GSDMM (which assume one topic per short document) address this gap LDA's inability to capture semantics, word order, and synonymy motivated embedding-based approaches (Word2Vec, BERT) and modern topic models like BERTopic that cluster documents in semantic embedding space

Must remember:

- LDA assumes bag of words: no word order or syntactic context
- Topics are shared globally across all documents in the corpus
- Each document is a mixture of topics, not a single-topic assignment
- Words are conditionally independent given the assigned topic
- These assumptions make preprocessing essential and explain LDA's weakness on short text
- Context-aware and embedding-based models were developed to overcome LDA's semantic blindness

Exam trap: Claiming LDA preserves context - it explicitly does not; word order is discarded Using LDA on tweets without preprocessing or alternative models - short text violates the mixture assumption in practice and yields incoherent topics

NLP-W11-T04

Implementing Topic Modelling with LDA

Tier A

Definition and why it matters: This lab implements classical topic modelling using Gensim's LDA on a small custom corpus. The pipeline covers preprocessing, bag-of-words vectorisation, model training, and topic interpretation - the standard workflow used in production analytics pipelines before deploying transformer-based alternatives

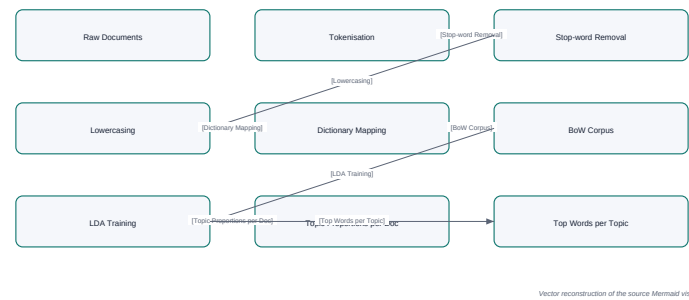
Core explanation: Concept flow: Raw Documents - Tokenisation - Stop-word Removal - Lowercasing - Dictionary Mapping - BoW Corpus - LDA Training - Topic Proportions per Doc - Top Words per Topic Library: Gensim; Role: Dictionary, LdaModel for BoW and LDA Library: NLTK; Role: Stop words, tokenisation A sample corpus might contain sentences about stock markets, machine learning, and football: "Stock markets reacted to inflation news." → themes: finance, economics "Deep learning models require large datasets." → themes: ML, data "The football team is preparing for the World Cup." → themes: sports Before running the model, manually hypothesise 2-3 themes per sentence - then compare against LDA output to build intuition Standard steps applied to each document: Tokenise into words Remove stop words (the, is, to, ...) Lowercase all tokens (normalisation) Output: a list of token lists, one per document Topic Index: 0; Confidence: 7%; Interpretation: Minor contribution Topic Index: 1; Confidence: 86%; Interpretation: Dominant topic Dictionary maps each unique word to an integer ID doc2bow converts each document into a sparse vector of (wordid, count) pairs This is the input representation LDA expects Code pattern: from gensim import corpora; from gensim.models import LdaModel; from nltk.corpus import stopwords; from nltk.tokenize import word_tokenize numtopics=3 is a hyperparameter. There is no automatic way to choose K; practitioners experiment or use coherence metrics Topic: 0; Top Words: learning, datasets, model, deep, large; Inferred Label: Machine Learning Topic: 1; Top Words: markets, reacted, inflation, news, stock; Inferred Label: Finance / Economics Example output for document 0: [(0, 0.07), (1, 0.86), (2, 0.06)] Topic: 2; Top Words: football, team, World, Cup, preparing; Inferred Label: Sports The dominant topic (highest probability) is often used as a proxy label, but the full distribution is the true output Example discovered topics from a mixed corpus: Code pattern: dictionary = corpora.Dictionary(processeddocs); corpus = [dictionary.doc2bow(doc) for doc in processeddocs] Topic labels are manual - inspect top words and assign meaningful names. This is standard practice in industry dashboards After running the model, evaluate quality: Code pattern: ldamodel = LdaModel(; corpus=corpus,; id2word=dictionary,; numtopics=3, K - must be set manually; randomstate=42;) Do topics look clean and semantically distinct? Are the same words repeated across multiple topics (topic overlap)? Code pattern: for docbow in corpus; print(ldamodel.getdocumenttopics(docbow)) Do the inferred labels match your manual hypotheses from Step 2? Does changing numtopics improve or degrade coherence? Replace the sample corpus with your own dataset (e.g., product reviews, news headlines) and rerun the full pipeline. Compare topic quality across domains Code pattern: ldamodel.printtopics(numwords=5); or; ldamodel.showtopics(numwords=10)

Must remember:

- LDA pipeline: preprocess → dictionary → BoW corpus → train → interpret
- Gensim LdaModel takes corpus, id2word, and numtopics
- getdocumenttopics() returns proportional weights, not hard labels
- printtopics() / showtopics() reveal top words for manual labelling
- Preprocessing (tokenise, stop words, lowercase) is mandatory for clean topics
- Always validate topic quality and experiment with K on your own corpus

Exam trap: Forgetting to set numtopics - LDA requires K upfront; there is no default optimal value Treating confidence scores as classification probabilities - they are topic proportions within a generative model, not calibrated class probabilities

Implementing Topic Modelling with LDA



Concept map: Implementing Topic Modelling with LDA

NLP-W11-T05

Why Alternatives to LDA Are Needed

Tier A

Definition and why it matters: LDA remains widely deployed in industry for exploratory analysis on large document collections. Its probabilistic framework, interpretability, and scalability made it the default topic model for over a decade. However, as NLP advanced and text sources diversified, LDA's simplifying assumptions became limiting rather than enabling

Core explanation: As NLP evolved from statistical co-occurrence to distributed representations, a new class of topic models emerged: Scenario: Long articles, reports, research papers; Recommended Approach: LDA (with careful preprocessing) Scenario: Short social media posts, reviews; Recommended Approach: GSDMM or BERTopic Scenario: Need semantic grouping of customer feedback; Recommended Approach: BERTopic Scenario: Large corpus, need fast baseline; Recommended Approach: LDA Scenario: Domain with rich contextual language; Recommended Approach: Embedding-based models Sentence/document embeddings (from BERT and similar transformers) capture semantic meaning Clustering in embedding space groups semantically similar documents without bag-of-words assumptions Classical TF-IDF on clusters still provides interpretable topic words This hybrid approach - deep embeddings for grouping, classical NLP for labelling - powers models like BERTopic

Must remember:

- LDA ignores context, word order, and semantic similarity (bag-of-words assumption)
- Topic count K must be manually tuned - no automatic selection
- LDA performs poorly on short text (< 50 words): tweets, captions, reviews
- Embedding-based methods address LDA's semantic blindness
- GSDMM targets short-text, single-topic documents; BERTopic uses transformer embeddings
- Model choice depends on text length, domain, and whether semantic grouping is required

Formula / decision rule: must be tuned by hand | Wrong

Exam trap: Limitation: Bag of words; What It Means: No context retained; Real-World Impact: "bank" (river) and "bank" (finance) treated identically unless co-occurrence differs Limitation: No word order; What It Means: Sentence structure ignored; Real-World Impact: Negation and phrasing lost ("not good" ~="good")

NLP-W11-T06

BERTopic: Semantic Topic Modelling

Tier A

Definition and why it matters: BERTopic is a modern topic modelling technique that: Uses a transformer model (typically BERT-based) to create dense sentence/document embeddings

Core explanation: Classical LDA models word co-occurrence in a bag-of-words space. It cannot recognise that "cloud computing" and "AWS infrastructure" belong to the same theme unless those exact words co-occur frequently. BERTopic addresses this by operating in semantic embedding space rather than raw word-count space Instead of modelling word distributions directly (as LDA does), BERTopic: Concept flow: Documents - BERT Embeddings - Cluster by Semantic Similarity - Extract Topic Words per Cluster - Named Topics Aspect: Representation; LDA: Bag of words; BERTopic: Dense embeddings Aspect: Semantics; LDA: Co-occurrence only; BERTopic: Contextual meaning Aspect: Word order; LDA: Ignored; BERTopic: Captured in embeddings Aspect: Short text; LDA: Poor; BERTopic: Better (embedding quality helps) Aspect: Topic

words; LDA: Global word distributions; BERTopic: Per-cluster TF-IDF Aspect: Topic count; LDA: Manual K; BERTopic: Clustering algorithm determines groups Groups semantically similar documents together Derives topic labels from the words most characteristic of each cluster Consider a corpus of ML platform reviews mentioning "SageMaker", "model deployment", "inference latency", and "endpoint scaling". LDA might split these across topics based on word frequency alone. BERTopic groups the reviews semantically because the embeddings capture that these phrases describe the same operational concern - model serving infrastructure Customer feedback analysis where paraphrasing is common Research paper clustering with varied terminology Any corpus where meaning matters more than exact word overlap Situations where LDA produces overlapping or incoherent topics

Must remember:

- BERTopic uses transformer embeddings to capture semantic document similarity
- Documents are clustered in embedding space, not bag-of-words space
- Topic words are extracted per cluster using a TF-IDF variant (c-TF-IDF)
- It addresses LDA's lack of context, semantics, and poor short-text handling
- The pipeline: embed → dimensionality reduction → cluster → topic representation
- Best for corpora where meaning and paraphrase variation matter

Exam trap: Saying BERTopic is "just LDA with BERT" - the architecture is fundamentally different: embed → reduce → cluster → c-TF-IDF Expecting BERTopic to need no classical NLP - it still uses TF-IDF (c-TF-IDF) for interpretable topic words

NLP-W11-T07 BERTopic Architecture: End-to-End Pipeline

Tier A

Definition and why it matters: BERTopic transforms raw documents into named, interpretable topics through five sequential stages. Each stage solves a specific problem: semantic representation, computational tractability, grouping, labelling, and diversity optimisation

Core explanation: Concept flow: Input Documents - BERT / Sentence Transformer - Dense Embeddings - UMAP - Dimensionality Reduction - HDBSCAN - Clustering - c-TF-IDF - Topic Words - MMR - Diversity Optimisation - Final Topics Property: Preserves local structure; UMAP: Yes (emphasised); PCA: Global variance Property: Non-linear; UMAP: Yes; PCA: No Component: Pretrained transformer (e.g., BERT, Sentence-BERT) Property: Clustering-friendly; UMAP: Yes; PCA: Moderate Purpose: Convert each document into a dense vector that encodes semantic meaning Captures context, synonyms, and paraphrases Output: high-dimensional embedding per document (e.g., 384 or 768 dimensions) Intuition: Documents about "Kubernetes scaling" and "container orchestration autoscaling" land near each other in vector space even if they share few exact words Component: UMAP (Uniform Manifold Approximation and Projection) Purpose: Compress high-dimensional embeddings into a lower-dimensional space suitable for clustering Concept flow: Embedding 1 - Cluster A - Embedding 2 - Embedding 3 - Cluster B - Embedding 4 - Embedding 5 - Cluster C UMAP keeps semantically similar documents close together while reducing computational cost for the clustering step Component: HDBSCAN (Hierarchical Density-Based Spatial Clustering) Purpose: Group documents by semantic similarity using density-based clustering Documents in the same cluster discuss similar themes No fixed K required - the number of topics emerges from data density Uses cosine similarity on reduced embeddings Outlier documents may be assigned to no cluster (noise points) Component: Class-based TF-IDF (c-TF-IDF) Purpose: Extract the most representative words for each cluster Standard TF-IDF scores words across all documents. c-TF-IDF computes term frequency and inverse document frequency per cluster: Words frequent in one cluster but rare in others become top topic terms Produces human-readable topic descriptions (e.g., markets, inflation, stock, reacted) Component: Maximal Marginal Relevance Purpose: Optimise topic word lists for diversity - reduce redundant or near-duplicate terms in the top-N words Not present in all BERTopic variants Enhances readability of topic labels in dashboards and reports

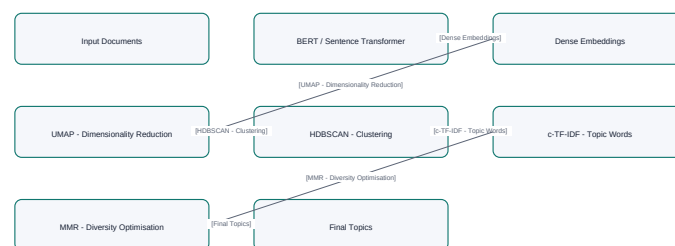
Must remember:

- Step: 1; Input: Raw text; Output: Dense vectors; Key Algorithm: Sentence Transformer
- Step: 2; Input: Dense vectors; Output: Reduced vectors; Key Algorithm: UMAP
- Step: 3; Input: Reduced vectors; Output: Cluster assignments; Key Algorithm: HDBSCAN

- Step: 4; Input: Clusters + documents; Output: Top words per topic; Key Algorithm: c-TF-IDF
- Step: 5; Input: Top words; Output: Diversified word list; Key Algorithm: MMR (optional)
- BERTopic pipeline: Embed → UMAP → HDBSCAN → c-TF-IDF → MMR (optional)
- BERT/sentence transformers produce semantic document embeddings
- UMAP reduces dimensionality while preserving local structure for clustering
- HDBSCAN clusters by density using cosine similarity; topic count is data-driven
- c-TF-IDF extracts representative words per cluster for interpretability
- MMR optionally diversifies top topic words to reduce redundancy

Exam trap: Listing only "BERT + clustering" - the full pipeline includes UMAP, HDBSCAN, c-TF-IDF, and optionally MMR Confusing UMAP with PCA - UMAP is non-linear and preserves local neighbourhood structure; PCA is linear

BERTopic Architecture: End-to-End Pipeline



Concept map: BERTopic Architecture: End-to-End Pipeline

NLP-W11-T08

GSDMM: Topic Modelling for Short Text

Tier A

Definition and why it matters: This one-topic-per-document assumption aligns with how people write tweets and reviews - one thought, one theme Full name: Gibbs Sampling Dirichlet Multinomial Mixture

Core explanation: LDA assumes each document is a mixture of topics. Long articles naturally satisfy this - a 2,000-word report might discuss technology, regulation, and market impact simultaneously. Short texts violate this assumption in practice: Product reviews (1-3 sentences) Image captions on social media These snippets typically express one theme. With fewer than 50 words, LDA lacks sufficient co-occurrence signal to infer meaningful topic mixtures GSDMM (Gibbs Sampling Dirichlet Multinomial Mixture) was designed specifically for this scenario Step: Drop missing values; Purpose: Ensure clean input Step: Unicode normalisation; Purpose: Handle encoding variants (UTF-8, ASCII) Step: Lowercasing; Purpose: Normalise case Step: Regex cleaning; Purpose: Remove non-alphanumeric characters Step: Stop-word removal; Purpose: Filter function words Step: Stemming (Porter); Purpose: Reduce elections - elect, remembered - rememb Short-text corpus example: tweets from a public archive (JSON format with a text column containing tweet content and meta-data columns) Parameter: K; Role: Number of topics; Tuning: Set based on domain knowledge Dependencies: pandas, numpy, json, nltk Parameter: alpha; Role: Document-cluster density; Tuning: Experiment per dataset Parameter: beta; Role: Word-cluster density; Tuning: Experiment per dataset Parameter: niters; Role: Sampling iterations; Tuning: Increase until convergence Rank clusters by document count (most populated = most important theme): Code pattern: git clone provides MovieGroupProcess class Inspect top words per cluster via mgp.clusterworddistribution(): Cluster 4: elect, vote, state, Georgia, people → Elections Cluster 10: great, thank, news, vaccine, get → Vaccination Cluster 8: nation, section, defence, security → National Security Build a DataFrame with columns: text, topic, stems threshold=0.3: documents below 30% confidence are grouped into an "Other" category topicdict: maps cluster indices to human-readable names Code pattern: from gsdmm import MovieGroupProcess; mgp = MovieGroupProcess(; K=15, number of topics (manual); alpha=0.1, cluster-document hyperparameter; beta=1.0, cluster-word hyperparameter; niters=30 Gibbs sampling iterations;); mgp.fit(docs, vocabsize) Code pattern: Cluster index 4: 630 documents - Topic: Elections/Voting; Cluster index 10: 233 documents - Topic: Vaccination/Health; Cluster index 8: 45 documents - Topic: National Security Code pattern: def createtopicsdataframe(datatext, mgp, threshold=0.3, topicdict, stemtexts); Assign topic if probability = threshold, else "Other"

Must remember:

- GSDMM = Gibbs Sampling Dirichlet Multinomial Mixture, designed for short text
- Assumes one topic per document (opposite of LDA's mixture assumption)
- Best for tweets, captions, and product reviews (< 50 words)
- Implemented via MovieGroupProcess class from the GSDMM library
- Pipeline: load JSON → clean → stem → stop words → fit → inspect clusters → package DataFrame
- Rank clusters by document count; label topics from top word distributions manually
- Use probability threshold to filter low-confidence assignments into "Other"

Example or contrast: Property: Document-topic model; LDA: Mixture of topics; GSDMM: One topic per document Property: Best for; LDA: Long documents, articles; GSDMM: Short text, tweets, reviews Property: Text length; LDA: Hundreds+ words; GSDMM: < 50 words typical

Exam trap: Using LDA for tweets in an exam - the correct short-text answer is GSDMM (or BERTopic) Confusing GSDMM's one-topic assumption with LDA's mixture assumption - this is the primary differentiator

Week 11 Exam Lens**Likely questions**

- Define and explain Latent Dirichlet Allocation (LDA); include the mechanism and one limitation.
- Compare Latent Dirichlet Allocation (LDA) with Key Assumptions of LDA and state when each is preferred.
- Apply Implementing Topic Modelling with LDA to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: (0.70, 0.30); (0.07, 0.86, 0.06); must be tuned by hand | Wrong

Mistakes to avoid: Confusing topic modelling with classification - topic modelling does not need labels and does not use predefined classes; exam questions often swap these properties "LDA assigns one topic per document" - false; it assigns proportions. The highest-proportion topic is often used as a proxy label, but that is a post-hoc simplification Claiming LDA preserves context - it explicitly does not; word order is discarded

Week 12: Natural Language Generation with LLMs

Week roadmap: Why Decoder-Only Models Dominate NLG → What Is a Large Language Model (LLM)? → Why LLMs Excel at Natural Language Generation → Natural Language Generation (NLG) → How LLMs Generate a Response → Controlling LLM Output: Decoding Strategies → 8 more topics.

Coverage: 14 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W12-T01 Why Decoder-Only Models Dominate NLG

Tier A

Definition and why it matters: Natural Language Generation (NLG) requires producing coherent, fluent text one token at a time. Not all transformer architectures are equally suited to this task. Decoder-only models - the architecture behind GPT, Gemini, and Llama - dominate NLG because their training objective directly matches how text is produced

Core explanation: Decoder-only transformers model the probability of the next token given all preceding tokens: Concept flow: Context - Token 1 - Token 2 - Token 3 - Token N Property: Training objective; Decoder-Only: Next-token prediction; Encoder-Only (BERT): Masked token prediction; Encoder-Decoder (T5): Seq2seq (input - output) $P(t_n | t_1, t_2, \dots, t_{(n-1)})$ Property: Natural for text generation; Decoder-Only: Yes - same as inference; Encoder-Only (BERT): No - bidirectional, not generative; Encoder-Decoder (T5): Yes, but more complex Generation is sequential and autoregressive: Property: Scales with data and parameters; Decoder-Only: Extremely well; Encoder-Only (BERT): Good for understanding; Encoder-Decoder (T5): Good for translation/summarisation Given context, predict token t_1 Property: Dominant in chat/completion APIs; Decoder-Only: Yes (GPT, Gemini, Llama); Encoder-Only (BERT): No; Encoder-Decoder (T5): Moderate (specialised tasks) Append t_1 to context; predict t_2 Append t_2 ; predict t_3 Continue until an end-of-sequence token or length limit Each generated token becomes part of the context for the next prediction. The model learns: given this sequence so far, what is the most likely next token? The simplicity of next-token prediction scales remarkably well with model size and training data - a key reason decoder-only models power modern chatbots, code assistants, and content generators BERT-style encoder-only models use masked language modelling - predicting missing tokens using bidirectional context. This excels at understanding (classification, NER, embeddings) but does not naturally generate text left-to-right. Decoder-only models are trained exactly as they are used at inference time

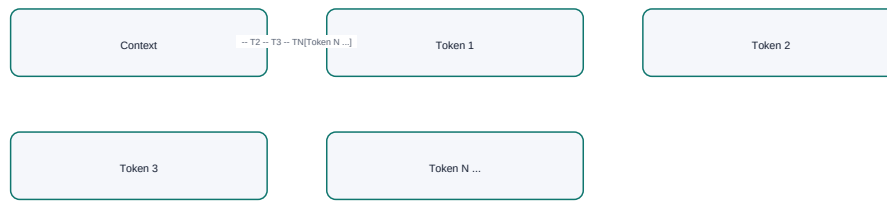
Must remember:

- Decoder-only transformers dominate NLG because they model next-token probability
- Generation is autoregressive: each token is appended to context for the next prediction
- Training objective (next-token prediction) matches inference behaviour exactly
- This simple objective scales well with parameters and data
- Encoder-only models (BERT) excel at understanding, not open-ended generation
- GPT, Gemini, and Llama are decoder-only architectures

Formula / decision rule: $P(t_n | t_1, t_2, \dots, t_{(n-1)}); t_1$

Exam trap: Claiming BERT is used for open-ended text generation - BERT is encoder-only; GPT-style decoder-only models dominate NLG Confusing autoregressive with parallel generation - tokens are generated one at a time; parallelism happens only across batch items, not within a single sequence

Why Decoder-Only Models Dominate NLG



Vector reconstruction of the source Mermaid visual

Concept map: Why Decoder-Only Models Dominate NLG

NLP-W12-T02

What Is a Large Language Model (LLM)?

Tier A

Definition and why it matters: A model earns the label Large Language Model based on three scaling dimensions - not a fixed numerical threshold. "Large" is relative and evolves as hardware and research advance

Core explanation: Dimension: Parameters; What Scales: Model weights; Typical Scale: Millions to billions (10^9+) Dimension: Training data; What Scales: Text corpus size; Typical Scale: Internet-scale: Wikipedia, Stack Overflow, Reddit, books, web crawl Dimension: Sequence length; What Scales: Context window for next-token prediction; Typical Scale: Thousands of tokens (modern models: 128K+) Concept flow: Context: I love natural language processing. NLP is very - LLM - Next token: powerful What matters is that scaling produces emergent capabilities - behaviours not present in smaller models, such as multi-step reasoning, code generation, and instruction following LLMs are pre-trained with one deceptively simple objective: Given a sequence of tokens (context), predict the next token Pre-training is self-supervised: the training signal comes from the text itself (predict the next word). No human annotation of "correct answers" is needed. This is why LLMs can be trained on trillions of tokens from the open web After pre-training, models may undergo fine-tuning or RLHF for instruction following - but the foundation is always next-token prediction at scale As parameters, data, and compute increase, LLMs exhibit capabilities that smaller models lack: Multi-paragraph coherent writing Code generation and debugging Translation and summarisation without task-specific training Following complex natural-language instructions (with alignment tuning) These emerge from scale - they are not explicitly programmed

Must remember:

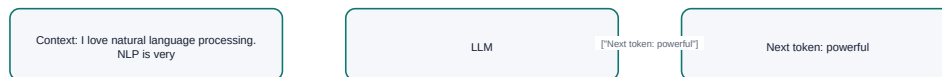
- LLMs are defined by scale: parameters, training data, and sequence length
- No strict quantitative threshold - "large" is relative
- Pre-training objective: given context, predict the next token
- Pre-training is self-supervised - no task labels required
- Scaling leads to emergent NLG capabilities
- Training data (Wikipedia, web, code) determines factual associations and style

Formula / decision rule: 10^9

Example or contrast: Context: "I love natural language processing. NLP is very"; Possible Next Tokens: powerful, interesting, useful, Context: "The cat sat on the"; Possible Next Tokens: mat, table, floor, desk, The chosen token depends on training data statistics and context. The model learns grammar, factual associations, writing style, and discourse structure - all implicitly, without task-specific labels

Exam trap: Citing a fixed parameter count as the LLM threshold - there is no strict cutoff; "large" is relative across eras Confusing pre-training with fine-tuning - pre-training uses next-token prediction on raw text; fine-tuning uses labelled or preference data

What Is a Large Language Model (LLM)?



Vector reconstruction of the source Mermaid visual

Concept map: What Is a Large Language Model (LLM)?

NLP-W12-T03

Why LLMs Excel at Natural Language Generation

Tier A

Definition and why it matters: Before transformer-based LLMs, generating multi-paragraph coherent text was difficult. RNNs and LSTMs degraded over long sequences. LLMs changed this through scale, attention, and a training objective aligned with generation

Core explanation: RNNs suffer from vanishing gradients - after several sentences, early context is effectively forgotten. Transformer self-attention lets every token attend to every other token, preserving context across hundreds or thousands of tokens

Concept flow: Prompt / Instructions - LLM - Formal summary - Child-friendly explanation - Bullet-point list

What It Seems Like: Reasoning and thinking; What Actually Happens: Statistical pattern matching over training data

What It Seems Like: Understanding the question; What Actually Happens: Conditioning on token sequences

What It Seems Like: Choosing the "right" answer; What Actually Happens: Selecting the highest-probability next token

Example: In a 10-paragraph essay prompt, an LLM can maintain thematic consistency from paragraph 1 to paragraph 10

LLMs generate essays, explanations, and multi-paragraph responses that maintain logical flow - a capability largely absent in pre-transformer NLG systems

The same model produces different outputs depending on the prompt (input instructions). Change the prompt from "summarise this" to "explain this to a 10-year-old" and the output style, depth, and vocabulary shift accordingly

LLMs appear intelligent and reasoning-capable, but they are fundamentally next-token prediction engines

Because generation is probabilistic and prompt-conditioned: Vague prompts produce vague outputs

Specific format instructions produce structured outputs

The same model can be brilliant or useless depending on prompt design

This directly motivates prompt engineering as a core LLM skill

Must remember:

- LLMs capture long-range dependencies via transformer attention (unlike RNNs)
- They generate coherent multi-sentence and multi-paragraph text
- Output adapts to prompt instructions without retraining
- LLMs are next-token prediction engines, not reasoning systems
- Token selection is probabilistic - highest-probability token wins
- Output quality depends heavily on prompt design

Formula / decision rule: $P(\text{mat} \mid \text{context})$

Example or contrast: The model might output mat, but table, floor, and desk were also candidates. Mat wins because its conditional probability $P(\text{mat} \mid \text{context})$ was highest - not because the model "understood" furniture semantics

Exam trap: Attributing genuine reasoning to LLMs - they predict tokens probabilistically; chain-of-thought is pattern mimicry, not symbolic reasoning

Ignoring prompt impact - exam questions about output quality should reference prompting, not just model size

NLP-W12-T04

Natural Language Generation (NLG)

Tier A

Definition and why it matters: Natural Language Generation (NLG) is the task of producing human-readable text from some form of structured or unstructured input. In modern systems, that input is typically a prompt - a natural-language instruction that tells the model what to generate

NLG sits on the output side of the NLP pipeline: where NLU extracts meaning from text, NLG creates text from meaning, data, or instructions

Core explanation: Concept flow: Hand-crafted Rules - Templates - Domain-specific Output - Pre-trained LLM - Prompt - Probabilistic Text Generation Rule-based - explicit if-then logic for sentence construction Template-driven - fill-in-the-blank patterns ("Your order {id} will arrive on {date}") Highly task-specific - an e-commerce NLG system could not generalise to healthcare without rebuilding from scratch Brittle - edge cases and language variation broke rules easily Poor generalisation - each new domain required new engineering Single pre-trained model serves multiple domains Implicit knowledge learned from training data - no explicit rules Prompt-driven adaptation - change the task by changing the prompt, not the code Fluent and coherent - probabilistic generation produces natural-sounding prose A traditional weather-report generator needed templates for every phrasing variant. An LLM generates varied, natural weather descriptions from a single prompt: "Write a morning weather briefing for Mumbai in a conversational tone." Summarisation - condense long text into a short summary Text expansion - expand bullet points into full explanations Paraphrasing - rewrite while preserving meaning Explanation - generate educational content on any topic Creative writing - stories, poetry, marketing copy

Must remember:

- NLG produces human-readable text from input (typically a prompt)
- Traditional NLG: rule-based, template-driven, domain-specific, brittle
- LLM-based NLG: single model, prompt-adapted, fluent, generalises across tasks
- Key shift: explicit rules → implicit knowledge from training data
- Common tasks: summarisation, expansion, paraphrasing, explanation
- Modern NLG quality depends on both model capability and prompt design

Example or contrast: Aspect: Knowledge source; Traditional NLG: Explicit rules and templates; LLM-Based NLG: Implicit patterns from training data Aspect: Domain adaptation; Traditional NLG: Rebuild system per domain; LLM-Based NLG: Change the prompt Aspect: Fluency; Traditional NLG: Often robotic; LLM-Based NLG: Human-like

Exam trap: Treating NLG as deterministic - LLM output is probabilistic; traditional NLG with fixed templates is deterministic Claiming LLMs use explicit rules - they use learned statistical patterns

NLP-W12-T05 How LLMs Generate a Response

Tier A

Definition and why it matters: LLMs do not:

Core explanation: Retrieve answers from a database Concept flow: Prompt Tokens - LLM - Token 1 - Token 2 - ... until stop - Full Response Perform symbolic reasoning Verify facts against ground truth Task: Summarisation; Description: Condense long text; Example Prompt: "Summarise this article in 3 sentences" LLMs only generate the most probable next token, repeated autoregressively until completion Task: Text expansion; Description: Expand brief input; Example Prompt: "Expand these bullet points into a paragraph" Task: Paraphrasing; Description: Rewrite preserving meaning; Example Prompt: "Rephrase this in simpler language" LLM output may sound authoritative but be factually incorrect. The model optimises for plausible text, not truth Task: Explanation; Description: Teach a concept; Example Prompt: "Explain quantum entanglement to a beginner" If training data states Bill Clinton was president during the data cutoff, the model may answer "Bill Clinton" even when the real-world answer has changed Knowledge cutoff date limits factual accuracy for events after training Output reflects $P(\text{token} \mid \text{context})$ from training statistics - not verified real-world facts. This is why critical applications need: Retrieval-Augmented Generation (RAG) Grounding with search APIs

Must remember:

- Single-turn: one prompt, one response, no prior context retained
- Multi-turn: conversation history enables pronoun resolution and follow-ups
- LLMs generate probable tokens - they do not retrieve or reason symbolically
- Output may be confident but incorrect (hallucination)
- Factual accuracy is limited by training data and knowledge cutoff
- Common NLG tasks: summarisation, expansion, paraphrasing, explanation
- Critical interpretation and better prompting mitigate risks

Formula / decision rule: $P(\text{token} \mid \text{context})$

Example or contrast: This course focuses on single-turn text generation: one prompt, one response, no memory of prior interactions Mode: Single-turn; Behaviour: Each prompt is independent; Example: "Summarise this paragraph" Mode: Multi-turn; Behaviour: Prior Q&A is retained as context; Example: Q1: "Who is the US president?" - Q2: "What is his age?" (resolves "his")

Exam trap: Code pattern: Prompt 1: "Who is the president of USA?"; Response: "Donald Trump."; Prompt 2: "What is his age?" (no shared context in single-turn); Response: "Whose age do you want?" or "Who is he?" In multi-turn mode, the model retains conversation history and correctly resolves pronouns

NLP-W12-T06 Controlling LLM Output: Decoding Strategies

Tier A

Definition and why it matters: LLMs are largely black boxes - internal weights cannot be directly controlled at inference time. However, decoding strategies and hyperparameters shape how the model samples from its output probability distribution, giving practitioners control over creativity, determinism, and diversity

Core explanation: Top-K restricts candidate tokens to the K most probable options, renormalises their probabilities, and samples from that subset K Value: $K = 1$; Behaviour: Greedy decoding - always picks highest-probability token; Use Case: Maximum determinism K Value: Small K (e.g., 5); Behaviour: Focused, conservative output; Use Case: Factual Q&A, data extraction K Value: Large K (e.g., 50); Behaviour: Diverse, creative, riskier output; Use Case: Brainstorming, creative writing P Value: Low (e.g., 0.7); Behaviour: Focused, safe generation P Value: High (e.g., 0.95); Behaviour: Diverse, expressive generation Top-P selects the smallest set of tokens whose cumulative probability exceeds threshold p , then samples from that set Temperature: 0.0 - 0.3; Effect: Deterministic, focused, conservative; Recommended For: Coding, data extraction, factual Q&A Temperature: 0.4 - 0.7; Effect: Balanced; Recommended For: General-purpose applications Temperature: 0.7 - 1.0; Effect: Creative, random, diverse; Recommended For: Brainstorming, poetry, creative writing Temperature: 1.0 (up to 2.0); Effect: Increasingly nonsensical; Recommended For: Not recommended Temperature scales the logits before softmax, controlling randomness: $P(\text{token}_i) = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$ Some APIs allow $T \leq 1$; beyond 1.0, output becomes incoherent as the model loses control over token selection

Must remember:

- LLM output variability comes from probabilistic sampling, not random model changes
- Top-K: sample from the K most probable tokens; $K=1$ is greedy decoding
- Top-P (nucleus): sample from smallest set with cumulative probability $\geq p$; more flexible than Top-K
- Temperature (0-1 typical): low = deterministic, high = creative; 1 causes nonsense
- Use low temperature for coding/Q&A; high temperature for creative writing
- Top-P with moderate values (e.g., 0.9) is widely used in production

Formula / decision rule: p_{-1} ; p_{-2}

Example or contrast: Candidate tokens: mat (p_1), table (p_2), desk (p_3), floor (p_4), Token: mat; Probability: 0.20; Cumulative: 0.20 Token: table; Probability: 0.15; Cumulative: 0.35

Exam trap: Confusing Top-K and Top-P - K fixes the count; P fixes the cumulative probability threshold Using high temperature for factual tasks - causes hallucination and inconsistency in code/data extraction

NLP-W12-T07 Choosing the Right Decoding Strategy

Tier B

Definition and why it matters: There is no single optimal decoding configuration for all applications. The right strategy depends on the task, acceptable risk of hallucination, and desired output diversity. The guidelines below provide starting points - always validate on your specific use case

Core explanation: Guideline: Sampling method; Recommendation: Top-P over Top-K; Rationale: Adaptive candidate set; balances control and flexibility Guideline: Top-P value; Recommendation: Moderate (e.g., 0.85-0.95); Rationale: Works well as a general default Guideline: Temperature for factual tasks; Recommendation: Low (0.0-0.3); Rationale: Minimises randomness in coding, extraction, Q&A Guideline: Temperature for creative tasks; Recommendation: High (0.7-1.0); Rationale: Enables diverse brainstorming and writing Concept flow: What is your task? - Low Temperature 0.0-0.3 - High Temperature 0.7-1.0 - Top-P ~ 0.9 - Test and Iterate Application: Code generation; Temperature: 0.0 - 0.2; Top-P: 0.9; Top-K: Optional Application: Data

extraction / JSON output; Temperature: 0.0 - 0.1; Top-P: 0.8 - 0.9 Application: Factual Q&A; Temperature: 0.1 - 0.3; Top-P: 0.9 Application: Marketing copy; Temperature: 0.7 - 0.9; Top-P: 0.95 Application: Poetry / creative writing; Temperature: 0.8 - 1.0; Top-P: 0.95 Application: Quiz generation (structured); Temperature: 0.3 - 0.5; Top-P: 0.9 Top-K always considers exactly K tokens, even when the probability distribution is sharply peaked (wasting slots on unlikely tokens) or flat (truncating valid options). Top-P adapts: Peaked distribution: few tokens exceed threshold → conservative output Flat distribution: many tokens qualify → diverse output This adaptability makes Top-P the default in Google AI Studio, OpenAI Playground, and most production APIs Start with recommended defaults (Top-P $\approx$ 0.9, temperature matched to task) Generate multiple outputs for the same prompt Evaluate fluency, accuracy, and format compliance Adjust one parameter at a time Document settings per use case in your application config

Must remember:

- No universally best decoding strategy - tune per application
- Top-P is generally preferred over Top-K for flexibility
- Low temperature for informational/factual tasks; high for creative tasks
- Moderate Top-P ($\approx$ 0.9) works well as a general starting point
- Always experiment and iterate - defaults are starting points, not final answers
- Document hyperparameters per use case in production configs

Exam trap: Claiming one strategy works universally - settings are application-specific Using high temperature for JSON/structured output - causes format violations and hallucinated fields

NLP-W12-T08 Prompts and Prompt Engineering

Tier A

Definition and why it matters: A prompt is the input provided to an LLM that initiates text generation. It is the sole interface for controlling model behaviour in most applications - there is no traditional API to "program" an LLM's logic directly Concept flow: Prompt - Input - Large Language Model - Generated Response - Output

Core explanation: Industrial LLM applications - customer support bots, code assistants, document summarisers, quiz generators - all depend on prompt quality. A well-crafted prompt can make a smaller, cheaper model outperform a larger one with a vague prompt Output depends entirely on input Specificity reduces ambiguity Iteration is expected - prompt design is experimental Constraints (format, length, tone) must be explicit

Must remember:

- A prompt is the LLM input that triggers text generation
- Prompts can include instructions, context, examples, and constraints
- Prompt engineering designs inputs for desired outputs
- LLM behaviour is described through prompts, not coded explicitly
- Small prompt changes can significantly affect results
- No universal best prompt - effectiveness is task-dependent

Example or contrast: Aspect: Control mechanism; Traditional Code: Explicit logic (if/else, functions); LLM + Prompt: Natural-language description Aspect: Behaviour change; Traditional Code: Modify code; LLM + Prompt: Modify prompt Aspect: Determinism; Traditional Code: High (same input - same output); LLM + Prompt: Variable (sampling involved)

Exam trap: Assuming bigger models eliminate need for good prompts - prompt quality always matters Treating prompts as one-time setup - prompt engineering is iterative and task-specific

NLP-W12-T09 Core Components of a Prompt

Tier A

Definition and why it matters: Unstructured prompts produce inconsistent outputs. A well-structured prompt reduces ambiguity and gives the model clear signals about role, task, and expected response format

Core explanation: Component: Role; Purpose: Who the model acts as; who the audience is; Required?: Recommended Component: Input; Purpose: Context, data, or instructions for the task; Required?: Required

Component: Output expectation; Purpose: Format, tone, depth, length; Required?: Recommended Component: Examples; Purpose: Demonstrations of desired behaviour; Required?: Optional Concept flow: Role - Complete Prompt - Input / Context + Instructions - Output Expectation - Examples - optional - LLM Response Element: Model role; Value: Math teacher Element: Audience; Value: 3rd graders Defines the persona the model should adopt and optionally the target audience Example: You are a math teacher explaining concepts to 3rd graders Constraint Type: Format; Examples: "Respond in 3 bullet points", "Return raw JSON" Constraint Type: Length; Examples: "Limit to 100 words" Constraint Type: Tone; Examples: Friendly, constructive, formal, sarcastic The role shapes vocabulary, analogies, and explanation depth Constraint Type: Style; Examples: Simple language, poetic, literary, technical Provides the information the model uses to generate a response. Can be: Constraint Type: Depth; Examples: Beginner overview vs expert analysis Context only - a paragraph to summarise Instruction only - "Explain overfitting in machine learning" Both - context plus specific instructions about what to do with it Specifies how the response should look: Clear structure reduces ambiguity and improves consistency

Must remember:

- Four prompt components: Role, Input, Output expectation, Examples (optional)
- Role defines model persona and target audience
- Input provides context, instructions, or both
- Output expectation specifies format, tone, depth, and length
- Clear structure reduces ambiguity and improves output quality
- Examples (one-shot/few-shot) further anchor format when needed

Example or contrast: Code pattern: You are a physics teacher conducting a class for 3rd grade; Explain the top 3 concepts of quantum physics; Use analogies to explain each concept; Tie back each concept to its respective analogy; Use simple language and a friendly tone Component: Role; Content: Physics teacher for 3rd graders Component: Input; Content: Explain top 3 quantum physics concepts with analogies

Exam trap: Omitting output format specification - leads to inconsistent structure across runs Confusing role with instruction - "You are a lawyer" is role; "Summarise this contract" is instruction

NLP-W12-T10

Zero-Shot, One-Shot, and Few-Shot Prompting

Tier A

Definition and why it matters: In prompting terminology, "shot" refers to the number of examples provided in the prompt. More shots = stronger format guidance, but higher token cost

Core explanation: Definition: No examples provided - only instructions Advantage: Simple and fast; Limitation: Output format may vary Advantage: Low token cost; Limitation: Ambiguous for structured tasks Advantage: Often effective for open-ended tasks; Limitation: Inconsistent for classification with specific labels Advantage: Model infers desired format from example; Limitation: Slightly higher token cost Advantage: More consistent output structure; Limitation: One example may not cover edge cases Advantage: More control than zero-shot; Limitation: Still not guaranteed correct Best for: explanations, brainstorming, general Q&A where format flexibility is acceptable Advantage: Strong guidance on style and format; Limitation: Longer prompts, higher token cost Definition: One example demonstrates the desired input-output pattern before the actual task Advantage: More reliable, less ambiguous; Limitation: Still no correctness guarantee Advantage: Common in production NLG systems; Limitation: Diminishing returns beyond ~5 examples Best for: classification, formatting tasks where output structure matters Definition: Multiple examples (typically 2-5) demonstrate the pattern Code pattern: Explain the concept of overfitting in machine learning Best for: production classification, structured extraction, consistent formatting Code pattern: Classify the sentiment of the following text as positive, negative, or neutral; Example::; Text: "The movie was absolutely fantastic."; Sentiment: positive; Now classify::; Text: "The new restaurant's food was delicious but the service was terribly slow."; Sentiment: Code pattern: Classify the sentiment as positive, negative, or neutral; Example 1::; Text: "The movie was absolutely fantastic."; Sentiment: positive; Example 2::; Text: "I waited for 3 hours and nobody ever called me back."; Sentiment: negative; Example 3:

Must remember:

- "Shot" = number of examples in the prompt

- Zero-shot: no examples; simple, fast, but format may vary
- One-shot: one example; better format consistency
- Few-shot: multiple examples; strongest guidance, highest token cost
- Few-shot is common in production but does not guarantee correctness
- Choose based on format strictness, cost budget, and task complexity

Example or contrast: Technique: Zero-shot; Examples: 0; Token Cost: Lowest; Consistency: Variable; Control: Lowest Technique: One-shot; Examples: 1; Token Cost: Low; Consistency: Better; Control: Moderate Technique: Few-shot; Examples: 2+; Token Cost: Higher; Consistency: Best; Control: Highest

Exam trap: Confusing "shot" with "turn" - shot = examples in prompt; turn = conversation exchange Claiming few-shot guarantees correctness - it improves format consistency, not accuracy

NLP-W12-T11

Practical Prompt Engineering Guidelines

Tier A

Definition and why it matters: No prompt works perfectly on the first attempt. Prompt engineering is an iterative experimental process - write, test, evaluate, refine, repeat. The guidelines below accelerate convergence but do not replace testing on your specific task and model

Core explanation: Describe exactly what you want the model to do. Vague tasks produce vague outputs Weak: "Analyse this text"; Strong: "Classify this product review as positive, negative, or neutral" Weak: "Write about AI"; Strong: "Write a 200-word introduction to supervised learning for beginners" Vague: "Be brief"; Specific: "Limit response to 100 words" Vague: "Make it professional"; Specific: "Use formal tone, no contractions, third person" Vague: "Summarise well"; Specific: "Summarise in 3 sentences covering cause, effect, and recommendation" State format requirements precisely: "Respond in 3 bullet points" "Return raw JSON with keys: title, summary, tags" "Use uppercase labels A, B, C, D only" The more specific the format, the closer the output matches requirements Specificity increases determinism and reduces unwanted variation When output must meet strict requirements, enumerate every constraint: Difficulty level (easy / intermediate / hard) Concept flow: Draft Prompt - Test on Sample Inputs - Output Acceptable? - Refine Prompt - Deploy / Document Number of items (exactly 3 questions) Option format (A, B, C, D uppercase only) Include explanations for correct answers JSON schema with required keys Create multiple prompt versions Run each against the same inputs Compare outputs for accuracy, format compliance, and tone Keep the best version; document what changed and why [] Role defined (who the model is, who the audience is) [] Task stated explicitly [] Output format specified (structure, length, tone) [] Constraints listed (difficulty, count, labels) [] Examples included if format is complex (one-shot / few-shot) [] Tested on at least 3 diverse inputs [] Edge cases handled (empty input, off-topic requests)

Must remember:

- Prompt engineering is iterative and experimental, not one-and-done
- Be explicit about the task with detailed instructions
- Specify output format precisely (structure, length, tone)
- Avoid vague language - specificity improves determinism
- Add constraints when format or content must be exact
- Test multiple prompt versions and refine based on results

Exam trap: Treating prompt design as a one-time step - it requires iteration Adding constraints verbally but not in the prompt - unstated constraints will not be followed

NLP-W12-T12

Google AI Studio Playground

Tier B

Definition and why it matters: Google AI Studio is Google's platform for testing Gemini models, generating API keys, and prototyping LLM applications. Beyond text generation, it supports image/video generation, vibe-coding, and app building - but for NLG workflows, the Playground is the primary interface Concept flow: Google AI Studio - Playground - Chat Interface - Build - App Creation - Monitoring - API Usage - API Keys - Documentation / SDK - Gemini Models

Core explanation: Enter prompts and receive generated responses (Ctrl+Enter to send) Thoughts / Chain-of-thought: expandable panel showing reasoning steps the model took (more steps for complex prompts) Setting:

Model; Purpose: Select Gemini variant; Recommended: Flash for speed/cost; Pro for quality Setting: System instructions; Purpose: Persistent behaviour (same as system prompt); Recommended: e.g., "Always respond in a poem" Setting: Temperature; Purpose: Creativity control (0-2); Recommended: Stay 0-1; ~0.65 for balanced Setting: Top-P; Purpose: Nucleus sampling threshold; Recommended: ~0.95 default Setting: Output length; Purpose: Max response tokens; Recommended: Set to control cost Model selection dropdown (e.g., Gemini Flash, Pro variants) Setting: Stop sequence; Purpose: Halt generation at specific string; Recommended: Optional Setting: Structured outputs; Purpose: Enforce JSON schema; Recommended: Enable for API integrations Setting: Thinking level; Purpose: Reasoning depth for complex puzzles; Recommended: High for multi-step problems Section: Build; Purpose: Vibe-code full applications Section: Dashboard; Purpose: Monitor API usage and costs Section: Documentation; Purpose: SDK examples in Python, JavaScript Section: Get API Key; Purpose: Generate keys for programmatic access

Must remember:

- Google AI Studio: test Gemini models, get API keys, monitor usage
- Playground provides chat UI with model, temperature, top-P, and safety settings
- System instructions = persistent system prompt across the session
- Multi-turn chat retains context; single-turn API calls do not
- Knowledge cutoff causes outdated factual answers
- Structured outputs, grounding, and function calling support production workflows
- Python SDK: google.genai with Client and generatecontent

Example or contrast: Mode: Multi-turn (default in Playground); Behaviour: Prior messages retained; "What is his age?" resolves to the previously mentioned person Mode: Single-turn (API design); Behaviour: Each request is independent; no pronoun resolution Knowledge cutoff: Models answer based on training data up to a cutoff date. A model trained before a leadership change may give outdated facts (e.g., answering "Joe Biden" when the current president has changed)

Exam trap: Trusting Playground answers as current facts - knowledge cutoff causes outdated responses Setting temperature 1 in production - produces incoherent output

NLP-W12-T13 Gemini API Key Setup

Tier B

Definition and why it matters: Programmatic access to Gemini models requires authentication via an API key. Without it, SDK calls fail with:

Core explanation: Code pattern: ValueError: No API key was provided. Please pass a valid API key Action: View usage; Purpose: Monitor API requests and costs per key Action: Copy key; Purpose: Use in applications Action: Revoke/regenerate; Purpose: Rotate compromised keys The key identifies your project and enables usage tracking and billing Open Google AI Studio Click Get API key (bottom-left) Select or create a GCP project Click Create API key → select Gemini API Copy the key immediately (store securely) Concept flow: Generate API Key - Store Securely - Load into Runtime - genai.Client(apikey=... - generatecontent - response.text Never hardcode API keys in source code committed to version control Store the key in Colab's Secrets panel (key icon) with name GEMINIAPIKEY Critical: passing the key to genai.Client(apikey=...) is required - importing the library alone is insufficient Add .env to .gitignore - never commit secrets Use separate keys per environment (dev/staging/prod) Monitor usage to detect unexpected spend Rotate keys if exposed Use Colab secrets or cloud secret managers, not plaintext in notebooks shared publicly Code pattern: env file (add to .gitignore); GEMINIAPIKEY=yourkeyhere Code pattern: import os; from dotenv import load_dotenv; load_dotenv(); apikey = os.getenv("GEMINIAPIKEY") Code pattern: from google.colab import userdata; geminiapikey = userdata.get("GEMINIAPIKEY") Code pattern: from google import genai; client = genai.Client(apikey=geminiapikey); response = client.models.generate_content(; model="gemini-2.5-flash";, contents="Explain how AI works in a few words";); print(response.text); "AI learns patterns from data to make predictions."

Must remember:

- Gemini SDK requires an API key for all programmatic calls
- Generate keys in Google AI Studio → Get API key → Create

- Store keys in .env (production) or Colab Secrets (notebooks)
- Initialise: `genai.Client(apikey=yourkey)`
- Never commit API keys to version control
- Monitor usage and costs via the AI Studio dashboard

Exam trap: Hardcoding API keys in notebooks shared on GitHub - immediate security breach Forgetting to pass `apikey` to `Client()` - most common integration error

NLP-W12-T14 OpenAI Platform and API Setup

Tier A

Definition and why it matters: The OpenAI Platform (platform.openai.com) provides chat interfaces, agent builders, media generation tools, API management, and fine-tuning - all accessible from a single dashboard
 Concept flow: OpenAI Platform - Chat - Prompting - Agents - Agentic AI - Audio / Images / Video - Usage and Billing - API Keys - Fine-tuning UI - GPT Models

Core explanation: Section: Chat; Purpose: Interactive prompt testing with model selection Section: Agents; Purpose: Build agentic AI workflows via UI Section: Audio / Images / Video; Purpose: Multimodal generation tools Section: Usage; Purpose: Track API consumption and costs Section: API Keys; Purpose: Create and manage secret keys Section: Fine-tuning; Purpose: Fine-tune base models without scripting Section: Logs / Storage; Purpose: Data management for applications Parameter: Model; Purpose: GPT variant selection; Recommended: Match task to model tier Parameter: System message; Purpose: Persistent instructions (tone, format); Recommended: Set role and constraints Parameter: User prompt; Purpose: Per-request task input; Recommended: Your actual query Parameter: Temperature; Purpose: Creativity (0-2); Recommended: 0-1 for most tasks Parameter: Max tokens; Purpose: Response length cap; Recommended: Limits cost per request Parameter: Top-P; Purpose: Nucleus sampling; Recommended: ~0.9 for balanced output Parameter: Tools; Purpose: Function calling attachments; Recommended: Optional per use case Parameter: Variables; Purpose: Reusable template variables; Recommended: e.g., `city = Mumbai` Billing tip: Add a small credit balance (5-10) and monitor usage - overspending is easy with pay-per-token pricing Navigate to API Keys in the left sidebar Click Create new secret key Select project, name the key (e.g., "demo") Copy immediately - the key is shown only once Store securely (.env or Colab Secrets) Error without key: `Pass to OpenAI(apikey=openaiapikey)` Full API reference: developers.openai.com/api/docs Includes model pricing, endpoint specifications, and copy-paste code for Python and JavaScript Code pattern:

```
from openai import OpenAI; import os; client = OpenAI(apikey=os.getenv("OPENAI_API_KEY")); response = client.responses.create(model="gpt-4.1", input="Write a one-sentence bedtime story about a unicorn"); print(response.output_text)
```

 Code pattern: `OpenAIError: The apikey client option must be set either by passing; apikey or by setting the OPENAI_API_KEY environment variable` Code pattern:

```
from google.colab import userdata; openaiapikey = userdata.get("OPENAI_API_KEY")
```

Must remember:

- OpenAI Platform: chat testing, agents, API keys, fine-tuning, usage monitoring
- Playground settings: model, system message, temperature, max tokens, top-P
- API keys shown once at creation - copy and store immediately
- Python: `OpenAI(apikey=...)` then `client.responses.create(...)`
- Set `OPENAI_API_KEY` environment variable or use Colab Secrets
- Monitor credit balance to avoid unexpected charges
- Documentation at developers.openai.com/api/docs

Formula / decision rule: 5-

Example or contrast: Aspect: Models; OpenAI Platform: GPT family; Google AI Studio: Gemini family
 Aspect: Key storage; OpenAI Platform: `OPENAI_API_KEY` env var; Google AI Studio: `GEMINI_API_KEY` env var
 Aspect: SDK; OpenAI Platform: `openai` package; Google AI Studio: `google.genai` package

Exam trap: Not saving the API key at creation - it cannot be retrieved later; only regenerated Committing keys to GitHub - use `.env + .gitignore`

Week 12 Exam Lens

Likely questions

- Define and explain Why Decoder-Only Models Dominate NLG; include the mechanism and one limitation.
- Compare Why Decoder-Only Models Dominate NLG with What Is a Large Language Model (LLM)? and state when each is preferred.
- Apply Why LLMs Excel at Natural Language Generation to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $P(t_n | t_1, t_2, \dots, t_{(n-1)})$; t_1 ; 10^9 ; $P(mat | context)$; $P(token | context)$; p_1 ; p_2 ; 5-

Mistakes to avoid: Claiming BERT is used for open-ended text generation - BERT is encoder-only; GPT-style decoder-only models dominate NLG Citing a fixed parameter count as the LLM threshold - there is no strict cutoff; "large" is relative across eras Attributing genuine reasoning to LLMs - they predict tokens probabilistically; chain-of-thought is pattern mimicry, not symbolic reasoning

Week 13: Capstone: QuizGenius AI

Week roadmap: QuizGenius AI: Capstone Project Overview → Package Installation and API Setup → Creating the Quiz Prompt Template → Building the Quiz Generation Logic → Building the Main Application Runner → Demo: Notebook Quiz Application → 7 more topics.

Coverage: 13 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

NLP-W13-T01 QuizGenius AI: Capstone Project Overview

Tier B

Definition and why it matters: QuizGenius AI is an LLM-powered application that dynamically generates multiple-choice quizzes on any user-specified topic. Built with the Gemini API, it demonstrates how generative AI integrates into interactive, educational applications - a template adaptable to any domain

Core explanation: The capstone progresses from a Jupyter notebook proof-of-concept to a full-stack Streamlit application with proper API key management, structured JSON output, and a browser-based UI Feature: Dynamic quiz generation; Description: 3-question MCQ on any user-specified topic Feature: Intermediate difficulty; Description: Challenging yet accessible questions Feature: Structured JSON output; Description: Consistent, parseable quiz data from the LLM Feature: Interactive UI; Description: Real-time answer submission and immediate feedback Feature: Secure API key management; Description: Keys stored via Colab Secrets or .env, never hardcoded Concept flow: User Topic - QuizGenius AI - Gemini API - Structured JSON Quiz - Interactive Quiz UI - Score and Feedback Layer: LLM; Technology: Google Gemini (via google.genai SDK) Layer: Notebook POC; Technology: Jupyter / Google Colab Layer: Full-stack UI; Technology: Streamlit (app.py) Layer: Backend logic; Technology: Python (quizengine.py) Layer: Secrets; Technology: env file / Colab userdata Layer: Version control; Technology: GitHub Industrial LLM applications rarely consume free-form text. They need machine-parseable output to drive downstream logic - rendering UI components, storing in databases, or triggering workflows. Enforcing JSON schema via prompt design and API configuration is a high-demand production skill Integrate Gemini API into a Python application Design prompts that produce consistent structured output Parse and validate LLM JSON responses Build interactive NLG applications beyond notebooks Manage API keys securely in development and deployment Adapt the template to other domains: HR: Policy comprehension quizzes for onboarding Cloud: AWS service knowledge checks for certification prep

Must remember:

- QuizGenius AI: Gemini-powered MCQ quiz generator on any topic
- Features: dynamic generation, JSON output, interactive UI, secure API keys
- Structured JSON from LLMs is critical for industrial applications
- Template is domain-agnostic - adapt to education, cloud, HR, etc
- Capstone builds from notebook POC to full-stack Streamlit app
- Intermediate difficulty, 3 MCQs with A/B/C/D options and explanations

Exam trap: Hardcoding API keys in source code - security violation; use secrets management Accepting free-form LLM text without JSON parsing - breaks automated UI rendering

NLP-W13-T02 Package Installation and API Setup

Tier B

Definition and why it matters: Before building QuizGenius AI, install the Gemini SDK, configure imports, and securely load the API key. This setup pattern applies to any Gemini-based Python project

Core explanation: Code pattern: `pip install google-genai` Code pattern: `import json; import re; from google import genai; from google.genai import types` Package: `json`; Purpose: Parse structured LLM responses Package: `re`; Purpose: Regex cleanup of JSON strings Package: `google.genai`; Purpose: Gemini SDK client and types Package: `google.genai.types`; Purpose: Configuration objects (e.g., response MIME type) Concept flow: `pip install google-genai` - Load API Key - `genai.Client` - Select Model - Ready for generate-content Store the key in Colab Secrets panel with name GEMINI_APIKEY Gemini 2.5 Flash is a lighter, faster, more cost-effective variant - ideal for demos and proof-of-concepts. Model names can also be stored in .env as MODELNAME to avoid hardcoding Stop early if the key is missing - prevents silent failures and

cryptic API errors downstream Code pattern: `apikey = "pastekeyhere"` NOT recommended for production Code pattern: `from google.colab import userdata; geminiapikey = userdata.get("GEMINIAPIKEY")` Code pattern: `env; GEMINIAPIKEY=yourkeyhere; MODELNAME=gemini-2.5-flash` Code pattern: `import os; from dotenv import load_dotenv; load_dotenv(); apikey = os.getenv("GEMINIAPIKEY")` Code pattern: `client = genai.Client(apikey=geminiapikey); modelname = "gemini-2.5-flash"` fast, cost-effective for POCs Code pattern: `if not geminiapikey; raise ValueError("GEMINIAPIKEY is missing. Please check your .env file.")`

Must remember:

- Install `google-genai`; import `genai` and types
- Load API key via Colab Secrets or `.env` - never hardcode
- Initialise `genai.Client(apikey=...)` before any API calls
- Use `gemini-2.5-flash` for fast, cost-effective POCs
- Add fail-safe: raise error if API key is missing
- Also import `json` and `re` for response parsing

Exam trap: Pasting API keys directly in notebook cells - keys get committed to Git or shared notebooks Forgetting `genai.Client(apikey=...)` - library import alone does not authenticate

NLP-W13-T03 Creating the Quiz Prompt Template

Tier A

Definition and why it matters: A prompt template is a reusable string with placeholders (`{topic}`, `{numquestions}`, `{difficulty}`) filled at runtime. It encodes role, constraints, and output schema in one place - ensuring every quiz request follows the same structure

Core explanation: Code pattern: You are a quizmaster; Generate a `{numquestions}`-question multiple choice quiz about `{topic}` Component: Role; Value: Quizmaster Component: Task; Value: Generate N-question MCQ about user topic Rule: Difficulty; Specification: Intermediate (configurable: easy / intermediate / hard) Rule: Options; Specification: Exactly 4 options: A, B, C, D (uppercase only) Rule: Correct answer; Specification: Marked clearly per question Rule: Explanation; Specification: Short explanation for each correct answer Concept flow: Prompt Template - Role: Quizmaster - Constraints: difficulty, ABCD, explanations - JSON Schema Specification - `.format` topic, numquestions, difficulty - Final Prompt - Gemini API Principle: Explicit task; How Applied: "Generate N-question MCQ about `{topic}`" Principle: Format specification; How Applied: Full JSON schema with sample keys Specificity prevents ambiguous output - the model knows exactly how many options, what labels, and what extras to include Principle: Constraints; How Applied: Difficulty, option labels, explanations Principle: Role; How Applied: Quizmaster persona "Raw JSON" and "this structure only" are critical phrases - they reduce markdown wrapping and extraneous text around the JSON User input is injected at call time. `.strip()` removes leading/trailing whitespace for consistency Code pattern: Return a raw JSON with this structure only: Code pattern: `{ "quiztitle": "Catchy Title Here",; "questions": [; { "id": 1,; "text": "Question text here?"; "options": { "A": "Option A text",` Code pattern: `prompt = prompttemplate.format(; topic=topic.strip();; numquestions=numquestions;; difficulty=difficulty; )`

Must remember:

- Prompt template: reusable string with `{topic}`, `{numquestions}`, `{difficulty}` placeholders
- Role: quizmaster; constraints: difficulty, 4 uppercase options, explanations
- Output: raw JSON with `quiztitle` and `questions` array
- Each question: `id`, `text`, `options` (A-D), `correctoption`, `explanation`
- Format with `.format()` at runtime after user input
- Explicit JSON schema in prompt is essential for parseable industrial output

Exam trap: Not specifying "raw JSON" - model wraps output in markdown code fences, breaking `json.loads()` Vague option format - "4 options" without "A, B, C, D uppercase" produces numbered or lowercase labels

NLP-W13-T04 Building the Quiz Generation Logic

Tier B

Definition and why it matters: The core backend function accepts a topic, calls the Gemini API with the prompt template, and returns parsed JSON quiz data

Core explanation: Concept flow: generatequiz topic - Build Prompt from Template - client.models.generatecontent - response.text - Regex JSON Extraction - json.loads - Return Quiz Dictionary Step: response.text; Purpose: Extract raw string from API response Step: Regex extraction; Purpose: Isolate JSON object from surrounding text Step: Markdown cleanup; Purpose: Remove json fences if present Step: json.loads(); Purpose: Parse into Python dictionary responsemime="application/json" instructs the API to return JSON-formatted output - a critical setting for industrial structured generation Always wrap LLM calls in try/except - API failures, malformed JSON, and network errors are common in production The production quizengine.py version adds: Input validation before API call (topic length, question count range, valid difficulty) Dynamic prompt formatting with {topic}, {numquestions}, {difficulty} Schema validation - verify "questions" key exists in parsed JSON Fail-fast on missing API key Code pattern: def generatequiz(topic: str) - dict::; print(f"Generating a quiz on topic: {topic}"); response = client.models.generatecontent(model=modelname, contents=prompttemplate.format(topic=topic), config=types.GenerateContentConfig(responsemime="application/json")); Code pattern: rawtext = response.text; Extract JSON block via regex; pattern = r'\{.\}'; match = re.search(pattern, rawtext, re.DOTALL); cleanjsonstring = match.group(0) if match else rawtext; Remove markdown artifacts; cleanjsonstring = re.sub(r'json|', '', cleanjsonstring).strip(); quizdata = json.loads(cleanjsonstring) Code pattern: try::; generation logic; except json.JSONDecodeError as e::; raise ValueError(f"Failed to parse quiz JSON: {e}"); except Exception as e::; raise RuntimeError(f"Quiz generation failed: {e}")

Must remember:

- generatequiz(topic) calls Gemini with formatted prompt template
- Set responsemime="application/json" for structured output
- Extract text → regex JSON extraction → markdown cleanup → json.loads()
- Wrap in try/except for JSONDecodeError and general exceptions
- Production version adds input validation and schema checks
- Never call the LLM before validating user inputs

Exam trap: Skipping responsemime="application/json" - increases free-text wrapping and parsing failures Not using regex to extract JSON - LLMs may add preamble text before the JSON object

NLP-W13-T05 Building the Main Application Runner

Tier B

Definition and why it matters: The application runner orchestrates the full quiz lifecycle: collect user input, generate quiz, run the interactive game loop, and display final results. This is the notebook POC entry point before the Streamlit full-stack version

Core explanation: Concept flow: runquizapp - Get Topic from User - generatequiz topic - Display Quiz Title - Loop: Ask Each Question - Answer Correct? - score += 1 - Show Correct Answer + Explanation - Next Question Access Pattern: quizdata["quiztitle"]; Meaning: Quiz name Access Pattern: quizdata["questions"]; Meaning: List of question dicts Access Pattern: q["text"]; Meaning: Question text Access Pattern: q["options"]; Meaning: Dict of A/B/C/D options total is computed dynamically - changing question count in the prompt does not require code changes Access Pattern: q["correctoption"]; Meaning: Correct label Access Pattern: q["explanation"]; Meaning: Why the answer is correct Understanding the data shape is essential for writing the loop: Code pattern: topic = input("What topic do you want to be quizzed on? "); topic = "mathematics" hard-coded alternative for testing Code pattern: quizdata = generatequiz(topic); if not quizdata::; print("Failed to generate quiz."); return Code pattern: print(quizdata["quiztitle"]); score = 0; total = len(quizdata["questions"]) Code pattern: for q in quizdata["questions"]::; print(q["text"]); for key, value in q["options"].items():; print(f"{key}: {value}"); useranswer = input("Your answer (A/B/C/D): ").strip().upper(); if useranswer == q["correctoption"]::; print("Correct!"); score += 1 Code pattern: print(f"Final score: {score}/{total}"); if score == total::; print("Perfect score! You are a master!"); elif score 0::; print("Good job! Keep studying."); else::; print("Better luck next time.")

Must remember:

- runquizapp(): input topic → generate → loop questions → score → feedback
- total = len(quizdata["questions"]) keeps question count dynamic

- Normalise user answers with `.strip().upper()`
- Correct: increment score; wrong: show correct option and explanation
- Final messages based on score/total ratio
- `time.sleep(1)` adds pause between questions for readability

Exam trap: Hardcoding `total = 3` - use `len(quizdata["questions"])` for dynamic question counts Case-sensitive answer comparison - normalise with `.strip().upper()`

NLP-W13-T06 Demo: Notebook Quiz Application

Tier B

Definition and why it matters: The notebook proof-of-concept runs with a single function call:

Core explanation: Code pattern: `runquizapp()` Capability: Dynamic topic handling; Evidence: Different subjects produce different quizzes Capability: LLM-generated titles; Evidence: "Math Master's Challenge", "Intermediate Science Challenge" Capability: JSON parsing; Evidence: Questions, options, and answers render correctly Capability: Scoring logic; Evidence: Correct/incorrect tracking with final ratio Capability: Explanations; Evidence: Shown after each answer Capability: Real-time interaction; Evidence: Input - generation - quiz - feedback in one session The terminal prompts for a topic, generates a quiz via Gemini, and runs the interactive question loop Experiment with different topics (cloud computing, NLP, history) Modify prompt template (difficulty, question count) Adapt the template to your own project idea Proceed to full-stack Streamlit implementation

Must remember:

- Run `runquizapp()` to start the interactive CLI quiz
- User enters topic; Gemini generates titled MCQ quiz in real time
- Each question: display options, accept A/B/C/D, show correct/incorrect + explanation
- Final score: X/Y with motivational message based on performance
- Demo validates end-to-end: prompt → API → JSON → game loop → scoring
- LLM content may be factually incorrect - treat as educational demo, not authoritative source

Example or contrast: Code pattern: What topic do you want to be quizzed on? mathematics; Generating a quiz on topic: mathematics; Math Master's Challenge; Q1: What is the value of the integral of ...?; Your answer: D; Wrong. The answer was A. [explanation]; Q2: If matrix A has eigenvalues 1, 2, 3, ...?; Your answer: B Code pattern: What topic do you want to be quizzed on? science; The Intermediate Science Challenge; [3 questions with answers and feedback]; Final score: 1/3; Good job! Keep studying

Exam trap: Factual accuracy not guaranteed - LLM-generated questions may contain errors Single-turn only - no conversation memory between runs

NLP-W13-T07 From Notebook POC to Full-Stack Application

Tier B

Definition and why it matters: The Jupyter notebook POC works for Python practitioners but excludes non-technical users. Industrial LLM products - ChatGPT, Gemini, Claude - became viral only when wrapped in accessible frontends, not when exposed as scripting APIs

Core explanation: Concept flow: Jupyter Notebook - CLI Interaction - Streamlit Frontend - Python Backend - Gemini API - POC - Production Notebook POC: Python users only; Full-Stack App: Any user via browser Notebook POC: Single file; Full-Stack App: Separated frontend + backend Notebook POC: Hard-coded secrets risk; Full-Stack App: env secrets management Notebook POC: No input validation; Full-Stack App: Validated inputs before API calls Notebook POC: CLI text interface; Full-Stack App: Interactive forms, buttons, visual feedback Component: Frontend; File: `app.py`; Technology: Streamlit Component: Backend; File: `quizengine.py`; Technology: Python + Gemini SDK Component: Secrets; File: `env`; Technology: GEMINI_APIKEY, MODELNAME Component: Dependencies; File: `requirements.txt`; Technology: Pinned versions Component: Docs; File: `README.md`; Technology: Project documentation Component: Ignore rules; File: `gitignore`; Technology: Excludes `.env`, caches Portfolio impact: A full-stack LLM application demonstrates end-to-end capability - prompting, API integration, validation, UI, and deployment - far beyond notebook-only projects Streamlit lets Python developers build browser UIs without HTML, CSS, or JavaScript knowledge. For NLP practitioners and AI engineers, this bridges the gap to user-facing applications. Frontend engineer-

ing teams can later replace Streamlit with React/Vue for polished production UIs AI engineer builds logic + Streamlit prototype Software engineering team refines frontend for production DevOps handles deployment, secrets, monitoring You do not need to learn frontend frameworks to demonstrate a complete LLM product GPT and transformer models existed years before ChatGPT's consumer launch. The application layer - accessible UI, session management, feedback loops - turned research into a product revolution. Building QuizGenius AI as full-stack follows the same pattern

Must remember:

- Notebook POC limits access to Python users; full-stack opens access to everyone
- ChatGPT succeeded because of accessible UI, not just model capability
- Full-stack: Streamlit frontend (app.py) + Python backend (quizengine.py)
- Streamlit enables Python-only frontend development without HTML/CSS
- Industry pattern: AI engineer prototypes → frontend team polishes
- Full-stack LLM apps are high-value portfolio pieces

Exam trap: Stopping at notebook POC for portfolio - full-stack demo is significantly more impressive Assuming users will run Python scripts - non-technical users need browser UIs

NLP-W13-T08 Application Flow and Architecture (v1)

Tier A

Definition and why it matters: QuizGenius AI follows a two-phase interaction loop:

Core explanation: Concept flow: U - FE - processed request - BE - LLM - A/B/C/D User enters a topic (e.g., "mathematics") Frontend captures and forwards to backend Backend calls Gemini LLM with the prompt template LLM returns JSON quiz data Concept flow: User - Frontend - Browser - Backend - Python Logic - Gemini LLM - Cloud Hosted Backend parses and sends to frontend Frontend displays quiz title and questions Component: Frontend; Responsibility: User interaction, form input, quiz display, answer collection, results rendering User selects answers for each question Component: Backend; Responsibility: Input validation, prompt construction, API call, JSON parsing, score calculation Frontend sends answers to backend Component: Gemini API; Responsibility: Quiz content generation (questions, options, explanations) Backend compares against correctoption fields Step: 1; Direction: User - Frontend; Payload: Topic string Backend calculates score and prepares feedback Step: 2; Direction: Frontend - Backend; Payload: Validated topic Frontend displays score, per-question review, and explanations Step: 3; Direction: Backend - LLM; Payload: Formatted prompt The loop repeats when the user requests a new quiz Step: 4; Direction: LLM - Backend; Payload: JSON quiz Step: 5; Direction: Backend - Frontend; Payload: Parsed quiz object Important: The LLM is not inside the backend. It is hosted on Google's cloud and accessed via API. The backend is a client, not the model host Step: 6; Direction: Frontend - User; Payload: Quiz UI Step: 7; Direction: User - Frontend; Payload: Selected answers Step: 8; Direction: Frontend - Backend; Payload: Answer dict Real projects start with a base architecture (v1) and refine after implementation: Add input validation layers Introduce .env for secrets Split into app.py and quizengine.py Add session state management in Streamlit Update architecture diagram to v2 after build

Must remember:

- Two-phase flow: (1) topic → quiz generation, (2) answers → scoring
- Frontend handles UI; backend handles logic and API calls
- Gemini LLM is external cloud service accessed via API, not embedded in backend
- Loop repeats for each new quiz session
- Architecture v1 is a starting point - update after implementation (v2)
- Backend parses JSON; frontend displays structured quiz data

Exam trap: Drawing LLM inside the backend box - it is an external cloud API Skipping the two-phase flow - generation and scoring are separate interactions

Application Flow and Architecture (v1)



Concept map: Application Flow and Architecture (v1)

NLP-W13-T09

Project Setup and Repository Structure

Tier A

Definition and why it matters: Create a GitHub repository for version control, collaboration, and portfolio visibility

Core explanation: Setting: Name; Value: quiz-genius-ai Setting: Visibility; Value: Public (or private) Setting: README; Value: Yes - project documentation Setting: gitignore; Value: Python template Setting: Licence; Value: MIT (common for open source) Concept flow: quiz-genius-ai/ - .env - secrets - .gitignore - requirements.txt - quizengine.py - backend - app.py - Streamlit frontend - README.md File: env; Purpose: GEMINI_APIKEY, MODELNAME - loaded by python-dotenv File: gitignore; Purpose: Prevents .env, py-cache/, .venv/ from being committed File: requirements.txt; Purpose: streamlit, google-genai, python-dotenv with pinned versions File: quizengine.py; Purpose: Validation, prompt template, generatequiz(), constants File: app.py; Purpose: Streamlit pages: setup, quiz, results File: README.md; Purpose: Setup instructions, architecture diagram, usage guide The .gitignore Python template already excludes .env. Never upload API keys to GitHub - revoked keys and billing surprises follow quickly Pin versions in requirements.txt after installation: Anyone cloning the repo can reproduce the exact environment Code pattern: git clone; cd quiz-genius-ai Code pattern: quiz-genius-ai/; env API keys and secrets (NEVER commit); gitignore Excludes .env, caches, IDE files; LICENSE MIT licence; README.md Documentation (updated as project evolves); requirements.txt Pinned Python dependencies; quizengine.py Backend: quiz generation + validation; app.py Frontend: Streamlit UI Code pattern: env (add to .gitignore); GEMINI_APIKEY=yourkeyhere; MODELNAME=gemini-2.5-flash Code pattern: conda create -n quiz-genius python=3.11 -y; conda activate quiz-genius; pip install -r requirements.txt Code pattern: streamlit==; google-genai==; python-dotenv==

Must remember:

- GitHub repo with README, .gitignore (Python), MIT licence
- Key files: .env, quizengine.py (backend), app.py (Streamlit frontend)
- env stores GEMINI_APIKEY and MODELNAME - never committed
- requirements.txt with pinned versions for reproducibility
- Virtual environment: conda create -n quiz-genius python=3.11
- Run: streamlit run app.py after activating environment

Example or contrast: Code pattern: conda activate quiz-genius; cd quiz-genius-ai; streamlit run app.py

Exam trap: Committing .env to GitHub - even private repos leak; use .gitignore No requirements.txt - others cannot reproduce your environment

NLP-W13-T10

Building the Backend (quizengine.py)

Tier A

Definition and why it matters: The backend module handles everything between user input and LLM response:

Core explanation: Load environment variables and initialise Gemini client Concept flow: User Inputs - validateinputs - ValueError - Format Prompt Template - generatecontent - JSON mode - Extract and Clean JSON - Validate Schema - Return quizdata dict Check: Empty topic; Error Message: "Topic cannot be empty" Validate user inputs before API calls Check: Topic 200 chars; Error Message: "Topic is too long. Please keep it under 200 characters" Format the prompt template Check: Questions not in [1, 10]; Error Message: "Number of questions

must be between 1 and 10" Call Gemini API with JSON response mode Check: Invalid difficulty; Error Message: "Difficulty must be one of: easy, intermediate, hard" Parse, clean, and validate the JSON response Check: All valid; Error Message: Returns None Benefit: Cost savings; Explanation: Invalid inputs never reach the paid API Validation runs before the LLM call to save API cost and ensure pipeline consistency Benefit: Consistency; Explanation: Gibberish topics and bizarre question counts are rejected Benefit: Security; Explanation: Length limits prevent prompt injection via oversized inputs Benefit: Reliability; Explanation: Schema check catches malformed LLM responses early Code pattern: `import json, re, os; from dotenv import load_dotenv; from google import genai; from google.genai import types; load_dotenv(); gemini_apikey = os.getenv("GEMINI_APIKEY"); modelname = os.getenv("MODELNAME", "gemini-2.5-flash"); if not gemini_apikey: Code pattern: VALID-DIFFICULTIES = ["easy", "intermediate", "hard"]; MAXTOPICLENGTH = 200; MAXQUESTIONS = 10; MINQUESTIONS = 1 Code pattern: def generate_quiz(topic: str, numquestions: int = 3, difficulty: str = "intermediate") - dict;; error = validate_inputs(topic, numquestions, difficulty); if error:; raise ValueError(error); prompt = prompt_template.format(; topic=topic.strip(),; numquestions=numquestions,`

Must remember:

- Backend: env setup → validation → prompt format → API call → JSON parse → schema check
- `validate_inputs()` checks topic, question count, and difficulty before API call
- `generate_quiz()` returns dict with quiz title and questions or raises errors
- `response_mimetype="application/json"` enforces structured output
- Validate "questions" key exists in parsed response
- Handle `JSONDecodeError` and general exceptions separately

Exam trap: Skipping validation and calling LLM directly - wastes tokens and money No schema validation after JSON parse - missing questions key crashes frontend

NLP-W13-T11 Building the Frontend (app.py)

Tier A

Definition and why it matters: Streamlit (st) enables Python developers to build browser UIs without HTML/CSS. QuizGenius AI uses three screens managed by session state - because Streamlit reruns the entire script on every user interaction

Core explanation: Code pattern: `st.set_page_config(; page_title="QuizGenius AI";, page_icon="";, layout="centered"; )` Setting: `page_title`; Effect: Browser tab title Setting: `page_icon`; Effect: Favicon Setting: `layout="centered"`; Effect: Content centred with side padding (vs wide) Concept flow: setup - quiz - results UI Element: Header; Streamlit Widget: `st.header`; Purpose: "Configure Your Quiz" UI Element: Topic; Streamlit Widget: `st.text_input`; Purpose: Free-text topic entry UI Element: Question count; Streamlit Widget: `st.slider`; Purpose: MINQUESTIONS to MAXQUESTIONS (default 3) UI Element: Difficulty; Streamlit Widget: `st.selectbox`; Purpose: easy / intermediate / hard UI Element: Submit; Streamlit Widget: `st.form_submit_button`; Purpose: "Generate Quiz!" Streamlit reruns the full script on every button click or input change. Session state persists data across reruns Validate topic is not empty Show spinner: "Generating your quiz..." Call `generate_quiz(topic, numquestions, difficulty)` from backend Percentage: 100%; Message: Trophy emoji - "Perfect score! You're a master!" Store result in `st.sessionstate.quizdata` Percentage: 70%; Message: "Great job! You know your stuff!" Set `currentstep = "quiz"` and `st.rerun()` Percentage: 40%; Message: "Good effort! Keep studying!" Display `quizdata["quiztitle"]` as header Percentage: ≤ 40%; Message: "Keep learning!" Loop through questions with `st.radio` for A/B/C/D selection Store selections in `st.sessionstate.useranswers` Code pattern: `def init_session_state():; defaults = {; "quizdata": None, Generated quiz JSON; "useranswers": {}, {question_index: "A"/"B"/"C"/"D"}; "submitted": False, Whether answers submitted; "currentstep": "setup" | "quiz" | "results"; }; for key, value in defaults.items(): Submit Answers button (width ratio 3) - validates all questions answered Back button (width ratio 1) - calls reset_quiz() and returns to setup Correct answer: green checkmark (st.success) Wrong answer: red cross (st.error) + correct option shown Explanation from q["explanation"] via st.info Take Another Quiz button resets state and returns to setup Code pattern: score = sum(; 1 for i, q in enumerate(questions); if useranswers.get(i) == q["correctoption"];); percentage = (score / total) 100 Code pattern: def main():; init_session_state(); st.title("QuizGenius AI"); st.caption("Generate AI-powered quizzes on any topic in seconds."); if st.sessionstate.currentstep == "setup"; render_setup_form(); elif st.sessionstate.currentstep == "quiz"; render_quiz()`

Must remember:

- Streamlit frontend: 3 screens (setup, quiz, results) routed by currentstep
- Session state persists quizdata, useranswers, submitted, currentstep
- Setup: topic input, question slider, difficulty selectbox, generate button
- Quiz: radio buttons per question, submit/back buttons
- Results: score percentage, tiered feedback, per-question review with explanations
- main() routes to correct screen based on session state
- Always call st.rerun() after state transitions

Exam trap: Not using session state - quiz data lost on every rerun Forgetting st.rerun() after state change
- UI does not transition to next screen

NLP-W13-T12 Running the Full-Stack Application

Tier B

Definition and why it matters: Streamlit starts a local web server (typically `http://localhost:8501`) and opens the browser automatically

Core explanation: Code pattern: `conda activate quiz-genius; cd quiz-genius-ai; streamlit run app.py` Element: Topic input; Action: Enter "science" (or any subject) Element: Question slider; Action: Select 3 (or 1-10) Element: Difficulty dropdown; Action: Select easy / intermediate / hard Element: Generate Quiz button; Action: Triggers API call with spinner Error case: empty topic → "Please enter a topic to generate a quiz." Header: LLM-generated title (e.g., "The Intermediate Science Challenge") Caption: "3 questions - select your answers and submit" Per question: radio buttons for A, B, C, D UI Element: "QuizGenius AI" title; Source in `app.py`: `main()` - `st.title()` Submit Answers (primary) | Back (secondary) UI Element: "Configure Your Quiz"; Source in `app.py`: `rendersetupform()` - `st.header()` Score header: "1/3" with emoji UI Element: Topic text box; Source in `app.py`: `st.textinput("Topic", placeholder=...)` Percentage and tiered message UI Element: Question slider; Source in `app.py`: `st.slider()` with MIN/MAX from backend Detailed review per question: UI Element: Difficulty dropdown; Source in `app.py`: `st.selectbox()` with `VALIDDIFFICULTIES` Correct: green checkmark UI Element: "Generate Quiz!" button; Source in `app.py`: `st.formsubmitbutton()` with `usecontainerwidth=True` Wrong: red cross + correct answer highlighted UI Element: Quiz title (dynamic); Source in `app.py`: `quizdata["quiztitle"]` in `renderquiz()` Explanation for each question UI Element: Question text (dynamic); Source in `app.py`: `f"Q{q['id']}: {q['text']}"` "Take Another Quiz" → resets to Screen 1 Issue: "GEMINI_APIKEY is missing"; Fix: Add key to `.env` file Issue: Import error for `quizengine`; Fix: Run from project root directory Issue: Streamlit not found; Fix: `pip install -r requirements.txt` in activated venv Run the app with different topics to verify:

Must remember:

- Launch: `streamlit run app.py` from project root with venv activated
- Three screens: setup (form) → quiz (radio buttons) → results (score + review)
- Empty topic shows error; valid topic triggers spinner then quiz display
- Results show score, percentage, tiered message, and per-question explanations
- "Take Another Quiz" resets session state to setup
- Map UI elements to `rendersetupform`, `renderquiz`, `renderresults`, and `main()`

Exam trap: Running `python app.py` instead of `streamlit run app.py` - Streamlit requires its own runner Not activating virtual environment - wrong package versions or missing dependencies

NLP-W13-T13 Updated Project Architecture (v2)

Tier A

Definition and why it matters: After implementing the full-stack application, the architecture diagram is updated to reflect actual file names, screen flows, backend steps, and secrets management. Real projects routinely revise architecture post-implementation based on optimisation needs and changing requirements

Core explanation: Concept flow: Setup Screen - Quiz Screen - Results Screen - Validate Inputs - Build Prompt - Generate Quiz via API - `.env` File - User - Gemini LLM - Cloud Screen: Setup; Function: `rendersetupform()`; Session State: `currentstep = "setup"` Screen: Quiz; Function: `renderquiz()`; Session State: `currentstep = "quiz"` Screen: Results; Function: `renderresults()`; Session State: `currentstep = "results"` Step:

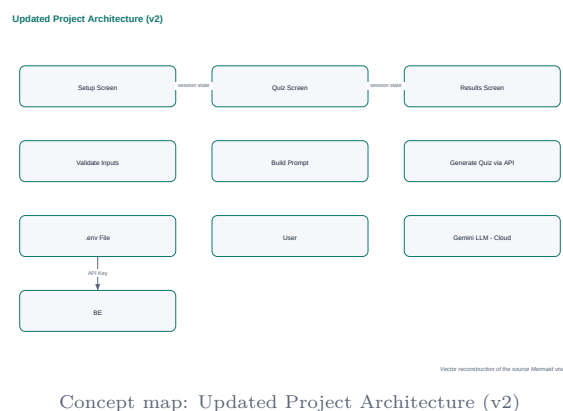
1; Function: `validateinputs()` - topic, question count, difficulty Step: 2; Function: `Format prompttemplate` with user parameters Step: 3; Function: `generatequiz()` - API call + JSON parse + schema validation All screens managed via `st.sessionstate` - persists data across Streamlit reruns Connected to backend via `python-dotenv` → `os.getenv()`. Never committed to Git Push completed project to GitHub with updated README and architecture diagram Add disclaimer: "AI-generated quizzes may contain inaccuracies" Experiment with prompt tweaks (difficulty descriptions, question styles) Adapt template to other domains (cloud certification, corporate training) Frontend teams can later replace Streamlit with production-grade React UI Code pattern: `GEMINI_APIKEY=yourkey; MODELNAME=gemini-2.5-flash` Code pattern: `quiz-genius-ai/; env; gitignore; LICENSE; README.md; requirements.txt; quizengine.py` Backend logic; `app.py` Streamlit frontend (3 screens)

Must remember:

- Architecture v2 reflects actual implementation details post-build
- Frontend: Streamlit `app.py` with 3 session-managed screens
- Backend: `quizengine.py` - validate → prompt → generate → parse
- `env` provides API key to backend; never in version control
- v2 adds file names, screen flow, validation pipeline, and secrets
- Update architecture diagrams after implementation - industry standard practice
- Project ready for GitHub portfolio with README and architecture snapshot

Example or contrast: Aspect: Frontend detail; v1 (Conceptual): "Browser UI"; v2 (Implemented): Streamlit, 3 named screens Aspect: Backend detail; v1 (Conceptual): "Python logic"; v2 (Implemented): `quizengine.py` with validation pipeline Aspect: Secrets; v1 (Conceptual): Not shown; v2 (Implemented): `env` file with API key connection

Exam trap: Not updating architecture after implementation - v1 diagrams become misleading documentation Omitting `.env` from architecture - secrets management is a core production concern



Week 13 Exam Lens

Likely questions

- Define and explain Creating the Quiz Prompt Template; include the mechanism and one limitation.
- Compare Creating the Quiz Prompt Template with Application Flow and Architecture (v1) and state when each is preferred.
- Apply Project Setup and Repository Structure to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Mistakes to avoid: Hardcoding API keys in source code - security violation; use secrets management Pasting API keys directly in notebook cells - keys get committed to Git or shared notebooks Not specifying "raw JSON" - model wraps output in markdown code fences, breaking `json.loads()`

Rapid Revision Master Sheet

Week 0: Course Introduction. Practitioner-Led NLP Education: What to Expect.

Week 1: NLP Foundations and Necessary Linguistics. NLP, NLU, and NLG: Understanding the Relationship.

Week 2: Text Preprocessing and Cleaning. Text Preprocessing and Cleaning: Foundations for NLP; Tokenisation: Splitting Text into Processable Units; Stemming: Rule-Based Morphological Reduction; Lematisation: Dictionary-Based Morphological Normalisation.

Week 3: POS Tagging and Named Entity Recognition. Structural Text Analysis: POS Tagging and Named Entity Recognition; POS Tagging with spaCy; Visualising POS Tags and Dependency Trees with spaCy; POS Tagging with NLTK; POS Tagging with Flair.

Week 4: Text Corpora and Corpus Linguistics. What Is a Text Corpus and Why It Matters; Bias and Representation in Text Corpora; Corpus Exploration Setup with NLTK; Gutenberg Corpus Analysis: Vocabulary and Word Frequency; Brown Corpus Analysis: Cross-Genre Language Comparison.

Week 5: Traditional Word Representations. One-Hot Encoding for Text; Implementing One-Hot Encoding in Python; Advantages and Limitations of One-Hot Encoding; Bag of Words (BoW); Implementing Bag of Words with scikit-learn.

Week 6: Modern Word Embeddings. Word2Vec: Learning Word Embeddings; Implementing Word2Vec with Gensim; GloVe: Global Vectors for Word Representation; Implementing GloVe with Gensim; Visualizing Word Embeddings.

Week 7: Sequential Modeling and Recurrent Networks. Sequence-to-Sequence (Seq2Seq) Modelling; Recurrent Neural Networks (RNNs); RNN Architecture Patterns: Many-to-One, One-to-Many, Many-to-Many; The Vanishing and Exploding Gradient Problem; Long Short-Term Memory (LSTM) Networks.

Week 8: The Transformer Revolution. From Sequential Models to Transformers: The Motivation; Attention Is All You Need: Origins of the Transformer; Transformer Architecture: Self-Attention, Math, and Positional Encoding; Popular Transformer Model Families: Encoders, Decoders, and Seq2Seq; Hugging Face Tokens and Model Implementation in Colab.

Week 9: Natural Language Understanding with BERT. BERT and Contextual Embeddings: Beyond Static Word Vectors; BERT Architecture and Pre-Training Objectives; Applications of BERT in Production NLP; BERT Variants: RoBERTa, ALBERT, DistilBERT, and Domain Fine-Tunes.

Week 10: Text Classification and Sentiment Analysis. Text Classification: Motivation and Real-World Applications; Sentiment Analysis: Levels, Examples, and Challenges; Sentiment Analysis with NLTK VADER: Rule-Based Lexicon Approach; Sentiment Analysis with BERT via Hugging Face Pipeline; Comparing VADER and BERT for Sentiment Analysis.

Week 11: Topic Modeling. Latent Dirichlet Allocation (LDA); Key Assumptions of LDA; Implementing Topic Modelling with LDA; Why Alternatives to LDA Are Needed; BERTopic: Semantic Topic Modelling.

Week 12: Natural Language Generation with LLMs. Why Decoder-Only Models Dominate NLG; What Is a Large Language Model (LLM)?; Why LLMs Excel at Natural Language Generation; Natural Language Generation (NLG); How LLMs Generate a Response.

Week 13: Capstone: QuizGenius AI. Creating the Quiz Prompt Template; Application Flow and Architecture (v1); Project Setup and Repository Structure; Building the Backend (quizengine.py); Building the Frontend (app.py).

Formula and Framework Recall

- Model behaviour = $f(\text{corpus statistics})$
- $\text{TF-IDF}(t, d)$ proportional to $\text{count}(t \text{ in } d) \times \log(N)/(\text{df}(t))$
- Model bias $\leftarrow$ Corpus bias
- $\text{TTR} = (|\text{types}|)/(|\text{tokens}|) = (2569)/(27333) \approx 0.094$
- $\cdot$, the one-hot vector
- Sparsity ratio $\approx 1 - (\text{words per sentence})/(|V|)$
- $\text{dog} * \text{puppy} = 0$
- $\text{TF}(x, d) = \text{count of } x \text{ in } d$

- $1 + \log(\text{TF})$
- $v_{\text{king}} = [0.2, -0.5, 0.8, \dots]$ in $\mathbb{R}^d$
- $v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}$
- $|V| \times |V|$
- $X \rightarrow H \rightarrow Y$
- Input tokens $\rightarrow$ RNN $\rightarrow$ Output tokens
- $(dL)/(dH_0) = (dL)/(dH_T) \prod_{t=1}^T \frac{dH_t}{dH_{t-1}}$
- $z_t = \sigma(W_z * [H_{t-1}, X_t])$
- $r_t = \sigma(W_r * [H_{t-1}, X_t])$
- $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}}) V$
- must be tuned by hand | Wrong
- $P(t_n | t_1, t_2, \dots, t_{n-1})$
- $P(\text{mat} | \text{context})$
- $P(\text{token} | \text{context})$

Must-Draw and Must-Compare Sheet

Use these seven structures when a question asks you to explain, compare, design, or evaluate. Draw the skeleton first, label every arrow or decision axis, then use the prose answer to explain why each element exists and what can fail.

- Preprocessing pipeline: normalize → tokenize → filter → stem/lemmatize → represent → model.
- Representation comparison: one-hot, bag-of-words, TF-IDF, static embeddings, and contextual embeddings.
- RNN family map: recurrent state, vanishing gradients, LSTM gates, and GRU simplification.
- Transformer block: tokens + positions → Q/K/V attention → residual/normalization → feed-forward.
- BERT path: masked-language pretraining → contextual encoder → task head → fine-tuning.
- Topic-model decision: LDA versus BERTopic versus GSDMM by representation, assumptions, and corpus size.
- LLM generation loop: prompt → logits → decoding rule → next token → stop condition → safety checks.

Scoring method: A diagram earns marks only when the labels are explained. After drawing, state the governing assumption, one metric or formula, one realistic example, and one limitation or trade-off. For a comparison, keep the dimensions parallel: purpose, data/input, mechanism, speed/cost, failure mode, and best-fit situation.

Final 30-Minute Answer Checklist

- Define the exact term before describing tools or examples.
- State the causal mechanism in a clear sequence; label each stage.
- Write the relevant formula, define its variables, and interpret the result.
- Compare alternatives on assumptions, cost, latency, risk, or customer value as appropriate.
- Use one realistic example and explicitly connect it to the concept.
- Close with a limitation, failure mode, ethical concern, or decision rule.
- Check that the answer addresses the command word: define, explain, compare, calculate, design, or evaluate.